Data Analysis with Python, Hands-On

Data Analysis with Python, Hands-On

Clean, Explore, Visualize and Report on Real Data, from Spreadsheets to SQL, with Six Projects, Exercises and Every Example Tested

Aadhavan Muthurengan

Data Analysis with Python, Hands-On
Clean, Explore, Visualize and Report on Real Data, from Spreadsheets to SQL, with Six Projects, Exercises and Every Example Tested

Copyright © 2026 Aadhavan Muthurengan. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain other noncommercial uses permitted by copyright law. Readers may freely use and adapt the example programs, exercises, project code and generated datasets in this book for their own work. All the code and data are available at github.com/aadhavanshines/data-analysis-with-python-hands-on under the MIT License.

Disclaimer. This book is provided for educational and informational purposes only. AI products, features, and policies change frequently; verify current details with each provider. The author makes no guarantees about results and is not liable for any losses arising from the use of this information. Nothing in this book is legal, medical, financial, or other professional advice.

Trademarks. Python and the Python logos are trademarks or registered trademarks of the Python Software Foundation. Microsoft, Excel, Windows and Visual Studio Code are trademarks of Microsoft Corporation. macOS is a trademark of Apple Inc. Linux is a trademark of Linus Torvalds. Jupyter is a trademark of NumFOCUS. GitHub is a trademark of GitHub, Inc. Kaggle is a trademark of Kaggle, Inc. PostgreSQL is a trademark of the PostgreSQL Community Association of Canada. MySQL is a trademark of Oracle Corporation. Net Promoter and NPS are trademarks of Bain & Company, Inc., Satmetrix Systems, Inc. and Fred Reichheld. SQLite is in the public domain; pandas, NumPy, matplotlib, openpyxl, pytest, Plotly, Altair, Streamlit, statsmodels, scikit-learn, DuckDB, Polars and pyarrow are projects whose names belong to their owners. All companies, people, cities, stocks and data in the example programs are fictional. Product and company names are

used solely for identification. This book is independent and is not affiliated with, sponsored by, or endorsed by the Python Software Foundation or any of these companies or projects.

Examples. All companies, people, cities, stocks, sales figures and other data in the example programs are fictional, generated by the program in Appendix F. Any resemblance to real persons or businesses is coincidental.

First edition, 2026.

To my father and my mother,
for your love, your sacrifices, and every lesson you taught me.

To my wife,
for your patience, your encouragement, and your belief in me.

And to our three children,
may you always build the things you dream of.

This book is for you.

Contents

Introduction 11

Part I: Getting Started 15

1. What Data Analysis Is 17
2. Setting Up and the Book's Data 21
3. NumPy Essentials 33

Part II: Working With Tables 37

4. Series and DataFrames 39
5. Loading and Saving Data 45
6. Selecting and Filtering 51
7. Cleaning Messy Data 57
8. Transforming Columns 63
9. Grouping and Aggregating 69
10. Combining Tables 75
11. Reshaping Data 79
12. Dates and Time Series 83
13. Working With Text 87

Part III: Seeing and Explaining 91

14. Charts With matplotlib 93
15. Choosing Honest Charts 99

16. Describing Data With Statistics 103
17. Comparing Groups and A/B Tests 109
18. SQL for Analysts 113
19. Reports, Automation and Testing Your Analysis 119
20. Using AI Assistants for Analysis, Safely 125

Part IV: Six Projects **129**

21. Project: Retail Sales Review 131
22. Project: Customer Survey 139
23. Project: Website A/B Test 145
24. Project: Stock Price Study 151
25. Project: Air Quality and Clinic Visits 155
26. Project: Household Budget 161

Part V: Going Further **167**

27. Bigger Data and Faster Code 169
28. Next Steps 175

Appendix A: Exercise Solutions **179**

Appendix B: pandas Cheat Sheet **193**

Appendix C: Common Errors and Fixes **197**

Appendix D: Glossary **199**

Appendix E: Sources for the Real-World Stories **203**

Appendix F: The Data Generator **205**

About the Author **213**

More Books by the Author 215

Introduction

Every organization runs on data now: sales, sign-ups, survey answers, sensor readings, bank transactions. Most of it sits in spreadsheets and databases, and most of it is never looked at carefully. The people who can turn a messy file into a clear answer, and then explain that answer honestly, are some of the most useful people in any team.

This book teaches you to be one of them, with Python. You'll load real-looking data, clean it, reshape it, summarize it, chart it, test whether a difference is real, and write the result up so that someone else can trust it. By the end you'll have worked through six complete projects, from a year of shop sales to an A/B test, a stock study, a public-health question and a household budget.

What Makes This Book Different

Every example is real, tested code. Each listing in this book is a file that was run on Python 3.12, 3.13, 3.14 and 3.15, with the exact versions of pandas, NumPy and matplotlib listed in Chapter 2, and the output printed beneath it is the output it produced. If you type what you see, you'll get what you see. Every chart was drawn by the code printed next to it.

One coherent set of data. Instead of a new toy table on every page, you'll work with the records of one fictional business, Larkspur Home, an online homeware shop: its orders, customers and products, plus a survey, a website experiment, stock prices, city air-quality readings and a household's bank statement. You'll get to know the data, which is how analysis works in real

life. Everything is generated by one program with a fixed random seed, printed in full in Appendix F, so every reader has exactly the same data.

The whole workflow, not just the library. pandas is the center of the book, but analysis is more than pandas: asking a sharp question, cleaning without fooling yourself, choosing an honest chart, knowing when a difference could be luck, writing SQL, testing your calculations and reporting the result. Each gets its own chapter.

Real stories, checked. Real-World Story and **Interesting Fact** boxes connect the techniques to things that actually happened: a famous economics paper with a spreadsheet error, thousands of COVID-19 cases lost to a file-size limit, genes renamed because spreadsheets kept turning them into dates. Each comes from a published source, listed in Appendix E.

Exercises with tested solutions. Most chapters end with exercises. Appendix A has a worked, tested solution for each.

Who This Book Is For

You should know basic Python: variables, lists, dictionaries, loops, functions and how to run a program. If you don't yet, *Python Programming Hands-On* teaches exactly that, and its style matches this book. You don't need any statistics or math beyond school arithmetic; the book explains every idea it uses, in plain words, with code you can run.

The book suits students, job-seekers preparing for analyst roles, spreadsheet users who've hit Excel's limits, and engineers, scientists and managers who want to answer their own questions from data.

How the Book Is Organized

- **Part I, Getting Started:** what analysis is, setting up, and the NumPy basics that pandas is built on.
- **Part II, Working With Tables:** the core of pandas: loading, selecting, cleaning, transforming, grouping, combining and reshaping data, plus dates and text.

- **Part III, Seeing and Explaining:** charts, honest chart design, descriptive statistics, comparing groups and A/B tests, SQL, reports and tests, and using AI assistants safely.
- **Part IV, Six Projects:** complete analyses from question to conclusion.
- **Part V, Going Further:** bigger data, faster code, and where to go next.
- **Appendices:** exercise solutions, a pandas cheat sheet, common errors and fixes, a glossary, sources for the real-world stories, and the program that makes the book's data.

How to Use the Code

Type the listings yourself if you can; you'll learn more than by reading. Each chapter's programs live in their own folder, next to a shared `data` folder, exactly as Chapter 2 sets up. Every program, exercise solution and dataset is also free to download from the book's code repository, **github.com/aadhavanshines/data-analysis-with-python-hands-on**, so you can check your work against the tested files; Chapter 2 shows how to get it. Change things and rerun: filter a different region, group by another column, try another chart. When something breaks, Appendix C lists the errors analysts meet most often and how to fix them.

> **Note:** All companies, people, products, cities and figures in the datasets are fictional. Real companies and events appear only in the clearly marked real-world boxes, as published by their sources.

Let's start with what data analysis actually is.

PART I

Getting Started

CHAPTER 1 · BEGINNER

What Data Analysis Is

Data analysis is the work of answering questions with data. Not "playing with data" or "finding insights", but answering specific questions, such as which products make money, whether a new web page sells more, or when pollution is worst, and then explaining the answer clearly enough that someone can act on it.

This chapter describes the workflow you'll follow throughout the book and gives you a first taste of what a few lines of Python can do.

The Workflow

Almost every analysis, large or small, follows the same six steps:

1. **Ask a question.** "How are sales?" is not a question you can answer; "Which product categories grew in the second half of the year, and which shrank?" is. A sharp question tells you what data you need and when you're done.
2. **Get the data.** From a CSV export, a spreadsheet, a database or a web service. Note where it came from and when.
3. **Clean it.** Real data has missing values, typos, duplicates, inconsistent spellings and numbers stored as text. Cleaning often takes more time than everything else together.
4. **Explore.** Count, sort, group and summarize; look at distributions and outliers; check that the data says what you think it says.

5. **Analyze and visualize.** Compare groups, follow trends over time, test whether a difference could be chance, and draw charts that make the answer visible.
6. **Communicate.** A short, honest summary with one or two charts, the numbers that matter, and what you're unsure about.

The steps loop back on each other. Exploring often reveals more cleaning to do; a chart often raises a better question. That's normal.

Four Kinds of Questions

It helps to know which kind of question you're answering:

Kind	Question	Example
Descriptive	What happened?	Revenue by month last year
Diagnostic	Why did it happen?	Garden sales fell after June because of the season
Predictive	What will happen?	Expected December orders
Prescriptive	What should we do?	Which checkout page to keep

This book concentrates on the first two and on the careful comparisons, such as A/B tests, that support the fourth. Those are the bread and butter of everyday analysis, and they're where most mistakes are made.

A First Taste

Here's what a few lines of Python with **pandas**, the library at the heart of this book, can do with a year of orders from Larkspur Home, the fictional shop whose data you'll use throughout:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["revenue"] = (
    orders["quantity"] * orders["unit_price"] * (1 - orders["discount"])
)

print(f"Orders: {len(orders):,}")
```

```
print(f"Revenue: ${orders['revenue'].sum():,.2f}")
print()
print("Revenue by category:")
print(orders.groupby("category")["revenue"].sum().sort_values().round(2))
```

```
Orders: 5,833
Revenue: $379,703.40

Revenue by category:
category
Decor        37739.0
Garden       48115.8
Kitchen      82609.2
Lighting    105073.4
Bedding     106166.0
Name: revenue, dtype: float64
```

In a dozen lines, the program read 5,833 orders, computed each order's revenue (quantity times price, less any discount), added it all up and summarized it by category. Bedding and Lighting bring in the most money; Decor the least. Doing the same in a spreadsheet takes a formula column, a pivot table and some clicking, and you'd have to repeat it all next month. The program just runs again.

Don't worry about the details yet: `read_csv`, `groupby` and the rest each get their own chapter.

Why Python?

Spreadsheets are excellent for small, one-off jobs, and you'll still use them. Python earns its place when:

- **The data is big.** A modern Excel worksheet holds at most 1,048,576 rows. pandas handles millions of rows on a laptop.
- **The work repeats.** A program runs the same steps, the same way, every month.
- **You need to show your work.** Code is a precise, reviewable record of exactly what was done to the data.
- **You want more than sums.** Statistics, joins across many files, text processing and charts are all a few lines away.

Real-World Story: In 2010, two Harvard economists published an influential paper suggesting that countries with very high public debt grow much more

slowly. In 2013, a graduate student, Thomas Herndon, tried to reproduce it and found, among other issues, that an averaging formula in the authors' spreadsheet stopped five rows short, leaving Australia, Austria, Belgium, Canada and Denmark out of one key calculation. The authors acknowledged the error. A calculation hidden in a cell is easy to get wrong and hard for anyone to check; a short program that someone else can read and rerun is much harder to get wrong unnoticed.

What You'll Need

- Basic Python: variables, lists, dictionaries, loops and functions.
- A computer with Python 3.12 or newer, and about 300 MB of disk space for the libraries.
- Curiosity about the data. The best analysts keep asking "Is that really true?" and "Compared with what?"

Key Takeaways

- Data analysis answers specific questions with data and communicates the answer honestly.
- The workflow is question, data, clean, explore, analyze and visualize, communicate, with plenty of looping back.
- Know whether you're describing, diagnosing, predicting or recommending.
- Python makes analysis repeatable, checkable and able to handle data far beyond a spreadsheet's limits.

CHAPTER 2 · BEGINNER

Setting Up and the Book's Data

In this chapter you'll install the four libraries the book uses, get the book's code and data, and take a first look at the datasets. It takes about fifteen minutes, and then everything else in the book just works.

Getting the Code and Data

Every program in this book, with its exact expected output, every exercise solution and all ten datasets are in a free repository on GitHub:

github.com/aadhavanshines/data-analysis-with-python-hands-on

Open that address in a web browser, click the green **Code** button and choose **Download ZIP**, then unzip it wherever you keep your projects. If you use Git, you can clone it instead: type `git clone`, a space, and the address above with `https://` in front, all on one line.

Either way you get a folder called `data-analysis-with-python-hands-on`. Its `code` folder holds one folder per chapter (`ch01` to `ch27`) and a shared `data` folder with the datasets, ready to use. The repository's README file repeats these steps and lists what each chapter contains.

You can also work without downloading anything: make the folders yourself and type the programs from the book, as the rest of this chapter shows. Many readers do both: they type each program and compare it with the tested file when something doesn't match.

Python and a Virtual Environment

You need Python 3.12 or newer. Check what you have from a terminal (on Windows, use `py` instead of `python3`):

```
$ python3 --version
```

Go to the folder you downloaded (or make your own, `data-analysis`) and create a **virtual environment** inside it: a private set of installed libraries for this project, so it can't clash with anything else on your computer:

```
$ cd data-analysis-with-python-hands-on
$ python3 -m venv .venv
```

Activate it each time you open a new terminal for this book. On macOS and Linux:

```
$ source .venv/bin/activate
```

On Windows:

```
$ .venv\Scripts\activate
```

Installing the Libraries

The book uses four libraries:

- **NumPy:** fast arrays of numbers; pandas is built on it.
- **pandas:** tables of data (DataFrames) and almost everything you do with them.
- **matplotlib:** charts.
- **openpyxl:** reading and writing Excel files, which pandas uses behind the scenes.

Install the versions the book was tested with, so your output matches the book exactly. In the downloaded folder, one command installs them all from the repository's `requirements.txt` file:

```
$ python -m pip install -r requirements.txt
```

If you made your own folder, name the versions instead:

```
$ python -m pip install numpy==2.5.4 pandas==3.0.6
  matplotlib==3.11.2
$ python -m pip install openpyxl==3.1.5 pytest==9.1.1
```

(pytest, for testing your analysis code, appears in Chapter 19.) Newer versions will almost always work too; occasionally a number in a printed table may differ in the last digit, or a message may be worded differently.

Check the installation:

```
import matplotlib
import numpy
import openpyxl
import pandas

for module in (numpy, pandas, matplotlib, openpyxl):
    print(f"{module.__name__:<11} {module.__version__}")
```

```
numpy       2.5.4
pandas      3.0.6
matplotlib  3.11.2
openpyxl    3.1.5
```

> **Tip:** Prefer **Jupyter notebooks**, where code, output and charts sit together on one page? Everything in this book works in a notebook too, and the end of this chapter shows how to use one.

The Folder Layout

Each chapter's programs go in their own folder, next to a shared `data` folder. In the download, these folders are inside `code`; if you type the programs yourself, create the same layout:

```
data-analysis/
    data/
        make_data.py
        orders.csv
        ...
    ch01/
        first_look.py
    ch02/
        check_setup.py
    ...
```

The programs read data with paths like `"../data/orders.csv"`: `..` means "the folder above this one", so a program in `ch05` finds the shared data folder. Run each program from inside its own chapter folder:

```
$ cd ch01
$ python first_look.py
```

In the download, each program's expected output is stored next to it, in a file with the same name ending in `.out` (`first_look.out`), so you can compare your run with the book's at any time.

Making the Data

All the datasets in this book are fictional, and one program makes all of them: `make_data.py`, printed in full in Appendix F. It uses NumPy's random number generator with a fixed **seed** (2026), so it makes exactly the same "random" data on every computer, every time. That's why your results will match the book's to the last digit.

The download already contains the datasets, made by this program, so you can skip this step. If you're typing the programs yourself, type or copy `make_data.py` into your `data` folder and run it there:

```
$ cd data
$ python make_data.py
```

```
orders.csv          5,833 rows
customers.csv       1,200 rows
products.csv           15 rows
orders_raw.csv        303 rows
survey.csv, sessions.csv, prices.csv, air_quality.csv,
clinic_visits.csv and budget.csv written too
```

The data was built with patterns on purpose: busy Novembers and Decembers, garden products that sell in spring, bigger orders when there's a discount, a checkout page that converts a little better, pollution that affects clinic visits two days later. Practicing on data with known patterns is the best way to learn: when your analysis finds them, you know your method works.

Meet the Data

This program lists every file with its size and columns:

```
from pathlib import Path

for path in sorted(Path("../data").glob("*.csv")):
    with path.open() as f:
        header = f.readline().strip()
        rows = sum(1 for _ in f)
    print(f"{path.name:<18} {rows:>6,} rows  {header[:44]}")
```

```
air_quality.csv     1,095 rows  date,city,pm25
budget.csv            231 rows  date,description,amount
clinic_visits.csv   1,095 rows  date,city,visits
customers.csv       1,200 rows  customer_id,signup_date,region,segment,birth
orders.csv          5,833 rows  order_id,order_date,customer_id,region,chann
orders_raw.csv        303 rows  order_id,order_date,customer_id,region,chann
prices.csv            523 rows  date,ALDR,BRKN,CDLX,INDEX
products.csv           15 rows  product,category,list_price,unit_cost
sessions.csv       20,000 rows  session,date,variant,device,pages,converted,
survey.csv            800 rows  respondent,date,age_group,plan,satisfaction,
```

Here's what each one holds.

File	One row per	Used in
`orders.csv`	Order at Larkspur Home in 2025	Most chapters; Project 1
`customers.csv`	Customer: signup date, region, segment, birth year	Chapters 8, 10, 18
`products.csv`	Product: category, list price, unit cost	Chapters 10, 18, 21
`orders_raw.csv`	A messy export of 300 orders	Chapter 7
`survey.csv`	Survey response: plan, ratings, comment	Chapters 13, 22
`sessions.csv`	Website visit in a checkout experiment	Chapters 17, 23
`prices.csv`	Trading day: prices of three companies and an index	Chapter 24
`air_quality.csv`	Day and city: fine-particle pollution	Chapters 14, 16, 25
`clinic_visits.csv`	Day and city: clinic visits for breathing problems	Chapters 14, 16, 25
`budget.csv`	Bank transaction of one household	Chapters 13, 26

CSV (comma-separated values) is the most common way to share tables: plain text, one row per line, columns separated by commas, with the column names on the first line. Every spreadsheet program and database can read and write it.

Working in Jupyter Notebooks

So far you've run programs from a terminal. Many analysts prefer a **Jupyter notebook**: a page in your web browser made of **cells**. A code cell holds a few lines of Python; when you run it, its output appears right underneath, tables as neat tables and charts as pictures. Text cells hold headings and notes. You build up an analysis one small step at a time and see each result as you go.

Jupyter came with `requirements.txt` if you downloaded the book's code. If you made your own folder, install it too:

```
$ python -m pip install notebook==7.6.3
```

With your virtual environment active, start Jupyter from the book's folder:

```
$ jupyter notebook
```

Your web browser opens and lists the files in that folder. (The terminal must stay open while you work; to stop Jupyter afterwards, click in the terminal and press Ctrl+C twice.) Open `code`, then `ch02`, then `first_notebook.ipynb`. Notebook files end in `.ipynb`.

Your First Notebook

The notebook starts with a text cell, a heading and one line of notes, and then four code cells. Click in the first code cell and press **Shift+Enter**: Jupyter runs the cell and moves to the next one. Press Shift+Enter four times to run them all.

The first cell loads the orders:

```
import pandas as pd

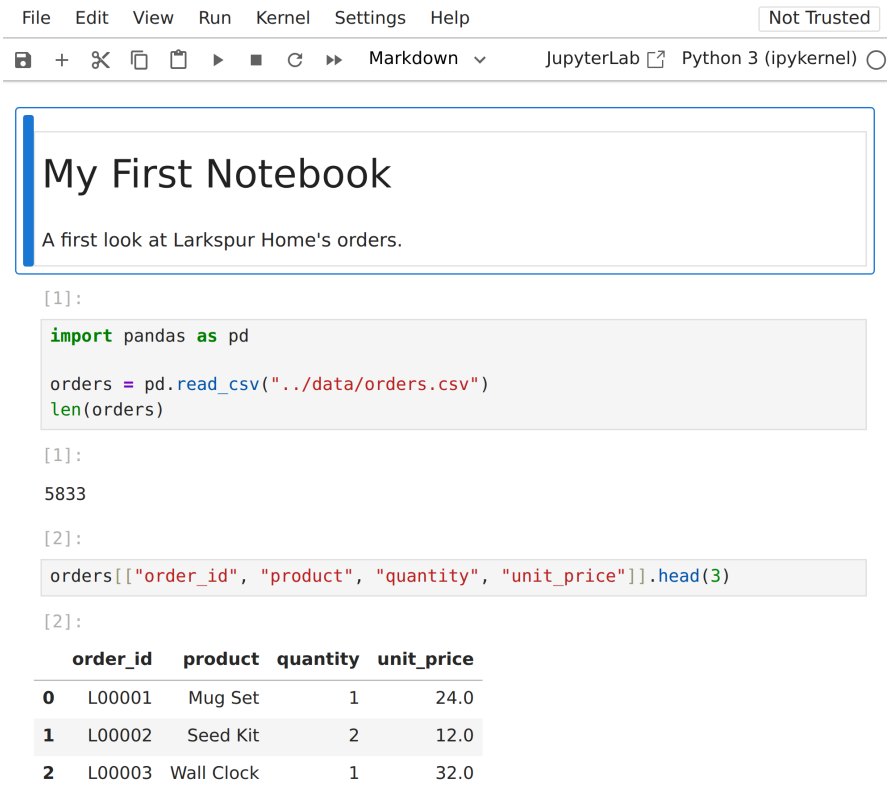
orders = pd.read_csv("../data/orders.csv")
len(orders)
```

There's no `print` here, yet the notebook shows 5833 under the cell. That's the first thing to know about notebooks: **the value of a cell's last line is shown automatically.** The second cell shows the first three orders, four of their columns, as a table (double brackets pick columns, which Chapter 6 explains):

```
orders[["order_id", "product", "quantity", "unit_price"]].head(3)
```

The number in brackets beside each cell, `[1]`, `[2]`, counts the order in which cells were run. The third cell adds a revenue column and totals it by category:

```
orders["revenue"] = orders["quantity"] * orders["unit_price"]
by_category = orders.groupby("category")["revenue"].sum()
by_category.sort_values()
```



The top of the notebook in Jupyter: the menu and toolbar, the "My First Notebook" heading, the first cell with its result 5833, and the second cell with a three-row table of order ID, product, quantity and unit price.

Variables live on between cells: the third cell uses `orders` from the first, and the fourth uses `by_category` from the third. The fourth cell draws a chart, which appears under the cell, with no `savefig` needed:

```
ax = by_category.sort_values().plot.barh(title="Revenue by category")
```

The chart is assigned to `ax` only so that the cell's last line has no value to show; without `ax =`, Jupyter would print a line describing the chart object above the picture.

```
[3]:
orders["revenue"] = orders["quantity"] * orders["unit_price"]
by_category = orders.groupby("category")["revenue"].sum()
by_category.sort_values()
```

```
[3]:
category
Decor        39707.0
Garden       50515.0
Kitchen      86295.0
Lighting    109942.0
Bedding     110935.0
Name: revenue, dtype: float64
```

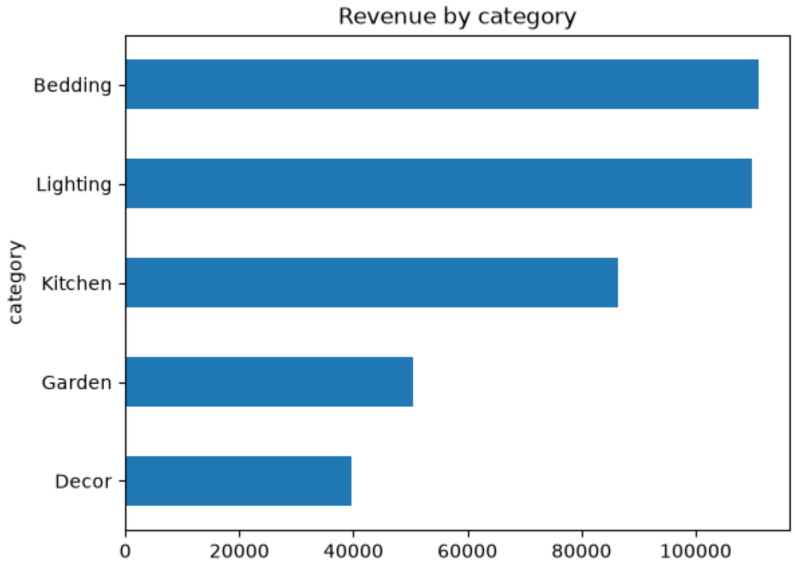
The third cell and its result: revenue by category from Decor, 39,707, to Bedding, 110,935.

> **Common Mistake:** Because variables live on, the order you run cells in matters. Run the third cell before the first, and you get `NameError: name 'orders' is not defined`. Change a cell near the top, and the cells below still show results from the old version until you run them again. When results look odd, choose **Kernel > Restart Kernel and Run All Cells**: Jupyter forgets everything and runs the notebook from top to bottom, which is exactly how anyone else will run it.

A few more things worth knowing:

- **Saving:** Jupyter saves automatically every few minutes, and Ctrl+S (Cmd+S on a Mac) saves at once. The `.ipynb` file keeps the outputs too, so you can reopen it later and see your results without running anything.
- **New notebooks:** choose **File > New > Notebook** and pick the Python 3 kernel. Save it in a chapter folder, so paths like `"../data/orders.csv"` work.
- **Text cells:** change a cell's type from Code to Markdown in the toolbar and write notes, with **#** for a heading. A notebook with a few sentences between the steps is a report someone else can follow.
- **"Not Trusted":** Jupyter marks a notebook it didn't run on your computer, like the ones you downloaded, as not trusted, and holds back anything interactive in its saved outputs until you run it yourself. Running the cells fixes it.

```
[4]:
ax = by_category.sort_values().plot.barh(title="Revenue by category")
```



The fourth cell and its chart: horizontal bars of revenue by category, Bedding and Lighting longest, Decor shortest.

A Notebook for Every Chapter

Each chapter folder in the download also has a notebook with all of that chapter's programs, in the book's order: `code/ch09/ch09.ipynb` for Chapter 9, and so on. Each code cell is one program, under the heading of the section it comes from, and the saved notebooks show every output and chart, so you can browse them on GitHub without running anything. The book's tests run every chapter notebook on Python 3.12 to 3.15 and check that each cell prints exactly what the book prints. Choose **Run > Run All Cells** to run a whole chapter.

Notebooks are best for exploring: trying an idea, looking at a table, adjusting a chart. Plain `.py` files are best for analyses you rerun, test and schedule, which is why this book's programs are files (Chapter 19 shows why). Many analysts explore in a notebook and then move the finished steps into a `.py` file. In fact, `first_notebook.py`, next to the notebook, is the same notebook

written as a plain file: each `# %%` line starts a new cell, and code editors such as Visual Studio Code can run those cells one at a time, notebook style.

Key Takeaways

- Every program, its expected output, the exercise solutions and the datasets are free at github.com/aadhavanshines/data-analysis-with-python-hands-on.
- Use a virtual environment per project, and install the exact library versions when you need reproducible results.
- The book's four libraries: NumPy (arrays), pandas (tables), matplotlib (charts) and openpyxl (Excel files).
- Keep each chapter's programs in their own folder beside a shared `data` folder.
- A fixed random seed makes "random" data identical everywhere, so your results match the book's.
- A Jupyter notebook runs code in cells and shows the last line's value, tables and charts under each cell; restart and run all cells before trusting the results.

CHAPTER 3 · BEGINNER

NumPy Essentials

pandas is built on **NumPy**, the library that gives Python fast arrays of numbers. You'll rarely use NumPy on its own in this book, but three of its ideas run through everything pandas does: operating on whole arrays at once, selecting with true/false masks, and **broadcasting**. This chapter covers them, plus NumPy's random number generator, which you'll use for simulations and statistics.

Arrays and Vectorized Operations

A NumPy **array** is like a list, but every item has the same type, and arithmetic works on all the items at once:

```
import numpy as np

prices = np.array([45.0, 60.0, 24.0, 70.0, 35.0])
quantities = np.array([2, 1, 4, 1, 3])

print(prices.dtype, prices.shape)
print(prices * quantities)  # element by element
print((prices * quantities).sum())  # total
print(prices[prices > 40])  # a boolean mask selects items
print(np.round(prices * 1.08, 2))  # every price plus 8% tax

grid = np.arange(12).reshape(3, 4)
print(grid)
print(grid.sum(axis=0))  # down each column
print(grid.sum(axis=1))  # across each row
```

```
float64 (5,)
[ 90.  60.  96.  70. 105.]
421.0
[45. 60. 70.]
[48.6  64.8  25.92 75.6  37.8 ]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[12 15 18 21]
[ 6 22 38]
```

Line by line:

- `prices.dtype` is `float64`: 64-bit decimal numbers. `prices.shape` is `(5,)`: one dimension with five items.
- `prices * quantities` multiplies the arrays item by item, with no loop. This is called a **vectorized** operation. It's shorter to write and runs in fast compiled code, which matters when an array has millions of items.
- `prices > 40` makes an array of `True`/`False` values, a **boolean mask**, and `prices[mask]` keeps the items where the mask is `True`. You'll use exactly the same idea to filter rows in pandas.
- `reshape(3, 4)` turns 12 numbers into a grid of 3 rows and 4 columns. With `axis=0`, `sum` adds down each column; with `axis=1`, across each row.

> **Common Mistake:** Mixing up the axes. `axis=0` means "collapse the rows", so you get one result per column. If you get the wrong shape of answer, try the other axis.

Broadcasting

When you combine arrays of different shapes, NumPy stretches the smaller one to fit, if it can. That's **broadcasting**:

```
import numpy as np

# Monthly sales (rows) for three stores (columns)
sales = np.array([[120, 95, 60], [130, 90, 75], [150, 110, 80]])
targets = np.array([125, 100, 70])  # one target per store

print(sales - targets)  # the 1-D targets stretch to every row
print(sales >= targets)
print((sales >= targets).mean(axis=0).round(2))  # months on target
print((sales / sales.sum(axis=1, keepdims=True)).round(2))  # shares
```

```
[[ -5  -5 -10]
 [  5 -10   5]
 [ 25  10  10]]
[[False False False]
 [ True False  True]
 [ True  True  True]]
[0.67 0.33 0.67]
[[0.44 0.35 0.22]
 [0.44 0.31 0.25]
 [0.44 0.32 0.24]]
```

`sales` has 3 rows and 3 columns; `targets` has 3 items, one per store. `sales - targets` subtracts the targets from *every* row, giving each store's distance from its target each month. `sales >= targets` gives a mask of on-target months, and its mean down each column is the share of months on target: True counts as 1 and False as 0. Finally, dividing each row by its own total (`keepdims=True` keeps the totals as a column, so they line up with the rows) gives each store's share of each month's sales.

Random Numbers and Simple Statistics

Simulations and many statistical methods need random numbers. Always create a generator with a **seed**, so your results can be repeated:

```
import numpy as np

rng = np.random.default_rng(42)  # a fixed seed: the same numbers each run
heights = rng.normal(loc=170, scale=8, size=10_000)

print(f"mean   {heights.mean():.1f}")
print(f"std    {heights.std():.1f}")
print(f"median {np.median(heights):.1f}")
print("5th and 95th percentiles:", np.percentile(heights, [5, 95]).round(1))
print("share above 185 cm:", (heights > 185).mean())

dice = rng.integers(1, 7, size=(100_000, 2)).sum(axis=1)
counts = np.bincount(dice)[2:]
print("two dice, sums 2 to 12:")
print(counts)
print("most common sum:", counts.argmax() + 2)
```

```
mean   169.9
std    8.1
median 169.9
5th and 95th percentiles: [156.6 183.1]
share above 185 cm: 0.0308
two dice, sums 2 to 12:
[ 2747  5475  8240 11059 13881 16845 13884 11087  8500  5523  2759]
```

```
most common sum: 7
```

The generator draws 10,000 heights from a **normal distribution**, the bell curve, with mean 170 and standard deviation 8. The sample's mean and standard deviation come out very close to those values. The 5th and 95th **percentiles** say that 90% of heights fall between about 157 and 183 cm. And because a mask is True or False, `(heights > 185).mean()` is the *share* of heights above 185: about 3%.

The second part rolls two dice 100,000 times. `np.bincount` counts how often each total appears; the counts rise to a peak at 7 and fall again, exactly as probability says they should. A quick simulation like this is often the easiest way to check your understanding of a probability question.

Interesting Fact: NumPy's random generator, `default_rng`, uses an algorithm called PCG64 by default. With the same seed and the same NumPy version, it produces the same numbers on every computer. That's why this book's data, made from seed 2026, is identical for every reader.

Exercises

1. **Temperatures.** Make a 3 by 4 array of temperatures for three cities over four days. Print each city's average, the warmest reading on each day, and how many readings are above 15.
2. **Coin flips.** Simulate 10,000 runs of 10 coin flips with a seeded generator. In what share of runs do 8 or more heads come up? (The exact answer is 56/1024, about 0.055.)

Key Takeaways

- NumPy arrays hold many numbers of one type, and arithmetic applies to every item at once.
- A boolean mask selects items; its mean is the share of True values.
- `axis=0` works down columns, `axis=1` across rows.
- Broadcasting stretches a smaller array to match a bigger one.
- Create random generators with a seed so results can be repeated.

PART II

Working With Tables

CHAPTER 4 · BEGINNER

Series and DataFrames

pandas has two main objects. A **Series** is one column of data with labels; a **DataFrame** is a table made of Series that share the same row labels. Almost everything in Part II is something you do to one of these two.

Interesting Fact: pandas was started in 2008 by Wes McKinney, then working at the investment firm AQR Capital Management, because he wanted tools for financial data that Python didn't have. AQR allowed him to release it as open source in 2009. The name comes from "panel data", an economics term for tables of observations over time.

Series

```
import pandas as pd

stock = pd.Series(
    [12, 0, 7, 25], index=["mugs", "lamps", "vases", "frames"]
)
print(stock)
print()
print(stock["vases"])  # by label
print(stock.iloc[0])  # by position
print(stock[stock > 5])  # a mask keeps matching items
print(stock.sum(), stock.mean(), stock.idxmax())
```

```
mugs      12
lamps      0
vases      7
frames    25
dtype: int64

7
```

```
12
mugs      12
vases      7
frames    25
dtype: int64
44 11.0 frames
```

A Series pairs values with an **index**: here the product names. You can select by label (`stock["vases"]`) or by position (`stock.iloc[0]`), filter with a mask exactly as with NumPy, and summarize: the total is 44 items, the mean 11, and `idxmax()` gives the *label* of the largest value.

The last line of the printed Series, `dtype: int64`, is its data type: 64-bit whole numbers.

DataFrames

Most data arrives as a table, and `read_csv` turns a CSV file into a DataFrame:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])

print(orders.shape)
print(list(orders.columns[:5]))
print(list(orders.columns[5:]))
print()
print(orders.head(3)[["order_id", "order_date", "product", "quantity"]])
print()
print(orders.dtypes)
```

```
(5833, 10)
['order_id', 'order_date', 'customer_id', 'region', 'channel']
['category', 'product', 'quantity', 'unit_price', 'discount']

  order_id order_date     product  quantity
0   L00001 2025-01-01     Mug Set         1
1   L00002 2025-01-01    Seed Kit         2
2   L00003 2025-01-01  Wall Clock         1

order_id                  str
order_date     datetime64[us]
customer_id               str
region                    str
channel                   str
category                  str
product                   str
quantity                int64
unit_price            float64
discount              float64
dtype: object
```

- `shape` is (rows, columns): 5,833 orders with 10 columns each.
- `columns` lists the column names. Selecting a list of columns, `orders[[...]]`, gives a smaller DataFrame, and `head(3)` its first three rows.
- The numbers 0, 1, 2 on the left are the **index**. `read_csv` numbers the rows from 0 unless you choose a column to use instead (Chapter 6).
- `dtypes` shows each column's type. Whole numbers are `int64`, decimals `float64`. Text columns show as `str`, the string type that pandas 3 uses by default. `order_date` is `datetime64[us]`, a date and time stored to the microsecond, because the program asked `read_csv` to parse it as a date with `parse_dates`.

> **Note:** Older pandas versions show text columns as `object`. If you see `object` where the book shows `str`, you have a pandas version before 3.0. Almost everything works the same; Chapter 2 shows how to install the book's version.

A Quick Summary of Any Table

Three methods give a fast overview of a new dataset:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")

print(orders[["quantity", "unit_price", "discount"]].describe().round(2))
print()
print(orders["channel"].value_counts())
print()
print(orders["category"].value_counts(normalize=True).round(3))
```

```
       quantity  unit_price  discount
count   5833.00     5833.00   5833.00
mean       1.61       42.36      0.04
std        0.86       26.11      0.07
min        1.00       12.00      0.00
25%        1.00       24.00      0.00
50%        1.00       35.00      0.00
75%        2.00       55.00      0.10
max        7.00      120.00      0.20

channel
web            2882
app            1767
```

```
marketplace    1184
Name: count, dtype: int64

category
Bedding     0.236
Kitchen     0.217
Lighting    0.191
Garden      0.185
Decor       0.171
Name: proportion, dtype: float64
```

`describe()` summarizes numeric columns: the count of values, mean, standard deviation, minimum, quartiles and maximum. Already you learn a lot: most orders are for one item (the median quantity is 1), the largest is 7; prices run from $12 to $120; discounts are at most 20%.

`value_counts()` counts each distinct value, largest first. Half the orders come through the website. With `normalize=True` it gives shares instead of counts: Bedding is the most common category at 23.6% of orders.

> **Tip:** Make `shape`, `head()`, `dtypes`, `describe()` and `value_counts()` the first things you run on any new dataset. Five lines, and you know its size, its columns, its types, its ranges and its categories.

Exercises

1. **Customers.** Load `customers.csv`. How many customers are there, how many in each segment, and what are the earliest and latest birth years?
2. **Margins.** Load `products.csv` and add a `margin` column: `1 - unit_cost / list_price`. Which three products have the highest margin?

Key Takeaways

- A Series is a labeled column; a DataFrame is a table of Series sharing one index.
- Select by label with `[]` or `loc`, by position with `iloc`.
- `read_csv` reads a CSV file into a DataFrame; `parse_dates` turns date columns into real dates.

- Start every analysis with `shape`, `head()`, `dtypes`, `describe()` and `value_counts()`.

CHAPTER 5 · BEGINNER

Loading and Saving Data

Data comes in many formats: CSV files, Excel workbooks, JSON from web services, and tables in databases. pandas reads and writes all of them with a family of `read_...` functions and `to_...` methods. This chapter covers the four you'll meet most often, and the options that save you from trouble later.

Reading CSV Files Carefully

`read_csv` has dozens of options. Four of them pay for themselves on almost every file:

```
import pandas as pd

orders = pd.read_csv(
    "../data/orders.csv",
    usecols=["order_id", "order_date", "region", "category", "quantity"],
    parse_dates=["order_date"],
    dtype={"region": "category", "category": "category"},
)
orders.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5833 entries, 0 to 5832
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   order_id    5833 non-null   str
 1   order_date  5833 non-null   datetime64[us]
 2   region      5833 non-null   category
 3   category    5833 non-null   category
 4   quantity    5833 non-null   int64
dtypes: category(2), datetime64[us](1), int64(1), str(1)
memory usage: 148.6 KB
```

- `usecols` reads only the columns you need, which saves memory and time on big files.
- `parse_dates` turns text dates into real dates, so you can sort, filter and group by them correctly.
- `dtype` sets column types up front. Here `region` and `category` become `category` columns: pandas stores each distinct value once and keeps small codes for the rows, which uses much less memory for columns with few distinct values (Chapter 27).
- `info()` prints the result: columns, non-missing counts, types and memory used. The 5,833 non-null counts tell you there are no missing values in these columns.

Other options you'll reach for: `sep=";"` for semicolon-separated files (common in Europe, where the comma is the decimal separator), `decimal=","`, `encoding="latin-1"` for older files that aren't UTF-8, `na_values=["n/a", "-"]` for files that mark missing values in their own way, and `nrows=1000` to read just the start of a huge file while you explore it.

Excel Workbooks

pandas reads and writes Excel files through openpyxl. One workbook can hold several sheets:

```
import pandas as pd

products = pd.read_csv("../data/products.csv")
customers = pd.read_csv("../data/customers.csv")

# One workbook, two sheets
with pd.ExcelWriter("shop.xlsx") as writer:
    products.to_excel(writer, sheet_name="Products", index=False)
    customers.to_excel(writer, sheet_name="Customers", index=False)

sheets = pd.read_excel("shop.xlsx", sheet_name=None)  # None: every sheet
for name, table in sheets.items():
    print(f"{name}: {table.shape[0]} rows, {table.shape[1]} columns")
print(sheets["Products"].head(3))
```

```
Products: 15 rows, 4 columns
Customers: 1200 rows, 5 columns
         product category  list_price  unit_cost
0     Chef Knife  Kitchen          45      21.46
1  Cast Iron Pan  Kitchen          60      32.76
2        Mug Set  Kitchen          24      12.48
```

`ExcelWriter` writes several DataFrames into one workbook, one per sheet. `read_excel` with `sheet_name=None` reads every sheet into a dictionary of DataFrames, keyed by sheet name.

Notice one change: `list_price` came back as whole numbers (45, 60, 24) instead of 45.0, 60.0 and 24.0. Excel stores numbers without a separate integer type, so whole-valued prices look like integers when read back. The values are the same, but it's a reminder that every trip through another format can change types. Check `dtypes` after reading.

> **Warning:** Excel is not a database. A worksheet holds at most 1,048,576 rows and 16,384 columns (the older `.xls` format only 65,536 rows), and when data is pasted into a full sheet, rows beyond the limit can be silently cut off. Keep large data in CSV files or a database, and use Excel for the summary, not the raw data.

Real-World Story: In October 2020, Public Health England reported that 15,841 positive COVID-19 test results from 25 September to 2 October had been left out of the daily case counts, delaying contact tracing for thousands of people. According to press reports at the time, the results were being combined in a spreadsheet whose size limit was exceeded, so the newest results were dropped without an error. The lesson for analysts: check row counts at every step. If 50,000 rows go in, make sure 50,000 come out.

JSON

Web services usually return **JSON**: nested lists and dictionaries in text form.

```
import json

import pandas as pd

products = pd.read_csv("../data/products.csv")
products.head(2).to_json("products.json", orient="records", indent=2)
print(open("products.json").read())
```

```
# Nested JSON, as many web services return it, flattened into columns
reply = json.loads("""
[{"id": 1, "customer": {"name": "Asha", "city": "Pune"}, "total": 42.5},
 {"id": 2, "customer": {"name": "Ben", "city": "Leeds"}, "total": 18.0}]
""")
print(pd.json_normalize(reply))
```

```
[
  {
    "product":"Chef Knife",
    "category":"Kitchen",
    "list_price":45.0,
    "unit_cost":21.46
  },
  {
    "product":"Cast Iron Pan",
    "category":"Kitchen",
    "list_price":60.0,
    "unit_cost":32.76
  }
]
   id  total customer.name customer.city
0   1   42.5          Asha          Pune
1   2   18.0           Ben         Leeds
```

`to_json(orient="records")` writes one JSON object per row, a format most web tools accept. When JSON is nested, as in the two-order reply here, `json_normalize` flattens it into columns, joining nested names with dots: `customer.name`, `customer.city`.

SQL Databases

Much of the world's data lives in databases, and pandas talks to them directly. Python includes **SQLite**, a database stored in a single file, so you can try this with nothing to install:

```
import sqlite3

import pandas as pd

orders = pd.read_csv("../data/orders.csv")
with sqlite3.connect("shop.db") as conn:
    orders.to_sql("orders", conn, if_exists="replace", index=False)
    top = pd.read_sql(
        """SELECT region, COUNT(*) AS orders, SUM(quantity) AS units
           FROM orders GROUP BY region ORDER BY units DESC""",
        conn,
    )
print(top)
```

```
  region  orders  units
0  North    1768   2797
1   East    1560   2497
2  South    1300   2123
3   West    1205   1970
```

`to_sql` writes a DataFrame into a database table (`if_exists="replace"` overwrites any older copy), and `read_sql` runs a query and returns the result as a DataFrame. The North region has the most orders and items. For other databases, such as PostgreSQL or MySQL, you connect with that database's Python driver and use the same `read_sql`. Chapter 18 teaches the SQL itself.

Exercises

1. **Just two columns.** Read only `plan` and `satisfaction` from `survey.csv`, with `plan` as a category. Print the types and the list of categories.
2. **One file per region.** Write `orders.csv` out as four files, `orders_east.csv` and so on, one per region, and print how many rows each holds. (Hint: loop over `orders.groupby("region")`, which gives each region's name and its rows.)

Key Takeaways

- `read_csv` options `usecols`, `parse_dates`, `dtype` and `na_values` prevent most loading problems.
- `read_excel` and `ExcelWriter` handle workbooks with several sheets; check types after reading.
- Excel has hard row limits; check row counts at every step of a pipeline.
- `json_normalize` flattens nested JSON; `read_sql` and `to_sql` connect pandas to databases.

CHAPTER 6 · BEGINNER

Selecting and Filtering

Most questions are about *some* of the data: one region, last month, orders over $100. This chapter covers the ways to pick out columns and rows, and to sort them, so you can zoom in on exactly the part you need.

Columns, Labels and Positions

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", index_col="order_id")

print(orders["product"].head(3))  # one column: a Series
print(orders[["product", "quantity"]].head(3))  # a list: a DataFrame
print(orders.loc["L00010", "product"])  # row and column labels
print(orders.loc["L00010":"L00012", ["region", "quantity"]])  # label slice
print(orders.iloc[0, :3].tolist())  # positions
```

```
order_id
L00001       Mug Set
L00002      Seed Kit
L00003    Wall Clock
Name: product, dtype: str
             product  quantity
order_id
L00001       Mug Set         1
L00002      Seed Kit         2
L00003    Wall Clock         1
Pillow Pair
         region  quantity
order_id
L00010    North         2
L00011    North         3
L00012     West         2
['2025-01-01', 'C0503', 'South']
```

This time `read_csv` used `index_col="order_id"`, so the order IDs became the index, the row labels. That makes looking up an order by ID natural.

- `orders["product"]`, one name in brackets, gives a single column as a Series.
- `orders[["product", "quantity"]]`, a *list* of names, gives a DataFrame with those columns. The double brackets are easy to miss: the outer pair selects, the inner pair makes a list.
- `loc[row, column]` selects by **labels**. `orders.loc["L00010", "product"]` is the product in order L00010.
- With labels, slices include both ends: `loc["L00010":"L00012"]` gives three rows. That's different from ordinary Python slices.
- `iloc[row, column]` selects by **position**, counting from 0, and its slices exclude the end, like normal Python: `iloc[0, :3]` is the first row's first three values.

> **Tip:** Use `loc` when you know the names, and `iloc` when you know the positions. Avoid `orders["L00010"]` style shortcuts for rows: plain brackets select columns, and mixing the two is a common source of confusing errors.

Filtering Rows With Conditions

Filtering works exactly like a NumPy mask: a condition on a column gives True or False for every row, and the DataFrame keeps the True rows.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])

big = orders[orders["quantity"] >= 5]
print("orders of 5 or more items:", len(big))

west_lamps = orders[
    (orders["region"] == "West") & (orders["category"] == "Lighting")
]
print("West lighting orders:", len(west_lamps))

promo = orders[orders["discount"].isin([0.1, 0.2])]
print("discounted orders:", len(promo))

december = orders[orders["order_date"].between("2025-12-01", "2025-12-31")]
print("December orders:", len(december))
```

```
print(
    orders.query("channel == 'app' and quantity > 4")[
        ["order_id", "product", "quantity"]
    ].head(3)
)
```

```
orders of 5 or more items: 56
West lighting orders: 239
discounted orders: 1749
December orders: 879
    order_id        product  quantity
650   L00651  Picture Frame         5
847   L00848     Chef Knife         5
966   L00967    Duvet Cover         6
```

- `orders["quantity"] >= 5` keeps 56 large orders.
- Combine conditions with `&` (and), `|` (or) and `~` (not), and put **each condition in parentheses**. There were 239 lighting orders from the West.
- `isin([...])` tests membership in a list: 1,749 orders had a 10% or 20% discount.
- `between(a, b)` tests a range, including both ends: 879 December orders.
- `query()` takes the condition as a string, which reads nicely for longer filters. Names inside the string are column names.

> **Common Mistake:** Writing `orders[orders["region"] == "West" and orders["category"] == "Lighting"]`. Python's `and` can't combine two whole columns of True/False values, and pandas raises "The truth value of a Series is ambiguous". Use `&` instead, with parentheses around each condition.

Sorting and Top Values

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["revenue"] = orders["quantity"] * orders["unit_price"]

cols = ["order_id", "product", "quantity", "revenue"]
# Ties at the top are broken by order_id, so the result never varies
top = orders.sort_values(["revenue", "order_id"], ascending=[False, True])
print(top[cols].head(3))
print(orders.nlargest(3, "quantity")[cols])
print(
    orders.sort_values(["region", "revenue"], ascending=[True, False])
    .groupby("region")
```

```
    .head(1)[["region"] + cols[1:]]
)
```

```
     order_id     product  quantity  revenue
1801   L01802  Floor Lamp         5    600.0
3058   L03059  Floor Lamp         5    600.0
3474   L03475  Floor Lamp         5    600.0
     order_id      product  quantity  revenue
2753   L02754      Planter         7    154.0
966    L00967  Duvet Cover         6    420.0
1236   L01237      Mug Set         6    144.0
     region     product  quantity  revenue
268    East  Floor Lamp         4    480.0
1801  North  Floor Lamp         5    600.0
3058  South  Floor Lamp         5    600.0
3474   West  Floor Lamp         5    600.0
```

`sort_values` sorts by one or more columns, and `ascending` says which way for each: here revenue largest first, then order ID smallest first. `nlargest(3, "quantity")` is a shortcut for "sort and take the top 3", and it's faster on big tables. The third example finds the biggest order in each region: sort by region and then by revenue (largest first), then `groupby("region").head(1)` keeps the first row of each region. You'll see `groupby` properly in Chapter 9.

Several orders tie at $600 (five floor lamps each). When values tie, a sort by revenue alone can put the tied rows in a different order on a different computer, because the fastest sorting method depends on the processor. That's why the first sort adds `order_id` as a second column: it breaks the ties, so everyone gets the same three rows. When the order of tied rows matters, always add a tie-breaker.

Exercises

1. **Full-price lamps.** How many orders were for a product with "Lamp" in its name and no discount? How many of each lamp? (Hint: `str.contains`, Chapter 13.)
2. **New business customers.** From `customers.csv` (parse `signup_date` as a date), find Business customers who signed up in 2025 or later. How many are there, and who were the first three?

Key Takeaways

- One column name gives a Series; a list of names gives a DataFrame.
- `loc` selects by label (slices include the end); `iloc` by position (slices exclude it).
- Filter with conditions combined by `&`, `|` and `~`, each in parentheses; `isin`, `between` and `query` keep filters readable.
- `sort_values` and `nlargest` find the top rows; add a tie-breaker column when values can tie.

CHAPTER 7 · BEGINNER

Cleaning Messy Data

Real data is messy. People type region names three different ways, systems export prices with currency symbols, dates arrive in two formats in the same column, and the same row appears twice. If you analyze messy data as it is, you get wrong answers without any error message. Cleaning is where careful analysts earn their reputation.

This chapter cleans `orders_raw.csv`, an export of 300 orders that has every common problem, then shows how to check data automatically so problems can't sneak back in.

Look Before You Clean

First, find out what's wrong:

```
import pandas as pd

raw = pd.read_csv("../data/orders_raw.csv")

print(raw.dtypes)
print()
print("missing values per column:")
print(raw.isna().sum()[raw.isna().sum() > 0])
print()
print("region spellings:")
print(raw["region"].value_counts().sort_index())
print("duplicate rows:", raw.duplicated().sum())
print("first prices:", raw["unit_price"].head(3).tolist())
print("first dates:", raw["order_date"].head(3).tolist())
```

```
order_id           str
order_date         str
customer_id        str
region             str
channel            str
category           str
product            str
quantity       float64
unit_price         str
discount       float64
dtype: object

missing values per column:
quantity    14
dtype: int64

region spellings:
region
East      51
East      10
North     71
North     10
South     31
South      6
West      42
West       6
east      22
north     24
south     13
west      17
Name: count, dtype: int64
duplicate rows: 3
first prices: ['$18.00', '$35.00', '$50.00']
first dates: ['05/01/2025', '2025-01-06', '2025-01-08']
```

The problems, one by one:

- **Types are wrong.** `unit_price` and `order_date` are text (`str`), not numbers and dates. `quantity` is `float64` rather than whole numbers, because a column with missing values can't be a plain integer column.
- **Missing values.** 14 quantities are missing. `isna()` marks missing values, and summing it counts them per column.
- **Inconsistent text.** Look closely at the region counts: "East" appears twice. One of them is "East " with a trailing space, invisible in the output but a different value to the computer. There are also lowercase versions. Twelve spellings for four regions.
- **Duplicates.** Three rows are exact copies of others.
- **Formats.** Prices have a dollar sign, and dates come in two layouts: day/month/year and year-month-day.

A Date Trap

Mixed date formats deserve their own warning, because the obvious fix silently gives wrong answers:

```
import pandas as pd

dates = pd.Series(["05/01/2025", "2025-01-06", "2025-01-08"])

guessed = pd.to_datetime(dates, format="mixed", dayfirst=True)
print(guessed.dt.strftime("%d %b %Y").tolist())

iso = pd.to_datetime(dates, format="%Y-%m-%d", errors="coerce")
dmy = pd.to_datetime(dates, format="%d/%m/%Y", errors="coerce")
print(iso.fillna(dmy).dt.strftime("%d %b %Y").tolist())
```

```
['05 Jan 2025', '01 Jun 2025', '01 Aug 2025']
['05 Jan 2025', '06 Jan 2025', '08 Jan 2025']
```

Asking pandas to guess each date's format with `dayfirst=True` reads "05/01/2025" correctly as 5 January, but then also reads the ISO date "2025-01-06" as year, *day*, month: 1 June. No error, no warning, just a wrong date. The safe approach is to try each known format explicitly, with `errors="coerce"` turning non-matching dates into missing values (`NaT`, "not a time"), then combine them with `fillna`.

> **Warning:** Never let software guess dates when a column might mix formats. "03/04/2025" is 3 April in Britain and India and 4 March in the United States. Find out which format each source uses, and parse it explicitly.

Cleaning, Step by Step

```
import pandas as pd

raw = pd.read_csv("../data/orders_raw.csv")
clean = raw.drop_duplicates().copy()

clean["region"] = clean["region"].str.strip().str.title()
clean["unit_price"] = (
    clean["unit_price"].str.replace("$", "", regex=False).astype(float)
)
# Two date layouts: try each one explicitly, never guess
iso = pd.to_datetime(
    clean["order_date"], format="%Y-%m-%d", errors="coerce"
```

```
)
dmy = pd.to_datetime(
    clean["order_date"], format="%d/%m/%Y", errors="coerce"
)
clean["order_date"] = iso.fillna(dmy)
assert clean["order_date"].notna().all()

too_big = clean["quantity"] > 50
print("rows with an impossible quantity:", too_big.sum())
clean.loc[too_big, "quantity"] = None
clean["quantity"] = clean["quantity"].fillna(clean["quantity"].median())
clean["quantity"] = clean["quantity"].astype(int)

print("rows:", len(raw), "->", len(clean))
print("regions:", sorted(clean["region"].unique()))
print(clean.dtypes[["order_date", "quantity", "unit_price"]])
print(clean[["order_date", "region", "quantity", "unit_price"]].head(3))
```

```
rows with an impossible quantity: 1
rows: 303 -> 300
regions: ['East', 'North', 'South', 'West']
order_date    datetime64[us]
quantity               int64
unit_price           float64
dtype: object
  order_date region  quantity  unit_price
0 2025-01-05  South         1        18.0
1 2025-01-06   West         1        35.0
2 2025-01-08   East         1        50.0
```

1. `drop_duplicates()` removes the three repeated rows. `.copy()` makes a new, independent DataFrame, so changes to `clean` never affect `raw`.
2. `str.strip()` removes spaces at the ends, and `str.title()` capitalizes each word, turning twelve spellings into four.
3. `str.replace("$", "", regex=False)` removes the dollar sign, and `astype(float)` makes the column numeric. `regex=False` matters: in a regular expression (Chapter 13), `$` means "end of text", not a dollar sign.
4. Dates are parsed with each format explicitly. The `assert` stops the program if any date matched neither format, rather than letting missing dates slip through.
5. One quantity was 400, impossible for a homeware order (the real maximum is 7). Treat it as an entry error: set it to missing, along with the other 14 missing quantities, and fill all of them with the **median** quantity. With the missing values gone, the column can become whole numbers.

What to do with missing values is a judgment call, and you should say what you chose in your report:

Approach	When it makes sense
Drop the rows (`dropna`)	Few rows affected, missing at random
Fill with median or mean (`fillna`)	A rough value is fine for totals
Fill from neighbors (`ffill`, `interpolate`)	Time series with short gaps (Chapter 25)
Keep as missing	The gap itself is meaningful; most pandas methods skip missing values
Fix at the source	Always the best option when you can

Real-World Story: Spreadsheet programs helpfully convert anything that looks like a date into a date. A 2016 study in *Genome Biology* found that about one in five genetics papers that shared gene lists as Excel files contained gene names that had been silently converted into dates: the gene SEPT2 became 2 September, and MARCH1 became 1 March. In 2020, the committee that names human genes gave up and renamed them: SEPT1 is now SEPTIN1 and MARCH1 is MARCHF1. Cleaning means checking types, not just values.

Validate With Checks

Once data is clean, write down your expectations as checks that run every time:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])

checks = {
    "order IDs are unique": orders["order_id"].is_unique,
    "no missing values": orders.notna().all().all(),
    "quantities between 1 and 20": orders["quantity"].between(1, 20).all(),
    "prices are positive": (orders["unit_price"] > 0).all(),
    "dates are in 2025": (orders["order_date"].dt.year == 2025).all(),
    "known regions only": orders["region"]
    .isin(["North", "South", "East", "West"])
    .all(),
}
for name, passed in checks.items():
    print(f"{'PASS' if passed else 'FAIL'}  {name}")
```

```
PASS  order IDs are unique
PASS  no missing values
PASS  quantities between 1 and 20
PASS  prices are positive
PASS  dates are in 2025
PASS  known regions only
```

Each line states one rule about the data: unique IDs, nothing missing, sensible quantities, positive prices, dates in the right year, only known regions. When next month's file arrives with a new problem, a check fails before any wrong number reaches a report. In a larger project, you'd turn these into tests (Chapter 19).

Exercises

1. **Weights as text.** Clean this Series into numbers: `[" 12 kg", "7kg", "n/a", "15 KG ", ""]`. Lowercase it, remove "kg", strip spaces, and convert with `pd.to_numeric(..., errors="coerce")`, which turns anything unconvertible into a missing value. What's the mean of the known weights?
2. **Show the duplicates.** In `orders_raw.csv`, print every row that has a duplicate, both copies, sorted by order ID. (Hint: `duplicated(keep=False)` marks all copies.)

Key Takeaways

- Inspect first: types, missing values, distinct spellings, duplicates and formats.
- Clean text with `str.strip()` and `str.title()`; convert types with `astype` and `to_datetime`.
- Parse dates with explicit formats; guessing can silently swap days and months.
- Choose how to handle missing values and impossible values, and say what you chose.
- Turn your expectations into automatic checks.

CHAPTER 8 · BEGINNER

Transforming Columns

Raw data rarely has the column you need. Orders have quantity, price and discount, but you want revenue; customers have a birth year, but you want age bands. This chapter covers the ways to make new columns: arithmetic, `assign`, conditions, mappings and bins. It ends with the most important performance habit in pandas: work on whole columns, not row by row.

New Columns From Old Ones

```
import numpy as np
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])

orders["revenue"] = (
    orders["quantity"] * orders["unit_price"] * (1 - orders["discount"])
)
orders = orders.assign(
    month=orders["order_date"].dt.month,
    weekend=orders["order_date"].dt.dayofweek >= 5,
)
orders["size"] = np.select(
    [orders["revenue"] < 30, orders["revenue"] < 100],
    ["small", "medium"],
    default="large",
)
orders["channel"] = orders["channel"].map(
    {"web": "Website", "app": "Mobile app", "marketplace": "Marketplace"}
)
cols = ["order_id", "revenue", "month", "weekend", "size", "channel"]
print(orders[cols].head(4))
print(orders["size"].value_counts())
```

```
  order_id  revenue  month  weekend    size     channel
0   L00001     24.0      1    False   small  Mobile app
1   L00002     21.6      1    False   small     Website
2   L00003     32.0      1    False  medium     Website
3   L00004     18.0      1    False   small  Mobile app
size
medium    3260
small     1475
large     1098
Name: count, dtype: int64
```

- **Arithmetic on columns** works row by row automatically: `revenue` multiplies each order's quantity, price and (1 minus) discount.
- `assign` adds several columns at once and returns a new DataFrame. Here it adds the month and a weekend flag from the date's `.dt` accessor (`dayofweek` counts Monday as 0, so 5 and 6 are Saturday and Sunday).
- `np.select` chooses between values by conditions, checked in order: revenue under $30 is "small", under $100 "medium", and everything else the `default`, "large". It's the column version of `if`/`elif`/`else`.
- `map` with a dictionary replaces each value with its partner: short channel codes become readable names. Values missing from the dictionary would become missing, so make sure the dictionary covers every value.

Bins: Turning Numbers Into Groups

Ages, prices and incomes are often easier to analyze in groups:

```
import pandas as pd

customers = pd.read_csv("../data/customers.csv")
customers["age"] = 2025 - customers["birth_year"]

customers["age_band"] = pd.cut(
    customers["age"],
    bins=[0, 29, 44, 59, 120],
    labels=["under 30", "30-44", "45-59", "60+"],
)
print(customers["age_band"].value_counts().sort_index())

# qcut makes groups of (almost) equal size instead of equal width
customers["age_quartile"] = pd.qcut(
    customers["age"], 4, labels=["Q1", "Q2", "Q3", "Q4"]
)
print(
    customers.groupby("age_quartile", observed=True)["age"].agg(
        ["min", "max", "count"]
    )
```

```
)
```

```
age_band
under 30    235
30-44       341
45-59       364
60+         260
Name: count, dtype: int64
              min  max  count
age_quartile
Q1             20   32    305
Q2             33   45    298
Q3             46   58    306
Q4             59   70    291
```

`pd.cut` puts each value into a band you define. The bin edges `[0, 29, 44, 59, 120]` mean "over 0 up to 29", "over 29 up to 44", and so on: each band includes its right edge. `pd.qcut` instead makes bands with (almost) the same number of people in each: the age quartiles each hold about 300 customers, and the table shows where the boundaries fell.

> **Tip:** Use `cut` when the band edges mean something (legal ages, price points), and `qcut` when you want equal-sized groups for comparison, such as "the top quarter of customers".

Row by Row, or Whole Columns?

`apply(..., axis=1)` runs your own function once per row. It's flexible, but it calls Python for every row, which is slow on large tables. The same logic can almost always be written with whole-column operations:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")


def shipping(row):
    """Free shipping over $100; otherwise $6, or $4 for app orders."""
    total = row["quantity"] * row["unit_price"]
    if total >= 100:
        return 0.0
    return 4.0 if row["channel"] == "app" else 6.0


slow = orders.apply(shipping, axis=1)  # calls Python once per row

total = orders["quantity"] * orders["unit_price"]
```

```
fast = (
    pd.Series(6.0, index=orders.index)
    .mask(orders["channel"] == "app", 4.0)
    .mask(total >= 100, 0.0)
)  # whole-column operations

print("same answer:", slow.equals(fast))
print("total shipping charged:", fast.sum())
```

```
same answer: True
total shipping charged: 24808.0
```

The rule: free shipping on orders of $100 or more; otherwise $6, or $4 through the app. The vectorized version starts with $6 for everyone, then uses `mask` to replace values where a condition is true: $4 for app orders, then $0 for large orders. The later `mask` wins, so a large app order ships free, matching the function's logic. Both give the same answer, $24,808 in total, but the vectorized version works on whole columns at once. On this table the difference is a fraction of a second; on ten million rows, it's the difference between seconds and many minutes.

> **Common Mistake:** Writing a loop like `for i in range(len(df)): df.loc[i, "revenue"] = ...`. It works, but it's the slowest possible way. Look for a column operation, `np.select`, `mask`, `where` or `map` first.

Exercises

1. **Price bands.** Put each order into a price band with `pd.cut`: "budget" (up to $25), "mid" (up to $50) and "premium" (up to $200). How many orders fall in each band?
2. **Mood.** In `survey.csv`, map satisfaction scores 1 and 2 to "unhappy", 3 to "neutral", and 4 and 5 to "happy", and count each mood.

Key Takeaways

- Column arithmetic, `assign` and the `.dt` accessor make new columns in one line.
- `np.select` handles if/elif/else rules; `map` translates values with a dictionary.

- `cut` makes bands with chosen edges; `qcut` makes equal-sized groups.
- Prefer whole-column operations to `apply(axis=1)` and loops: same answer, far faster.

CHAPTER 9 · INTERMEDIATE

Grouping and Aggregating

"Revenue by region." "Average rating by plan." "Orders per month per channel." Most business questions have this shape: split the data into groups, calculate something for each group, and combine the results into a table. pandas calls it **split-apply-combine**, and does it with `groupby`.

groupby

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["revenue"] = (
    orders["quantity"] * orders["unit_price"] * (1 - orders["discount"])
)

print(orders.groupby("region")["revenue"].sum().round(0))
print()
by_region = orders.groupby("region").agg(
    orders=("order_id", "count"),
    revenue=("revenue", "sum"),
    avg_order=("revenue", "mean"),
    customers=("customer_id", "nunique"),
)
print(by_region.round(1).sort_values("revenue", ascending=False))
```

```
region
East      98878.0
North    113274.0
South     86223.0
West      81328.0
Name: revenue, dtype: float64

        orders   revenue  avg_order  customers
region
```

```
North     1768  113274.2       64.1        352
East      1560   98878.3       63.4        316
South     1300   86222.9       66.3        272
West      1205   81328.0       67.5        251
```

`orders.groupby("region")["revenue"].sum()` reads almost like English: group the orders by region, take the revenue column, sum it. The result is a Series indexed by region.

For several results at once, use `agg` with **named aggregations**: each argument names an output column and says which input column and which function to use. `count` counts rows, `sum` and `mean` do what you'd expect, and `nunique` counts distinct values, here distinct customers. Now the picture is richer: North has the most revenue because it has the most orders and customers, but West has the *highest average order*, $67.50. Which region is "best" depends on the question.

> **Tip:** Common aggregation functions: `count`, `sum`, `mean`, `median`, `min`, `max`, `std`, `nunique`, `first` and `last`. You can also pass your own function, such as `lambda s: s.quantile(0.9)`.

Pivot Tables and Crosstabs

A **pivot table** groups by two things at once, one along the rows and one along the columns:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
orders["revenue"] = orders["quantity"] * orders["unit_price"]
orders["quarter"] = orders["order_date"].dt.quarter

table = orders.pivot_table(
    index="category",
    columns="quarter",
    values="revenue",
    aggfunc="sum",
    margins=True,
    margins_name="Total",
)
print((table / 1000).round(1))  # thousands of dollars
print()
print(
    pd.crosstab(
        orders["region"], orders["channel"], normalize="index"
    ).round(2)
```

```
)
```

```
quarter      1     2     3      4  Total
category
Bedding   20.5  21.0  25.6   43.8  110.9
Decor      7.1   7.5   9.5   15.6   39.7
Garden     7.6  23.6   6.8   12.6   50.5
Kitchen   16.2  14.5  21.2   34.4   86.3
Lighting  17.5  22.5  25.6   44.4  109.9
Total     68.9  89.0  88.7  150.8  397.4

channel   app  marketplace   web
region
East     0.28         0.20  0.51
North    0.31         0.20  0.49
South    0.30         0.21  0.50
West     0.32         0.20  0.48
```

Rows are categories, columns are quarters, and each cell is total revenue before discounts, shown in thousands of dollars. `margins=True` adds totals. The table tells a story at a glance: everything grows in the fourth quarter, the holiday season, but Garden peaks in the second quarter, spring, and falls back after.

`pd.crosstab` counts how often each combination of two columns occurs; with `normalize="index"`, each row adds up to 1. Here that shows each region's channel mix: about half of every region's orders come through the website, and the mix is very similar everywhere.

Share of Total With transform

Sometimes you need a group's total *next to each row*, for example to compute each row's share of its group. `transform` returns one value per original row:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["revenue"] = orders["quantity"] * orders["unit_price"]

sales = orders.groupby(["region", "category"], as_index=False)[
    "revenue"
].sum()
# transform returns one value per row, aligned with the original rows
sales["share"] = sales["revenue"] / sales.groupby("region")[
    "revenue"
].transform("sum")
top = sales.sort_values("share", ascending=False).groupby("region").head(1)
print(top.round({"revenue": 0, "share": 3}).to_string(index=False))
```

```
region category  revenue  share
  West Lighting  24474.0  0.288
 North  Bedding  33780.0  0.285
 South  Bedding  25440.0  0.283
  East  Bedding  28445.0  0.274
```

First, revenue is summed by region and category (`as_index=False` keeps them as ordinary columns). Then `groupby("region")["revenue"].transform("sum")` gives each row its region's total, and dividing gives each category's share of its region. Sorting by share and keeping the first row per region shows each region's biggest category: Bedding everywhere except the West, where Lighting leads with almost 29%.

> **Common Mistake:** Comparing totals between groups of very different sizes. North has more revenue than West in every category simply because it has more customers. Shares, averages per customer or rates per order often answer the real question better than raw totals.

Exercises

1. **Best seller per category.** Which product sold the most items in each category? (Hint: group by category and product, sum the quantities, then use `idxmax` within each category.)
2. **Satisfaction table.** Make a pivot table of average satisfaction from `survey.csv`, with age groups as rows and plans as columns.

Key Takeaways

- `groupby` splits rows into groups, applies a calculation, and combines the results.
- Named aggregations with `agg` build summary tables with clear column names.
- `pivot_table` groups by two dimensions; `crosstab` counts combinations, optionally as shares.

- `transform` returns group results aligned with the original rows, for shares and comparisons within groups.
- Compare rates and averages, not just totals, when groups differ in size.

CHAPTER 10 · INTERMEDIATE

Combining Tables

Data is usually split across several tables: orders in one, products in another, customers in a third. To answer "what's our profit by customer segment?" you need all three. This chapter shows how to **merge** tables on shared columns, how the four kinds of join differ, and how to stack tables with the same columns on top of each other.

Merging on a Key

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
products = pd.read_csv("../data/products.csv")
customers = pd.read_csv("../data/customers.csv")

full = orders.merge(
    products[["product", "unit_cost"]],
    on="product",
    how="left",
    validate="many_to_one",
).merge(
    customers[["customer_id", "segment"]],
    on="customer_id",
    how="left",
    validate="many_to_one",
)
full["revenue"] = (
    full["quantity"] * full["unit_price"] * (1 - full["discount"])
)
full["profit"] = full["revenue"] - full["quantity"] * full["unit_cost"]

print(full.shape)
summary = full.groupby("segment")[["revenue", "profit"]].sum()
summary["margin"] = summary["profit"] / summary["revenue"]
```

```
print(summary.round(2))
```

```
(5833, 14)
           revenue     profit  margin
segment
Business   77525.2   35084.24    0.45
Consumer  302178.2  135831.46    0.45
```

`merge` matches rows with the same value in the `on` column, called the **key**. Each order gets its product's unit cost and its customer's segment. Two arguments make merges safe:

- `how="left"` keeps every order, even one whose product or customer were missing from the other table (it would get missing values instead of disappearing).
- `validate="many_to_one"` checks that each product and each customer appears only once in the right-hand table. If a product were listed twice by mistake, each matching order would be duplicated and every total would be wrong. With `validate`, pandas raises an error instead.

The result still has 5,833 rows, which confirms nothing was lost or duplicated. Profit is revenue minus quantity times unit cost, and both segments turn out to earn a 45% margin.

> **Tip:** After every merge, check the row count. If it grew, a key was duplicated; if it shrank with an inner join, some keys didn't match.

Four Kinds of Join

Small tables make the differences easy to see:

```
import pandas as pd

left = pd.DataFrame({"id": [1, 2, 3], "name": ["Asha", "Ben", "Chen"]})
right = pd.DataFrame({"id": [2, 3, 4], "city": ["Leeds", "Pune", "Oslo"]})

for how in ["inner", "left", "right", "outer"]:
    print(f"--- {how}")
    print(left.merge(right, on="id", how=how, indicator=True))
```

```
--- inner
   id  name   city _merge
0   2   Ben  Leeds   both
1   3  Chen   Pune   both
--- left
   id  name   city     _merge
0   1  Asha    NaN  left_only
1   2   Ben  Leeds       both
2   3  Chen   Pune       both
--- right
   id  name   city      _merge
0   2   Ben  Leeds        both
1   3  Chen   Pune        both
2   4   NaN   Oslo  right_only
--- outer
   id  name   city      _merge
0   1  Asha    NaN   left_only
1   2   Ben  Leeds        both
2   3  Chen   Pune        both
3   4   NaN   Oslo  right_only
```

- **inner:** only keys in both tables (2 and 3).
- **left:** every row of the left table; missing values where the right has no match (Asha has no city).
- **right:** every row of the right table.
- **outer:** every key from either table.

`indicator=True` adds a `_merge` column that says where each row came from, the best way to find rows that didn't match.

Stacking Tables

When tables have the same columns, for example the same report from different months, `concat` stacks them:

```
import pandas as pd

q1 = pd.DataFrame({"store": ["A", "B"], "sales": [120, 95]})
q2 = pd.DataFrame({"store": ["A", "B"], "sales": [130, 90]})

both = pd.concat([q1, q2], keys=["Q1", "Q2"], names=["quarter", "row"])
print(both)
print(both.groupby("store")["sales"].sum())
```

```
            store  sales
quarter row
Q1      0       A    120
        1       B     95
Q2      0       A    130
        1       B     90
store
A    250
B    185
Name: sales, dtype: int64
```

`keys` labels each piece, giving a two-level index (quarter, then the original row number), so you never lose track of where a row came from. After stacking, everything else works as usual: store A sold 250 in the two quarters together.

Exercises

1. **Customers with no orders.** Which customers in `customers.csv` never placed an order? Use a left merge with `indicator=True`. How many are there, and which are the first five?
2. **Buyer ages.** Merge orders with customers' birth years and find the median age of buyers in each category (use age = 2025 minus birth year).

Key Takeaways

- `merge` joins tables on key columns; `how` chooses inner, left, right or outer.
- `validate` catches duplicated keys before they multiply your rows.
- Check row counts after every merge; `indicator=True` shows unmatched rows.
- `concat` stacks tables with the same columns; `keys` records where each part came from.

CHAPTER 11 · INTERMEDIATE

Reshaping Data

The same numbers can be laid out in different shapes. A **wide** table has one row per thing and one column per month; a **long** table has one row per thing *per month*. People like wide tables for reading; pandas, charts and databases usually prefer long ones. This chapter shows how to switch between them.

Tidy Data

A table is **tidy** when:

1. each column is one variable (store, month, sales),
2. each row is one observation (one store in one month), and
3. each cell holds one value.

Tidy tables are easy to filter, group and chart, because every question is about columns. Spreadsheets you receive are often not tidy: months spread across columns, or two values squeezed into one cell. Reshaping fixes the first problem; text tools (Chapter 13) fix the second.

melt and pivot

```
import pandas as pd

wide = pd.DataFrame(
    {
        "store": ["Avon", "Bray"],
        "Jan": [120, 95],
        "Feb": [130, 90],
```

```
        "Mar": [150, 110],
    }
)
print(wide)

long = wide.melt(id_vars="store", var_name="month", value_name="sales")
print(long)

back = long.pivot(index="store", columns="month", values="sales")
print(back[["Jan", "Feb", "Mar"]])
```

```
  store  Jan  Feb  Mar
0  Avon  120  130  150
1  Bray   95   90  110
  store month  sales
0  Avon   Jan    120
1  Bray   Jan     95
2  Avon   Feb    130
3  Bray   Feb     90
4  Avon   Mar    150
5  Bray   Mar    110
month  Jan  Feb  Mar
store
Avon   120  130  150
Bray    95   90  110
```

- `melt` turns columns into rows. `id_vars="store"` keeps the store column; every other column becomes a pair of values in `month` and `sales`. Two stores times three months gives six rows.
- `pivot` goes back: `index` gives the rows, `columns` the new column headers, and `values` fills the cells. Selecting `[["Jan", "Feb", "Mar"]]` restores the calendar order, because pivot sorts the new columns alphabetically.

> **Common Mistake:** `pivot` fails with "Index contains duplicate entries" when two rows have the same index and column (the same store and month twice). Then you need `pivot_table` with an `aggfunc`, such as `sum`, to say how to combine them.

stack and unstack

When a Series has a two-level index, `unstack` moves the inner level into columns and `stack` moves it back:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
units = orders.groupby(["region", "channel"])["quantity"].sum()
print(units.head(4))  # a Series with a two-level index
print(units.unstack())  # inner level becomes columns
print(units.unstack().stack().equals(units))
```

```
region  channel
East    app             672
        marketplace     499
        web            1326
North   app             856
Name: quantity, dtype: int64
channel  app  marketplace   web
region
East     672          499  1326
North    856          563  1378
South    648          436  1039
West     643          431   896
True
```

Grouping by two columns gives a Series indexed by region and channel, convenient to compute but awkward to read. `unstack()` turns it into a region-by-channel table. `stack()` reverses it exactly, as the final `True` confirms.

Exercises

1. **Long prices.** `prices.csv` has one column per company. Melt it into a long table with columns `date`, `ticker` and `close`. How many rows does it have, and what was each ticker's highest close?
2. **Orders by discount.** Make a table with regions as rows, the three discount levels as columns, and the number of orders in each cell.

Key Takeaways

- Tidy data has one variable per column and one observation per row.

- `melt` turns wide tables long; `pivot` turns long tables wide (`pivot_table` when there are duplicates).
- `unstack` and `stack` move index levels into and out of columns.

CHAPTER 12 · INTERMEDIATE

Dates and Time Series

Time is in almost every dataset: when an order was placed, when a sensor read, when a price closed. pandas has strong tools for dates: pulling out parts like the weekday, regrouping daily data into weeks or months, rolling averages, and changes from one period to the next.

Date Parts

Once a column is a real date (`parse_dates` when reading, or `pd.to_datetime` later), the `.dt` accessor gives its parts:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
d = orders["order_date"]

print(d.min().date(), "to", d.max().date())
print(
    orders.assign(
        weekday=d.dt.day_name(),
        month=d.dt.month_name(),
        week=d.dt.isocalendar().week,
    )[["order_date", "weekday", "month", "week"]].head(3)
)

by_day = orders.groupby(d.dt.day_name())["order_id"].count()
order = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday",
]
```

```
print(by_day.reindex(order))
```

```
2025-01-01 to 2025-12-31
  order_date    weekday    month  week
0 2025-01-01  Wednesday  January     1
1 2025-01-01  Wednesday  January     1
2 2025-01-01  Wednesday  January     1
order_date
Monday       815
Tuesday      815
Wednesday    799
Thursday     787
Friday       788
Saturday     960
Sunday       869
Name: order_id, dtype: int64
```

`dt.day_name()`, `dt.month_name()` and `dt.isocalendar().week` give the weekday, month and week number. Grouping by weekday shows Saturday is the busiest day, with Sunday second. Weekday names would sort alphabetically (Friday first), so `reindex` puts them in calendar order.

Resampling: Days to Months

Resampling regroups a time series into new time periods. It needs the dates as the index:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
orders["revenue"] = orders["quantity"] * orders["unit_price"]
daily = orders.set_index("order_date")["revenue"].resample("D").sum()

monthly = daily.resample("MS").sum()  # MS: month start labels
print((monthly / 1000).round(1).to_string())
print()
print("month-on-month change, last 3 months:")
print(monthly.pct_change().tail(3).round(3).to_string())
```

```
order_date
2025-01-01    22.6
2025-02-01    20.3
2025-03-01    26.0
2025-04-01    29.5
2025-05-01    29.0
2025-06-01    30.5
2025-07-01    29.3
2025-08-01    28.5
2025-09-01    30.8
2025-10-01    40.3
2025-11-01    49.0
```

```
2025-12-01    61.5
Freq: MS

month-on-month change, last 3 months:
order_date
2025-10-01    0.306
2025-11-01    0.215
2025-12-01    0.256
Freq: MS
```

`resample("D").sum()` gives daily revenue (before discounts, this time), with a zero for any day without orders, so no day is silently missing. `resample("MS").sum()` then totals each month, labeled by its first day ("MS" means month start; "ME" would label by month end). The holiday season stands out: December's $61,500 is about three times February's.

`pct_change()` gives the change from each period to the previous one: October was 30.6% up on September, and growth continued through December.

> **Tip:** Common resampling codes: `D` (day), `W` (week, ending Sunday), `MS` and `ME` (month), `QS` and `QE` (quarter), `YS` and `YE` (year).

Rolling Averages

Daily numbers are noisy. A **rolling average** smooths them by averaging each day with the days before it:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
orders["revenue"] = orders["quantity"] * orders["unit_price"]
daily = orders.set_index("order_date")["revenue"].resample("D").sum()

week = daily.rolling(7).mean()  # average of each day and the 6 before
print(
    pd.DataFrame({"daily": daily, "7-day average": week})
    .loc["2025-11-24":"2025-11-30"]
    .round(0)
)
print(
    "best 7 days ended on",
    week.idxmax().date(),
    f"with ${week.max():,.0f} a day",
)
```

```
             daily  7-day average
order_date
2025-11-24  1250.0         1659.0
2025-11-25  1476.0         1647.0
2025-11-26  2059.0         1766.0
2025-11-27  1753.0         1730.0
2025-11-28  1689.0         1689.0
2025-11-29  2317.0         1755.0
2025-11-30  2009.0         1793.0
best 7 days ended on 2025-12-12 with $2,359 a day
```

`rolling(7).mean()` averages each day with the six before it. Daily revenue in the last week of November jumped between $1,250 and $2,317, while the 7-day average moved gently between about $1,650 and $1,800. The best seven days of the year ended on 12 December, at $2,359 a day: the pre-holiday peak.

`shift(1)` moves a series one step later in time (each day gets yesterday's value), which is how you compare each period with the one before, or line up a cause with an effect that comes later (Chapter 25).

Exercises

1. **Weekly conversion.** `sessions.csv` records an experiment over four weeks of October. What share of sessions converted in each week? (Hint: the mean of a 0/1 column is the share of 1s.)
2. **Best and worst months.** For the city of Avonford in `air_quality.csv`, find the month with the highest and the lowest average PM2.5.

Key Takeaways

- Use real date types; then `.dt` gives weekdays, months, weeks and more.
- `resample` regroups time series into new periods, with explicit zeros for empty ones.
- `pct_change` measures change between periods; `shift` compares each period with an earlier one.
- Rolling averages smooth noisy series and make trends visible.

CHAPTER 13 · INTERMEDIATE

Working With Text

Text columns hold some of the most useful information in a dataset, and some of the hardest to use: product names, comments, bank transaction descriptions, codes. pandas gives every text column a `.str` accessor with Python's string methods and regular expressions, applied to the whole column at once.

String Methods on Whole Columns

The survey has a free-text comment from each respondent:

```
import pandas as pd

survey = pd.read_csv("../data/survey.csv", keep_default_na=False)
comments = survey["comment"]

print(comments.str.len().describe().round(1))
print()
print("mention 'slow':", comments.str.contains("slow").sum())
print(
    "mention price or value:",
    comments.str.contains(r"price|value", regex=True).sum(),
)
print()
words = comments.str.lower().str.split().explode()
stopwords = {"to", "is", "was", "too", "the"}
print(words[~words.isin(stopwords)].value_counts().head(5))
```

```
count    800.0
mean      11.4
std        3.0
min        0.0
25%       10.0
```

```
50%       13.0
75%       13.0
max       16.0
Name: comment, dtype: float64

mention 'slow': 114
mention price or value: 216

comment
support    171
good       159
value      159
easy       138
use        138
Name: count, dtype: int64
```

- `keep_default_na=False` matters here: by default, `read_csv` treats empty fields as missing, but an empty comment is a real answer ("nothing to say"), so the program keeps it as an empty string.
- `str.len()` gives each comment's length; `describe()` summarizes those lengths.
- `str.contains("slow")` marks comments containing a word; summing counts them: 114 mention slowness.
- `str.contains(r"price|value", regex=True)` uses a **regular expression**, a small pattern language. `|` means "or", so it finds comments mentioning either word.
- Lowercasing, splitting into words, and `explode` (one row per word) turns comments into a word list. Short common words such as "to" and "is", called **stopwords**, are removed before counting, or they'd top every list. "Support" is the most common remaining word.

Extracting and Categorizing

Bank statements describe transactions in capital letters with store numbers mixed in. Regular expressions pull out the parts you need:

```
import pandas as pd

budget = pd.read_csv("../data/budget.csv")

budget["store_no"] = budget["description"].str.extract(r"#(\d+)")
rules = {
    "Groceries": r"FRESHMART",
    "Coffee": r"CAFE",
    "Transport": r"TRANSIT|RIDESHARE",
```

```
    "Housing": r"RENT|POWER",
    "Income": r"SALARY",
}
budget["category"] = "Other"
for name, pattern in rules.items():
    budget.loc[budget["description"].str.contains(pattern), "category"] = (
        name
    )

print(budget[budget["store_no"].notna()].head(3))
print(budget.groupby("category")["amount"].sum().round(2).sort_values())
```

```
          date    description  amount store_no   category
3   2025-01-04  FRESHMART #25 -112.47       25  Groceries
8   2025-01-11  FRESHMART #10  -78.76       10  Groceries
14  2025-01-18  FRESHMART #10  -97.40       10  Groceries
category
Housing     -18680.00
Groceries    -4977.82
Other        -2005.79
Coffee         -534.13
Transport      -354.61
Income       50400.00
Name: amount, dtype: float64
```

`str.extract(r"#(\d+)")` finds a `#` followed by one or more digits (`\d+`), and returns the part in parentheses, the store number. Rows without a match get a missing value.

To categorize transactions, the program applies a dictionary of patterns in turn: any description matching `FRESHMART` is groceries, `TRANSIT|RIDESHARE` is transport, and so on. Anything unmatched stays "Other". The totals show the household's year: $50,400 of income, $18,680 on housing, and about $5,000 on groceries. Project 6 (Chapter 26) builds on this.

Some regular expression pieces worth knowing:

Pattern	Matches
`\d`, `\d+`	One digit; one or more digits
`\w+`	A word (letters, digits, underscore)
`\s`	A space or tab
`^`, `$`	Start, end of the text
`A|B`	A or B
`(...)`	A group to extract

Pattern	Matches
`(?P<name>...)`	A named group (becomes a column name in `extract`)

> **Tip:** `str.contains(pattern, case=False)` ignores capitals. When you search for a literal character that has a special meaning in patterns, such as `$`, `.` or `+`, use `regex=False` or escape it with a backslash.

Exercises

1. **Cafe visits.** How many transactions were at each cafe? Show the names in title case ("Cafe Luma").
2. **Document codes.** Split codes like `"INV-2025-0041"` and `"po-2024-0007"` into three columns, `kind`, `year` and `num`, using `str.extract` with named groups. Make the kinds uppercase and `num` a whole number.

Key Takeaways

- The `.str` accessor applies string methods to whole columns.
- `str.contains`, `str.extract` and `str.replace` accept regular expressions for flexible matching.
- `str.split` with `explode` turns text into one row per word; remove stopwords before counting.
- A dictionary of patterns turns messy descriptions into clean categories.

PART III

Seeing and Explaining

CHAPTER 14 · INTERMEDIATE

Charts With matplotlib

A good chart shows in a second what a table takes a minute to read. **matplotlib** is Python's standard charting library: pandas uses it behind its own `plot` methods, and almost every other Python charting tool builds on it. This chapter draws the four charts you'll use most: line, bar, histogram and scatter.

The Shape of Every matplotlib Program

Every chart in this chapter follows the same five steps:

1. `fig, ax = plt.subplots()` creates a **figure** (the whole image) and an **axes** (one chart area inside it).
2. Draw on the axes: `ax.plot`, `ax.bar`, `ax.hist`, `ax.scatter`.
3. Label it: `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `ax.legend`.
4. `fig.tight_layout()` adjusts spacing so labels don't overlap.
5. `fig.savefig("name.png")` saves it. (`plt.show()` opens a window instead, if you're working interactively.)

> **Note:** The charts in this book were drawn by exactly the code shown, with a print-friendly style (fonts and colors chosen for paper) applied on top. Your charts will use matplotlib's default colors and fonts, but show the same data.

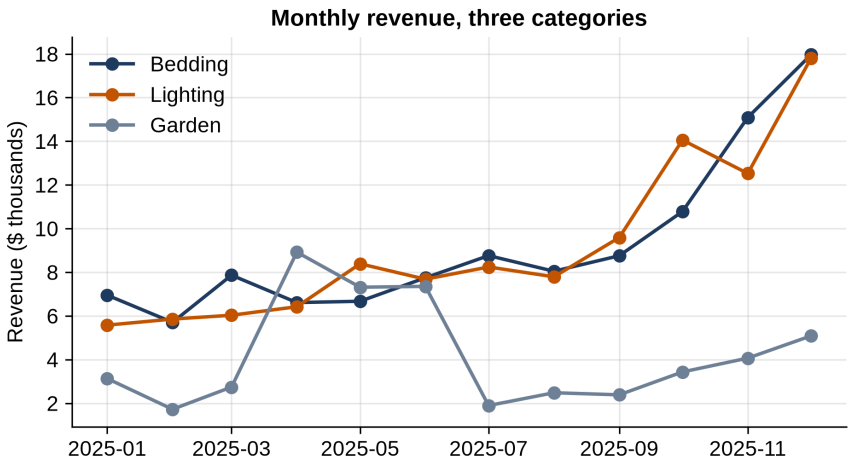
Line Charts: Change Over Time

```
import matplotlib.pyplot as plt
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
orders["revenue"] = orders["quantity"] * orders["unit_price"]
monthly = (
    orders.set_index("order_date")
    .groupby("category")["revenue"]
    .resample("MS")
    .sum()
    .unstack(level=0)
)

fig, ax = plt.subplots()
for category in ["Bedding", "Lighting", "Garden"]:
    ax.plot(
        monthly.index, monthly[category] / 1000, marker="o", label=category
    )
ax.set_title("Monthly revenue, three categories")
ax.set_ylabel("Revenue ($ thousands)")
ax.legend()
fig.tight_layout()
fig.savefig("line_chart.png")
print("Saved line_chart.png")
```

```
Saved line_chart.png
```



Monthly revenue in 2025 for Bedding, Lighting and Garden: Bedding and Lighting rise steeply toward December, while Garden peaks in April and falls after June.

The data step groups revenue by category and resamples each to months, then `unstack(level=0)` makes one column per category. Plotting three categories, rather than all five, keeps the chart readable. The story is immediate: Bedding and Lighting climb toward the holidays; Garden has its own season in spring.

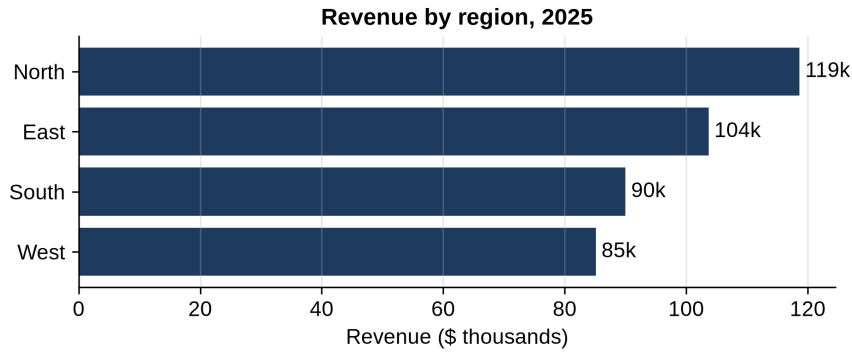
Bar Charts: Comparing Categories

```
import matplotlib.pyplot as plt
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["revenue"] = orders["quantity"] * orders["unit_price"]
by_region = orders.groupby("region")["revenue"].sum().sort_values()

fig, ax = plt.subplots(figsize=(6.4, 2.8))
bars = ax.barh(by_region.index, by_region / 1000)
ax.bar_label(bars, fmt="%.0fk", padding=3)
ax.set_title("Revenue by region, 2025")
ax.set_xlabel("Revenue ($ thousands)")
ax.grid(axis="y", visible=False)
fig.tight_layout()
fig.savefig("bar_chart.png")
print("Saved bar_chart.png")
```

```
Saved bar_chart.png
```



Horizontal bars of 2025 revenue by region, sorted: North 119k, East 104k, South 90k, West 85k.

Horizontal bars (`barh`) leave room for long category names, and sorting the bars makes ranking obvious. `bar_label` writes each value at the end of its bar, so readers don't have to estimate from the axis.

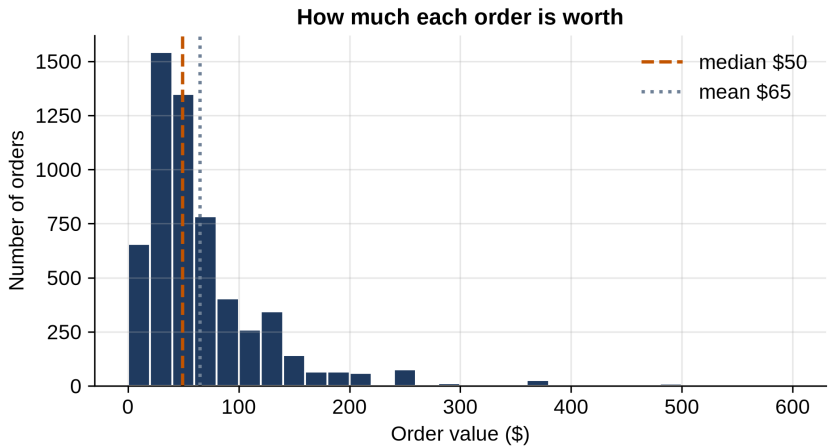
Histograms: How Values Are Spread

```
import matplotlib.pyplot as plt
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
value = orders["quantity"] * orders["unit_price"] * (1 - orders["discount"])

fig, ax = plt.subplots()
ax.hist(value, bins=range(0, 620, 20), edgecolor="white")
ax.axvline(
    value.median(),
    color="C1",
    linestyle="--",
    label=f"median ${value.median():.0f}",
)
ax.axvline(
    value.mean(),
    color="C2",
    linestyle=":",
    label=f"mean ${value.mean():.0f}",
)
ax.set_title("How much each order is worth")
ax.set_xlabel("Order value ($)")
ax.set_ylabel("Number of orders")
ax.legend()
fig.tight_layout()
fig.savefig("histogram.png")
print("Saved histogram.png")
```

```
Saved histogram.png
```



Histogram of order values in $20 bins: most orders are worth under $100, with a long tail of rare orders up to about $600; the median, $50, is marked to the left of the mean, $65.

A **histogram** counts how many values fall into each bin, here $20 wide. Order values are **skewed**: most orders are small, but a few large ones stretch the tail far to the right. The dashed line marks the median ($50) and the dotted line the mean ($65). The long tail pulls the mean up, which is why the median is often the better "typical" value for money (Chapter 16).

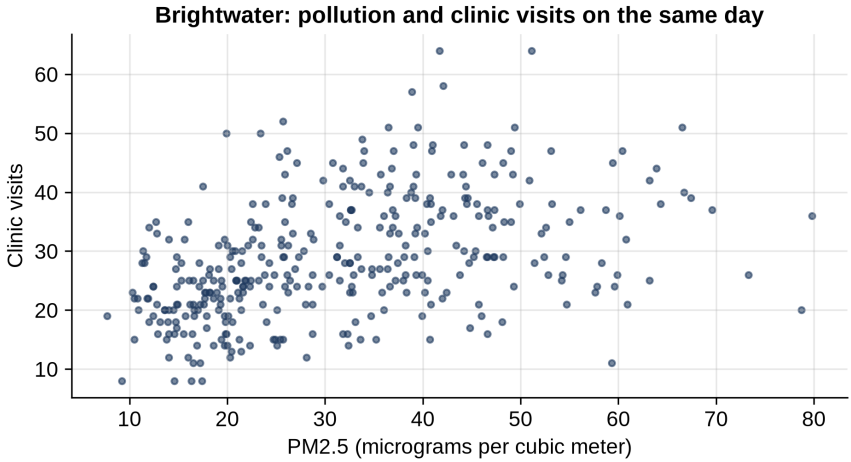
Scatter Plots: Two Measures Together

```
import matplotlib.pyplot as plt
import pandas as pd

air = pd.read_csv("../data/air_quality.csv", parse_dates=["date"])
visits = pd.read_csv("../data/clinic_visits.csv", parse_dates=["date"])
city = air.merge(visits, on=["date", "city"]).query("city == 'Brightwater'")

fig, ax = plt.subplots()
ax.scatter(city["pm25"], city["visits"], s=10, alpha=0.6)
ax.set_title("Brightwater: pollution and clinic visits on the same day")
ax.set_xlabel("PM2.5 (micrograms per cubic meter)")
ax.set_ylabel("Clinic visits")
fig.tight_layout()
fig.savefig("scatter.png")
print("Saved scatter.png")
```

```
Saved scatter.png
```



Scatter plot of daily PM2.5 pollution against clinic visits in Brightwater: visits tend to rise with pollution, but the cloud of points is wide.

Each dot is one day in one city. Visits tend to be higher on more polluted days, but the cloud is wide: same-day pollution explains only part of the variation. Chapter 16 measures this with correlation and finds a much stronger link two days later.

`alpha=0.6` makes the dots partly transparent, so overlapping points show up darker, and `s=10` makes them small. Both help when there are hundreds of points.

Exercises

1. **Pages per session.** Draw a bar chart of how many sessions in `sessions.csv` viewed 1, 2, 3, and so on pages. Label the axes and save it as `pages.png`.

Key Takeaways

- Every matplotlib chart: create figure and axes, draw, label, tidy the layout, save.
- Line charts for change over time, bars for comparing categories, histograms for spreads, scatter plots for two measures.
- Sort bars and label values; transparency helps crowded scatter plots.
- Skewed data pulls the mean away from the median; a histogram shows why.

CHAPTER 15 · INTERMEDIATE

Choosing Honest Charts

A chart is an argument. The same numbers can look like a crisis or like nothing, depending on choices most readers never notice: where the axis starts, which data is included, how things are compared. This chapter is about choosing the right chart, and drawing it honestly.

Which Chart?

You want to show	Use	Avoid
Change over time	Line chart	Pie charts, 3-D effects
Comparing categories	Bar chart, sorted	Many-slice pie charts
How values are spread	Histogram, box plot	Averages alone
Relationship of two measures	Scatter plot	Two lines on two y-axes
Parts of a whole	Bars of shares; a pie only with 2 or 3 parts	Stacked 3-D pies
Many groups, same measure	Small multiples (one small chart per group)	One crowded chart

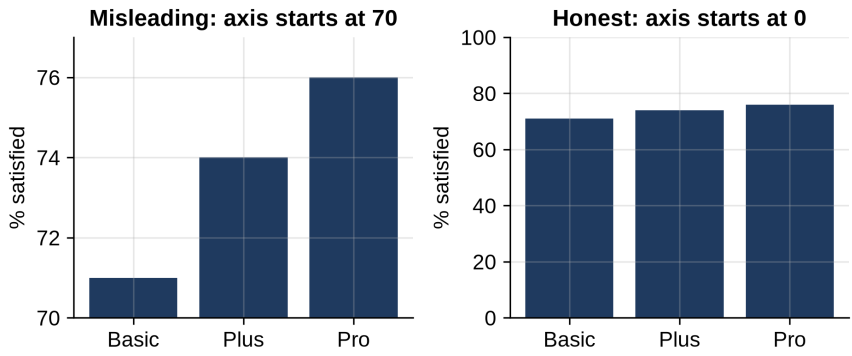
Where the Axis Starts

```
import matplotlib.pyplot as plt

plans = ["Basic", "Plus", "Pro"]
satisfied = [71, 74, 76]  # percent of customers satisfied

fig, (bad, good) = plt.subplots(1, 2, figsize=(6.4, 2.8))
bad.bar(plans, satisfied)
bad.set_ylim(70, 77)
bad.set_title("Misleading: axis starts at 70")
good.bar(plans, satisfied)
good.set_ylim(0, 100)
good.set_title("Honest: axis starts at 0")
for ax in (bad, good):
    ax.set_ylabel("% satisfied")
fig.tight_layout()
fig.savefig("truncated_axis.png")
print("Saved truncated_axis.png")
```

```
Saved truncated_axis.png
```



Two bar charts of the same satisfaction rates, 71%, 74% and 76% for Basic, Plus and Pro. On the left, the axis starts at 70, so Pro's bar looks six times taller than Basic's; on the right, the axis starts at 0 and the bars look almost equal.

Both charts show the same three numbers: 71%, 74% and 76%. On the left, the axis starts at 70, so Pro's bar looks six times as tall as Basic's. On the right, starting at zero, the real difference, 5 points, looks like what it is: modest.

Bar charts must start at zero, because readers compare the bars' lengths. Line charts may zoom in, because readers look at the shape of the line, but say so clearly on the axis.

Small Multiples

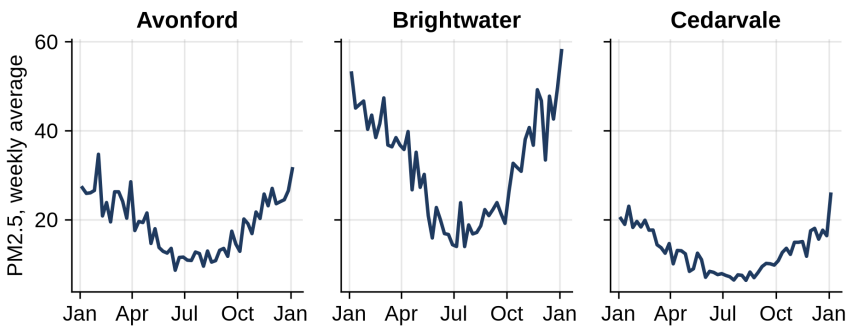
When you want to compare the same measure across groups, draw one small chart per group, with the same scales:

```
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import pandas as pd

air = pd.read_csv("../data/air_quality.csv", parse_dates=["date"])
weekly = air.set_index("date").groupby("city")["pm25"].resample("W").mean()

cities = ["Avonford", "Brightwater", "Cedarvale"]
fig, axes = plt.subplots(1, 3, figsize=(6.4, 2.6), sharey=True)
for ax, city in zip(axes, cities):
    ax.plot(weekly[city].index, weekly[city])
    ax.set_title(city)
    ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=[1, 4, 7, 10]))
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b"))
axes[0].set_ylabel("PM2.5, weekly average")
fig.tight_layout()
fig.savefig("small_multiples.png")
print("Saved small_multiples.png")
```

```
Saved small_multiples.png
```



Three small line charts of weekly PM2.5 in Avonford, Brightwater and Cedarvale, sharing one y-axis: all three peak in winter and dip in summer; Brightwater is the most polluted throughout.

`sharey=True` gives all three charts the same y-axis, so their heights can be compared directly: Brightwater is the most polluted city all year, and all three follow the same winter peak. One chart with three overlapping noisy lines would be much harder to read.

Include All the Data

Real-World Story: The night before the Space Shuttle Challenger launched in January 1986, engineers discussed whether cold weather could damage the rubber O-ring seals in the rocket boosters. One chart they discussed showed only the past flights *with* O-ring damage, plotted against temperature, and on its own it showed no clear pattern. Plotting *all* flights, including those without damage, shows that every launch below about 65°F had damage, and the forecast that morning was far colder. The presidential commission that investigated the accident criticized how the data had been analyzed; the statistician Edward Tufte later made the chart a famous lesson in his book *Visual Explanations*, though some historians argue his account simplifies the engineers' debate. The lesson stands: decide which data belongs in a chart before you look at the result, and include the cases where "nothing happened".

A Checklist for Honest Charts

- Bar axes start at zero; any zoomed axis is clearly labeled.
- The title says what the chart shows, and the axes have units.
- Comparable charts share scales.
- Nothing is left out that would change the picture, including "nothing happened" cases.
- Colors mean something, and the chart still works in black and white.
- One message per chart. If it needs a paragraph to explain, split it.

Key Takeaways

- Pick the chart for the job: lines for time, bars for comparison, histograms for spread, scatter for relationships.
- Bar charts start at zero; truncated axes exaggerate differences.
- Small multiples with shared scales compare groups better than one crowded chart.
- Include all relevant data, including cases where nothing happened.

CHAPTER 16 · INTERMEDIATE

Describing Data With Statistics

Statistics gives you a vocabulary for "typical", "spread out", "unusual" and "related". You don't need formulas to use it well; you need to know what each measure tells you, and when it misleads. This chapter covers the measures analysts use every day, and two famous traps.

Typical Values and Spread

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
value = orders["quantity"] * orders["unit_price"] * (1 - orders["discount"])

q1, q3 = value.quantile([0.25, 0.75])
print(f"mean     {value.mean():8.2f}")
print(f"median   {value.median():8.2f}")
print(f"std      {value.std():8.2f}")
print(f"IQR      {q3 - q1:8.2f}  (middle half: {q1:.0f} to {q3:.0f})")
print(f"skew     {value.skew():8.2f}  (positive: a long right tail)")
print(f"top 1%   {value.quantile(0.99):8.2f} and above")
```

```
mean        65.10
median      49.50
std         57.67
IQR         51.20  (middle half: 29 to 80)
skew         2.94  (positive: a long right tail)
top 1%     288.00 and above
```

- The **mean** (average) order is $65.10, but the **median**, the middle value when orders are sorted, is $49.50. Half the orders are worth less than $49.50.
- The **standard deviation** ($57.67) measures how far values typically are from the mean. It's large here because of the long tail of big orders.
- The **interquartile range** (IQR) is the width of the middle half of the values, from the 25th to the 75th percentile: $29 to $80. It ignores the extremes, so it's a steadier measure of spread for skewed data.
- **Skew** is positive (2.94): the tail stretches to the right, as the histogram in Chapter 14 showed.
- The top 1% of orders are worth $288 or more.

> **Tip:** For money, waiting times and anything with a long tail, report the median (and percentiles) rather than, or alongside, the mean. "The typical order is $50" is more honest than "the average order is $65" when a few large orders pull the average up.

Correlation, and Looking for Delays

Correlation (written *r*) measures how closely two measures move together in a straight-line way: +1 is a perfect rising line, 0 no linear relationship, -1 a perfect falling line.

```
import pandas as pd

air = pd.read_csv("../data/air_quality.csv", parse_dates=["date"])
visits = pd.read_csv("../data/clinic_visits.csv", parse_dates=["date"])
city = (
    air.merge(visits, on=["date", "city"])
    .query("city == 'Brightwater'")
    .set_index("date")
)

for lag in range(5):
    r = city["visits"].corr(city["pm25"].shift(lag))
    print(f"pollution {lag} days earlier: r = {r:.2f}")
```

```
pollution 0 days earlier: r = 0.43
pollution 1 days earlier: r = 0.50
pollution 2 days earlier: r = 0.76
pollution 3 days earlier: r = 0.48
pollution 4 days earlier: r = 0.43
```

The program correlates daily clinic visits in Brightwater with pollution on the same day, the day before, two days before, and so on, using `shift`. Same-day correlation is modest (0.43), but pollution two days earlier gives a strong 0.76. The effect takes time to show up, and you'd miss it entirely by only looking at the same day.

Correlation is a starting point, not a conclusion. It doesn't prove that one thing causes the other: both might be driven by a third factor, such as the season. Here the data was generated with a two-day delay built in, so you know the answer; with real data, a result like this is a reason to look closer, not proof.

Simpson's Paradox

Sometimes a comparison reverses when you split the data into groups:

```
import pandas as pd

# Two support teams; tickets are easy or hard.
tickets = pd.DataFrame(
    {
        "team": ["Red", "Red", "Blue", "Blue"],
        "kind": ["easy", "hard", "easy", "hard"],
        "tickets": [90, 10, 20, 80],
        "solved": [81, 5, 19, 48],
    }
)

overall = tickets.groupby("team")[["solved", "tickets"]].sum()
print((overall["solved"] / overall["tickets"]).round(2))
print()
tickets["rate"] = (tickets["solved"] / tickets["tickets"]).round(2)
print(tickets.pivot(index="kind", columns="team", values="rate"))
```

```
team
Blue    0.67
Red     0.86
dtype: float64

team  Blue  Red
kind
easy  0.95  0.9
hard  0.60  0.5
```

Overall, the Red team solves 86% of its tickets and the Blue team 67%. Red looks better. But split by difficulty, Blue is better at *both* kinds: 95% against 90% for easy tickets, and 60% against 50% for hard ones. The reversal happens because Blue handles mostly hard tickets (80 of its 100), which have lower success rates for everyone. Comparing overall rates mixes "how good is the team" with "how hard is its work".

Real-World Story: In 1973, the University of California, Berkeley, admitted about 44% of male applicants to its graduate programs but only about 35% of female applicants. When statisticians Peter Bickel, Eugene Hammel and J. W. O'Connell looked department by department, they found that women did as well as or better than men in most departments. Women had applied in larger numbers to the departments that were hardest to get into. Their 1975 paper in *Science* made this one of the best-known examples of Simpson's paradox.

The defense: whenever you compare groups, ask whether they differ in some other way that affects the outcome, and compare within like-for-like subgroups.

Look at the Data, Not Just the Numbers

Interesting Fact: In 1973, the statistician Francis Anscombe published four small datasets, now called **Anscombe's quartet**, that have almost identical means, variances and correlations, yet look completely different when plotted: one is a neat line, one a curve, and two are thrown off by a single outlier. Summary statistics can hide very different realities. Always chart your data before trusting a summary.

Exercises

1. **Coffee spending.** Describe the household's cafe purchases in `budget.csv`: how many, and the mean, median and spread of the amounts. (The amounts are negative; flip their sign first.)
2. **Two ratings.** How strongly are the survey's satisfaction and recommend scores correlated?

Key Takeaways

- Report medians and percentiles for skewed data; the mean is pulled by long tails.
- The IQR measures spread without being thrown by extremes.
- Correlation measures linear association; check delayed effects with `shift`, and never read it as proof of cause.
- Simpson's paradox: overall comparisons can reverse within groups. Compare like with like.
- Chart the data before trusting any summary.

CHAPTER 17 · INTERMEDIATE

Comparing Groups and A/B Tests

Is the new checkout page better, or did it just get lucky? Is one region's higher average order real, or random noise? When you compare groups, some difference almost always appears by chance. This chapter shows two simple, computer-powered ways to tell real differences from noise: the **bootstrap** and the **permutation test**. Neither needs any formulas, only a loop and a random number generator.

The Experiment

Larkspur Home tested a new checkout page. Each visitor was randomly shown the old page (A) or the new one (B), and the site recorded whether the visit ended in a purchase:

```
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")

summary = sessions.groupby("variant").agg(
    sessions=("session", "count"),
    conversions=("converted", "sum"),
    revenue=("revenue", "sum"),
)
summary["rate"] = summary["conversions"] / summary["sessions"]
summary["revenue_per_session"] = summary["revenue"] / summary["sessions"]
print(summary.round(4))
lift = summary.loc["B", "rate"] / summary.loc["A", "rate"] - 1
print(f"B converts {lift:.1%} better than A")
```

```
         sessions  conversions   revenue    rate  revenue_per_session
variant
A           10024          402  27372.98  0.0401               2.7307
B            9976          476  31845.27  0.0477               3.1922
B converts 19.0% better than A
```

About 10,000 sessions each. A converted 4.01% of sessions and B 4.77%: B converts 19% better, and earns more per session. But with only about 400 to 480 conversions per group, how sure can we be that B is really better?

The Bootstrap: How Precise Is the Difference?

The **bootstrap** asks: if we ran the experiment again, how much would the result move? It imitates rerunning the experiment by resampling the data you have:

```
import numpy as np
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")
a = sessions.loc[sessions["variant"] == "A", "converted"].to_numpy()
b = sessions.loc[sessions["variant"] == "B", "converted"].to_numpy()

rng = np.random.default_rng(1)
diffs = np.empty(5_000)
for i in range(len(diffs)):
    diffs[i] = rng.choice(b, len(b)).mean() - rng.choice(a, len(a)).mean()

low, high = np.percentile(diffs, [2.5, 97.5])
print(f"difference in conversion rate: {b.mean() - a.mean():.4f}")
print(f"95% bootstrap interval: {low:.4f} to {high:.4f}")
print("interval excludes zero:", bool(low > 0 or high < 0))
```

```
difference in conversion rate: 0.0076
95% bootstrap interval: 0.0020 to 0.0132
interval excludes zero: True
```

Each loop draws a new sample from each group *with replacement* (the same session can be picked more than once), the same size as the original, and records the difference in conversion rates. After 5,000 repetitions, the middle 95% of those differences forms a **95% interval**: here 0.20 to 1.32 percentage points. The interval doesn't include zero, so the data supports B being genuinely better. It also shows how uncertain the size of the improvement is: somewhere from a little to a lot.

Interesting Fact: The bootstrap was introduced by the statistician Bradley Efron in 1979. The name comes from the phrase "to pull oneself up by one's bootstraps": the method gets an estimate of its own uncertainty from nothing but the data itself. It only became practical once computers could repeat a calculation thousands of times in seconds.

The Permutation Test: Could It Be Luck?

The **permutation test** asks a different question: if the page made no difference at all, how often would we see a gap this big just by chance?

```
import numpy as np
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")
converted = sessions["converted"].to_numpy()
is_b = (sessions["variant"] == "B").to_numpy()
observed = converted[is_b].mean() - converted[~is_b].mean()

rng = np.random.default_rng(2)
count = 0
trials = 5_000
for _ in range(trials):
    shuffled = rng.permutation(is_b)  # pretend the labels mean nothing
    diff = converted[shuffled].mean() - converted[~shuffled].mean()
    count += abs(diff) >= abs(observed)

print(f"observed difference: {observed:.4f}")
print(f"p-value: {count / trials:.3f}")
```

```
observed difference: 0.0076
p-value: 0.010
```

If the page doesn't matter, the A and B labels are meaningless, so shuffling them shouldn't change anything. The program shuffles the labels 5,000 times and counts how often the shuffled difference is at least as large as the real one (0.76 points). That share is the **p-value**: 0.010, or 1%. A gap this large would rarely happen by luck alone.

> **Warning:** A small p-value says the difference is unlikely to be pure chance. It does *not* say the difference is large or important, and a large p-value doesn't prove there's no difference, only that this data can't show one. Report the size of the effect with its interval, not just the p-value.

Good Habits for Experiments

- **Decide in advance** what you'll measure, for how long, and what difference matters. Stopping the test the moment it "looks significant" inflates false positives.
- **Randomize** who sees which version. Comparing this month's new page with last month's old page mixes in everything else that changed.
- **Check the split.** If A got noticeably more traffic than B, something is broken in the assignment (Chapter 23 shows how to check).
- **Look at segments**, such as mobile and desktop, but be cautious: test enough segments and one will look special by chance.

Exercises

1. **Mobile only.** Compute a 95% bootstrap interval for the difference in conversion rate (B minus A) using only mobile sessions. Does it include zero?

Key Takeaways

- Random variation creates differences; check whether yours is bigger than chance would produce.
- The bootstrap resamples your data to estimate a 95% interval for a difference.
- The permutation test shuffles labels to estimate a p-value: how often chance alone gives a gap this big.
- Report effect sizes with intervals, decide on the analysis before the data arrives, and randomize.

CHAPTER 18 · INTERMEDIATE

SQL for Analysts

SQL (Structured Query Language, often pronounced "sequel") is the language of databases, and almost every analyst job asks for it. The good news: if you understand pandas filtering, grouping and merging, you already understand SQL's ideas. This chapter teaches the SQL you'll use daily, running against SQLite from Python, with the pandas equivalent of each piece.

A Database in Memory

```
import sqlite3

import pandas as pd

conn = sqlite3.connect(":memory:")  # a database that lives in memory
pd.read_csv("../data/orders.csv").to_sql("orders", conn, index=False)
pd.read_csv("../data/products.csv").to_sql("products", conn, index=False)


def sql(query):
    print(pd.read_sql(query, conn))
    print()


sql("""
SELECT category, COUNT(*) AS orders, ROUND(AVG(quantity), 2) AS avg_qty
FROM orders
WHERE channel = 'app'
GROUP BY category
HAVING COUNT(*) > 300
ORDER BY orders DESC
""")

sql("""
```

```
SELECT o.product,
       ROUND(SUM(o.quantity * (o.unit_price - p.unit_cost)), 0) AS profit
FROM orders AS o
JOIN products AS p ON p.product = o.product
GROUP BY o.product
ORDER BY profit DESC
LIMIT 3
""")

sql("""
SELECT * FROM (
  SELECT region, product, SUM(quantity) AS units,
         RANK() OVER (PARTITION BY region
                      ORDER BY SUM(quantity) DESC) AS rnk
  FROM orders
  GROUP BY region, product
) WHERE rnk = 1
""")
```

```
   category  orders  avg_qty
0   Bedding     438     1.56
1   Kitchen     361     1.63
2  Lighting     346     1.64
3    Garden     319     1.58
4     Decor     303     1.56

         product   profit
0     Floor Lamp  37749.0
1    Duvet Cover  24677.0
2  Cast Iron Pan  17515.0

  region        product  units  rnk
0   East     Chef Knife    191    1
1  North    Pillow Pair    232    1
2  South  Throw Blanket    170    1
3   West     Chef Knife    191    1
```

`sqlite3.connect(":memory:")` creates a database that lives only while the program runs, perfect for practice. `to_sql` loads the orders and products into tables, and the small `sql` helper runs a query and prints the result.

Reading a Query

The first query, clause by clause, in the order the database actually works through them:

Clause	Meaning	pandas equivalent
`FROM orders`	Start from this table	`orders`
`WHERE channel = 'app'`	Keep matching rows	`orders[orders["channel"] == "app"]`
`GROUP BY category`	Split into groups	`.groupby("category")`
`SELECT ..., COUNT(*), AVG(quantity)`	Calculate per group	`.agg(...)`
`HAVING COUNT(*) > 300`	Keep matching groups	filter the grouped result
`ORDER BY orders DESC`	Sort	`.sort_values(..., ascending=False)`

`WHERE` filters rows *before* grouping; `HAVING` filters groups *after*. Every category had more than 300 app orders, with Bedding first.

Joins

The second query joins orders to products to compute profit per product:

- `JOIN products AS p ON p.product = o.product` matches rows like `merge(on="product")`. `AS o` and `AS p` give the tables short names.
- `JOIN` alone is an inner join; `LEFT JOIN` keeps every row of the first table, like `how="left"`.
- `LIMIT 3` keeps the first three rows, like `head(3)`.

Floor lamps earn the most profit, $37,749, followed by duvet covers and cast iron pans.

Window Functions

The third query finds the best-selling product in each region. A **window function** calculates across a group of rows without collapsing them:

- `RANK() OVER (PARTITION BY region ORDER BY SUM(quantity) DESC)` ranks products within each region, largest first. `PARTITION BY` is like `groupby` for the ranking.
- The outer query keeps rank 1.

In the West and the East, chef knives sell the most items; in the North pillow pairs, and in the South throw blankets. The pandas equivalent is `groupby(...).rank()` or the sort-then-`head(1)` pattern from Chapter 6.

> **Tip:** Common window functions: `ROW_NUMBER()`, `RANK()`, `SUM(...) OVER (...)` for running totals, `LAG(column)` for the previous row's value (like `shift`), and `AVG(...) OVER (... ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` for rolling averages.

SQL or pandas?

Use SQL when the data lives in a database: let the database do the filtering and grouping, and bring back only the small result. A query that returns 12 monthly totals is much faster than pulling 50 million rows into Python to total them. Use pandas for everything after that: reshaping, statistics, charts and anything awkward in SQL. Most analysts use both in the same project, often in the same program.

> **Warning:** Never build a query by pasting user input into the SQL text, as in `f"... WHERE region = '{region}'"`. A malicious or unlucky value can change the query (**SQL injection**). Pass values as parameters instead: `pd.read_sql("... WHERE region = ?", conn, params=[region])`.

Exercises

1. **Busiest months.** Write a query that counts orders per month (SQLite's `strftime('%m', order_date)` gives the month) and lists the three busiest.
2. **Segments.** Join orders to customers and report, for each segment, the number of distinct buyers and the average quantity per order.

Key Takeaways

- SQL's `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY` and `JOIN` map directly onto pandas filtering, grouping, sorting and merging.
- `WHERE` filters rows before grouping; `HAVING` filters groups after.
- Window functions such as `RANK() OVER (PARTITION BY ...)` rank and total within groups without collapsing rows.
- Let the database reduce the data; use pandas for the rest. Always pass values as parameters.

CHAPTER 19 · ADVANCED

Reports, Automation and Testing Your Analysis

An analysis that only runs on your laptop, by hand, once, is fragile. The habits in this chapter make your work repeatable and trustworthy: put calculations in functions, test those functions, and generate the report with a program so next month's report is one command away.

Reusable Calculations

Calculations you use in several places belong in a module, written once:

```
"""Reusable calculations for the shop reports."""

import pandas as pd


def add_revenue(orders: pd.DataFrame) -> pd.DataFrame:
    """Return a copy with a revenue column: quantity x price, less
    the discount."""
    out = orders.copy()
    out["revenue"] = (
        out["quantity"] * out["unit_price"] * (1 - out["discount"])
    )
    return out


def top_products(orders: pd.DataFrame, n: int = 3) -> pd.Series:
    """The n products with the most revenue, largest first."""
    revenue = add_revenue(orders).groupby("product")["revenue"].sum()
    return revenue.nlargest(n)
```

`add_revenue` works on a copy, so it never changes the caller's DataFrame, and its docstring says exactly what it calculates. `top_products` builds on it. Type hints (`pd.DataFrame`, `pd.Series`) document what goes in and out.

Testing an Analysis

The calculations that feed a report deserve tests, just like any other code. **pytest** runs every function whose name starts with `test_`:

```
import pandas as pd
import pytest

from metrics import add_revenue, top_products

SAMPLE = pd.DataFrame(
    {
        "product": ["Lamp", "Mug", "Lamp"],
        "quantity": [2, 1, 1],
        "unit_price": [40.0, 10.0, 40.0],
        "discount": [0.0, 0.5, 0.1],
    }
)


def test_revenue_applies_discount():
    assert add_revenue(SAMPLE)["revenue"].tolist() == [80.0, 5.0, 36.0]


def test_input_is_not_changed():
    add_revenue(SAMPLE)
    assert "revenue" not in SAMPLE.columns


def test_top_products_order():
    top = top_products(SAMPLE, n=2)
    assert list(top.index) == ["Lamp", "Mug"]
    assert top["Lamp"] == pytest.approx(116.0)
```

Run it from the chapter folder:

```
$ python -m pytest -q test_metrics.py
```

```
...                                                                  [100%]
3 passed in 0.01s
```

The tests use a tiny table where you can work out the right answers by hand: 2 lamps at $40 is $80; a $10 mug at half price is $5; a $40 lamp with 10% off is $36. They also check that the input isn't changed, a classic source of silent

bugs, and that the top products come out in the right order. `pytest.approx` compares decimals safely, since decimal arithmetic can be off in the last digit.

> **Tip:** When an analysis gives a surprising number, write a test with a tiny example before you trust it. If the test passes, the surprise is probably in the data, which is exactly what you want to know.

A Report From a Program

```
from datetime import date

import matplotlib.pyplot as plt
import pandas as pd

from metrics import add_revenue, top_products

orders = add_revenue(
    pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
)
monthly = orders.set_index("order_date")["revenue"].resample("MS").sum()

fig, ax = plt.subplots(figsize=(6.4, 2.8))
ax.plot(monthly.index.strftime("%b"), monthly / 1000, marker="o")
ax.set_ylabel("$ thousands")
ax.set_title("Revenue by month")
fig.tight_layout()
fig.savefig("report_revenue.png")

best = monthly.idxmax().strftime("%B")
lines = [
    "# Larkspur Home: 2025 sales report",
    f"Generated {date(2026, 1, 5):%d %B %Y}.",
    "",
    f"- Revenue: ${orders['revenue'].sum():,.0f}"
    f" from {len(orders):,} orders",
    f"- Best month: {best} (${monthly.max():,.0f})",
    f"- Average order: ${orders['revenue'].mean():.2f}",
    "",
    "## Top products",
    "",
]
for product, revenue in top_products(orders).items():
    lines.append(f"- {product}: ${revenue:,.0f}")
lines += ["", "![Revenue by month](report_revenue.png)"]

with open("report.md", "w") as f:
    f.write("\n".join(lines) + "\n")
print(open("report.md").read())
```

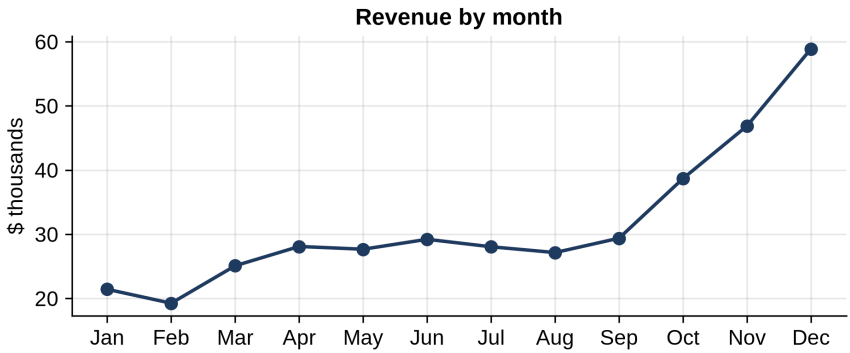
```
# Larkspur Home: 2025 sales report
Generated 05 January 2026.

- Revenue: $379,703 from 5,833 orders
- Best month: December ($58,888)
- Average order: $65.10

## Top products

- Floor Lamp: $69,156
- Duvet Cover: $47,971
- Cast Iron Pan: $36,798

![Revenue by month](report_revenue.png)
```



The report's chart: monthly revenue from January to December 2025, flat around $20,000 to $30,000 for most of the year, rising steeply to about $59,000 in December.

The program loads the data, uses the tested functions, draws a chart, and writes a short report in **Markdown**, a plain-text format with `#` for headings and `-` for lists that many tools display nicely and can convert to a web page or a document. Every number in the report comes from the data, so it can't be mistyped, and next month you rerun one command. (The report's date is fixed so the book's output never changes; in your own reports, use `date.today()`.)

Making It Repeatable

- **One command** reproduces everything from the raw data. If you do anything by hand, write it down, then automate it.
- **Raw data is read-only.** Never edit the original file; cleaning happens in code.

- **Pin versions** in a `requirements.txt` file, as Chapter 2 did, so the analysis gives the same answer next year.
- **Keep code in version control** (such as Git), so you can see what changed when a number changes.
- **Schedule it** with your operating system's scheduler (cron on macOS and Linux, Task Scheduler on Windows) to run reports automatically.

Key Takeaways

- Put calculations in functions with clear names and docstrings; never change the caller's data.
- Test analysis code with tiny hand-checkable examples using pytest.
- Generate reports from code, so every number comes from the data and next month is one command away.
- Keep raw data untouched, pin library versions, and use version control.

CHAPTER 20 · ADVANCED

Using AI Assistants for Analysis, Safely

AI assistants that write code can speed up analysis a lot: they remember pandas syntax you've forgotten, draft charts, explain errors and suggest approaches. They can also produce code that runs perfectly and answers the wrong question. This chapter shows how to use them well: what to ask, what to check, and what never to paste in.

What AI Assistants Are Good At

- **Syntax and recipes:** "How do I resample to weeks ending on Monday?", "How do I label bars in matplotlib?"
- **Explaining errors:** paste the error message and the few lines around it.
- **First drafts:** a chart, a cleaning step, a SQL query, which you then read and test.
- **Explaining code:** "Explain this groupby line by line."
- **Ideas:** "What are three ways to check whether this A/B test split is fair?"

They're weakest at things that depend on *your* data and *your* question: what a column really means, whether the answer is right, and which comparison is fair.

A Plausible, Wrong Answer

Asked for "average order value by region", an assistant might suggest the first calculation below. It runs, and gives sensible-looking numbers:

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["revenue"] = orders["quantity"] * orders["unit_price"]

# Suggested by an AI assistant for "average order value by region":
per_customer = orders.groupby(["region", "customer_id"])["revenue"].mean()
suggested = per_customer.groupby("region").mean()

# What the question actually asks: total revenue / number of orders
correct = (
    orders.groupby("region")["revenue"].sum()
    / orders.groupby("region")["order_id"].count()
)

print(pd.DataFrame({"suggested": suggested, "correct": correct}).round(2))
```

```
        suggested  correct
region
East        66.31    66.46
North       67.28    67.10
South       69.08    69.22
West        68.73    70.63
```

The suggested code averages each customer's orders first, then averages the customers. That's "the average customer's average order", a different question: it gives a customer with one order as much weight as one with twenty. Average order value is total revenue divided by the number of orders. The numbers are close, which is exactly what makes the mistake dangerous: in the West the two differ by almost $2, and nothing looks wrong.

The fix isn't to stop using assistants. It's to know your definitions, and to check.

Checks That Catch Most Mistakes

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["revenue"] = orders["quantity"] * orders["unit_price"]
by_region = orders.groupby("region")["revenue"].sum()

# Cheap checks that catch most wrong answers, from people or AI
```

```
assert abs(by_region.sum() - orders["revenue"].sum()) < 0.01, (
    "parts != whole"
)
assert len(by_region) == orders["region"].nunique(), "a region went missing"
assert (by_region > 0).all(), "a region has no revenue"
print("totals add up; all regions present and positive")
print(by_region.round(0).to_string())
```

```
totals add up; all regions present and positive
region
East     103671.0
North    118630.0
South     89987.0
West      85106.0
```

- **Parts add up to the whole.** Regional totals should sum to the overall total.
- **Nothing went missing.** The number of groups matches the number of distinct values.
- **Values are plausible.** Positive revenue, dates in range, shares between 0 and 1.
- **A tiny example by hand.** Compute the answer for three rows yourself and compare (Chapter 19).
- **A second method.** Compute the same number another way, such as SQL against pandas, or a loop against a vectorized version.

How to Ask

- **Describe the data:** column names, types and a few example rows (made up or anonymized, if the data is sensitive).
- **Define the metric exactly:** "average order value = total revenue / number of orders".
- **Ask for explanations:** "explain each step", "what assumptions does this make?"
- **Ask for checks:** "add assertions that would catch a wrong result".

What Never to Paste In

> **Warning:** Don't paste personal data (names, emails, addresses, health or financial details), confidential business data, passwords or access keys into an AI assistant unless your organization has approved that tool for that data. Describe the structure instead, or use a small made-up sample. Check your employer's policy first: many organizations have rules about which AI tools may see which data.

Key Takeaways

- AI assistants are excellent for syntax, error messages, drafts and explanations.
- Code that runs isn't code that's right: define your metrics exactly and check the results.
- Parts-add-up, nothing-missing, plausibility, tiny examples and a second method catch most errors.
- Never paste sensitive data into a tool your organization hasn't approved for it.

PART IV

Six Projects

CHAPTER 21 · PROJECT

Project: Retail Sales Review

It's January, and Larkspur Home's owner wants a review of 2025 before planning the new year. Her questions:

1. How much did we sell, and how much did we keep as profit?
2. Which categories are growing, and which are shrinking?
3. Are customers coming back?
4. Do discounts work?

This project answers each one with the skills from Part II, then puts the main results on one page of charts.

Revenue, Profit and Growth by Category

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
products = pd.read_csv("../data/products.csv")
df = orders.merge(
    products[["product", "unit_cost"]], on="product", validate="many_to_one"
)
df["revenue"] = df["quantity"] * df["unit_price"] * (1 - df["discount"])
df["profit"] = df["revenue"] - df["quantity"] * df["unit_cost"]

print(
    f"Revenue ${df['revenue'].sum():,.0f}, "
    f"profit ${df['profit'].sum():,.0f} "
    f"({df['profit'].sum() / df['revenue'].sum():.1%} margin)"
)
print()

h1 = df["order_date"].dt.month <= 6
```

```
growth = (
    df[~h1].groupby("category")["revenue"].sum()
    / df[h1].groupby("category")["revenue"].sum()
    - 1
)
cats = df.groupby("category").agg(
    revenue=("revenue", "sum"), profit=("profit", "sum")
)
cats["margin"] = cats["profit"] / cats["revenue"]
cats["H2 vs H1"] = growth
print(
    cats.sort_values("revenue", ascending=False).round(
        {"revenue": 0, "profit": 0, "margin": 3, "H2 vs H1": 3}
    )
)
```

```
Revenue $379,703, profit $170,916 (45.0% margin)

            revenue   profit  margin  H2 vs H1
category
Bedding   106166.0  47872.0   0.451     0.671
Lighting  105073.0  48944.0   0.466     0.764
Kitchen    82609.0  38081.0   0.461     0.812
Garden     48116.0  20162.0   0.419    -0.380
Decor      37739.0  15856.0   0.420     0.725
```

The shop took $379,703 and kept $170,916 after product costs: a 45% gross margin. (Gross margin ignores rent, staff, shipping and marketing, so it isn't the shop's final profit; say so in any report.)

Comparing the second half of the year with the first:

- **Kitchen, Lighting, Decor and Bedding grew by 67% to 81%**, mostly because of the holiday season.
- **Garden fell 38%.** That's not a problem with Garden: its season is spring. A naive "Garden is shrinking, cut it" conclusion would be wrong. Comparing H2 with H1 mixes the trend with the season; with two years of data, you'd compare each half with the same half a year earlier.
- **Lighting has the best margin**, 46.6%, and Garden and Decor the lowest, about 42%.

Customers

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
orders["revenue"] = (
    orders["quantity"] * orders["unit_price"] * (1 - orders["discount"])
)

per_customer = orders.groupby("customer_id").agg(
    orders=("order_id", "count"), revenue=("revenue", "sum")
)
print("customers who ordered:", len(per_customer))
print(
    "ordered more than once:", (per_customer["orders"] > 1).mean().round(3)
)
print()

ranked = per_customer["revenue"].sort_values(ascending=False)
top_fifth = ranked.head(len(ranked) // 5).sum() / ranked.sum()
print(f"top 20% of customers bring {top_fifth:.0%} of revenue")
print()

first = orders.groupby("customer_id")["order_date"].transform("min")
orders["new_or_returning"] = (orders["order_date"] == first).map(
    {True: "new", False: "returning"}
)
mix = (
    orders.groupby([orders["order_date"].dt.quarter, "new_or_returning"])[
        "revenue"
    ]
    .sum()
    .unstack()
)
print((mix.div(mix.sum(axis=1), axis=0)).round(2))
```

```
customers who ordered: 1191
ordered more than once: 0.961

top 20% of customers bring 40% of revenue

new_or_returning   new  returning
order_date
1                 0.67       0.33
2                 0.23       0.77
3                 0.10       0.90
4                 0.02       0.98
```

- 1,191 of the 1,200 customers ordered at least once, and 96% of those ordered more than once: a loyal base.
- The top fifth of customers bring 40% of revenue. That's concentrated, but far from the famous "80/20" pattern; this shop doesn't depend on a handful of big buyers.

- Each order is marked "new" if it's the customer's first of the year. In the first quarter, two-thirds of revenue came from first orders of the year; by the fourth quarter, 98% came from returning customers. (A dataset that starts in January makes everyone "new" at first, so the early quarters overstate new business. Say so, or use the customer's signup date instead.)

Do Discounts Work?

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
customers = pd.read_csv("../data/customers.csv")
df = orders.merge(customers[["customer_id", "segment"]], on="customer_id")

table = df.pivot_table(
    index="discount", columns="segment", values="quantity", aggfunc="mean"
)
print("average items per order:")
print(table.round(2))
print()
print(df.groupby("discount").size().rename("orders"))
```

```
average items per order:
segment   Business  Consumer
discount
0.0           2.34      1.40
0.1           2.63      1.59
0.2           2.82      1.83

discount
0.0    4084
0.1    1173
0.2     576
Name: orders, dtype: int64
```

Orders with a discount contain more items: from 1.40 to 1.83 items for consumers as the discount rises from 0 to 20%, and from 2.34 to 2.82 for business customers. Splitting by segment matters (remember Simpson's paradox, Chapter 16): business customers buy more per order *and* could receive discounts at a different rate, so mixing them would blur the comparison.

But "more items" isn't "more profit". A 20% discount on a 45% margin gives away almost half the profit on each item; the extra items would need to more than make up for it. And this is observational data: discounts weren't randomly assigned, so customers who got them may differ in other ways. The

honest conclusion: discounts are associated with larger orders; whether they pay for themselves needs a proper experiment, like the one in Chapter 23.

One Page of Charts

```
import matplotlib.pyplot as plt
import pandas as pd

orders = pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
orders["revenue"] = (
    orders["quantity"] * orders["unit_price"] * (1 - orders["discount"])
)
monthly = orders.set_index("order_date")["revenue"].resample("MS").sum()
by_cat = orders.groupby("category")["revenue"].sum().sort_values()
by_channel = orders.groupby("channel")["revenue"].sum().sort_values()
weekday = orders.groupby(orders["order_date"].dt.dayofweek)["order_id"]
weekday = weekday.count() / 52

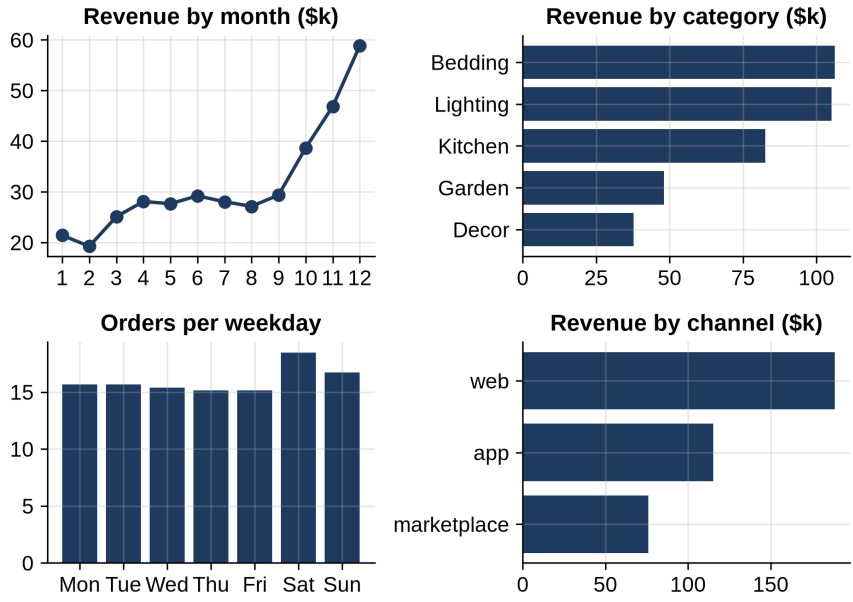
fig, axes = plt.subplots(2, 2, figsize=(6.4, 4.6))
axes[0, 0].plot(monthly.index.month, monthly / 1000, marker="o")
axes[0, 0].set_title("Revenue by month ($k)")
axes[0, 0].set_xticks(range(1, 13))
axes[0, 1].barh(by_cat.index, by_cat / 1000)
axes[0, 1].set_title("Revenue by category ($k)")
axes[1, 0].bar(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"], weekday)
axes[1, 0].set_title("Orders per weekday")
axes[1, 1].barh(by_channel.index, by_channel / 1000)
axes[1, 1].set_title("Revenue by channel ($k)")
fig.tight_layout()
fig.savefig("dashboard.png")
print("Saved dashboard.png")
```

```
Saved dashboard.png
```

`plt.subplots(2, 2)` returns a 2 by 2 grid of axes, addressed as `axes[row, column]`. Four small charts answer the four questions a manager asks first: when, what, which day, and through which channel.

What to Tell the Owner

- 2025 revenue was $380,000 with a 45% gross margin; December alone brought in about three times February's sales.
- Lighting and Bedding lead in both revenue and growth; Garden is seasonal, not shrinking.
- Customers return: 96% ordered more than once.



A two-by-two dashboard for 2025: revenue by month rising toward December; revenue by category with Bedding and Lighting largest; orders per weekday, highest on Saturday; and revenue by channel, with the website largest.

- Discounts go with bigger orders, but whether they're profitable needs a test, not this data.

Your Turn

- Which products have the highest return rate of customers (bought again within 60 days)?
- Is the app's share of orders growing through the year?
- Build the same review for one region, and compare it with the whole shop.

Key Takeaways

- Start from the owner's questions, and answer each with a small, focused program.

- Seasonality can masquerade as growth or decline; compare like periods.
- Watch for artifacts of the data window, such as "new" customers in January.
- Observational associations, such as discounts and order size, aren't proof of cause.

CHAPTER 22 · PROJECT

Project: Customer Survey

In September, 800 customers answered a short survey: how satisfied they are (1 to 5), how likely they are to recommend the service (0 to 10), and an optional comment. The product team wants to know which plan's customers are happiest, what the unhappy ones complain about, and one number to track over time.

Net Promoter Score

Interesting Fact: The **Net Promoter Score** (NPS) was introduced by Frederick Reichheld in a 2003 *Harvard Business Review* article titled "The One Number You Need to Grow". Customers who answer 9 or 10 to "How likely are you to recommend us?" are promoters, 7 or 8 passives, and 0 to 6 detractors. NPS is the percentage of promoters minus the percentage of detractors, from -100 to +100. It's popular because it's simple, though later research questioned how well it predicts growth, so treat it as one measure among several.

```
import pandas as pd

survey = pd.read_csv("../data/survey.csv", keep_default_na=False)

survey["group"] = pd.cut(
    survey["recommend"],
    bins=[-1, 6, 8, 10],
    labels=["detractor", "passive", "promoter"],
)
shares = pd.crosstab(survey["plan"], survey["group"], normalize="index")
shares["NPS"] = 100 * (shares["promoter"] - shares["detractor"])
print(shares.round(2).reindex(["Basic", "Plus", "Pro"]))
```

```
print()
overall = survey["group"].value_counts(normalize=True)
print(
    f"overall NPS: {100 * (overall['promoter'] - overall['detractor']):.0f}"
)
print(survey.groupby("plan")["satisfaction"].mean().round(2).to_string())
```

```
group  detractor  passive  promoter    NPS
plan
Basic       0.51     0.32      0.18 -33.08
Plus        0.32     0.38      0.31  -1.26
Pro         0.21     0.37      0.42  21.43

overall NPS: -12
plan
Basic    3.22
Plus     3.71
Pro      4.02
```

`pd.cut` with bin edges `[-1, 6, 8, 10]` sorts each answer into the three groups (the first bin starts at -1 so that 0 is included). `crosstab` with `normalize="index"` gives each plan's mix, and NPS follows directly.

The plans are very different:

- **Basic:** half the customers are detractors; NPS is -33.
- **Plus:** about even; NPS -1.
- **Pro:** promoters outnumber detractors two to one; NPS +21.

Overall NPS is -12, but that single number hides the real story: the cheapest plan is pulling the average down. Average satisfaction tells the same story more gently: 3.22, 3.71 and 4.02 out of 5.

What the Comments Say

```
import pandas as pd

survey = pd.read_csv("../data/survey.csv", keep_default_na=False)
themes = {
    "speed": r"slow|fast",
    "price": r"price|value",
    "support": r"support",
    "ease of use": r"easy|confusing",
}
low = survey[survey["satisfaction"] <= 2]
high = survey[survey["satisfaction"] >= 4]

rows = []
for theme, pattern in themes.items():
```

```
    rows.append(
        {
            "theme": theme,
            "unhappy (1-2)": low["comment"].str.contains(pattern).mean(),
            "happy (4-5)": high["comment"].str.contains(pattern).mean(),
        }
    )
print(pd.DataFrame(rows).set_index("theme").round(2))
print()
print("blank comments:", (survey["comment"] == "").sum())
```

```
             unhappy (1-2)  happy (4-5)
theme
speed                 0.55         0.25
price                 0.21         0.28
support               0.21         0.22
ease of use           0.23         0.24

blank comments: 31
```

The program groups comments into themes with regular expressions, then compares how often each theme comes up among unhappy (1 or 2) and happy (4 or 5) customers. Speed stands out: 55% of unhappy customers mention it ("too slow", "support was slow"), against 25% of happy ones (who mention "fast delivery"). Price, support and ease of use come up at similar rates in both groups, so they don't explain unhappiness here. 31 people left no comment.

A theme count is only a first pass. Before reporting, read a sample of real comments in each theme: keyword matching can't tell "support was slow" from "great support", which is why this program counts themes, not sentiment.

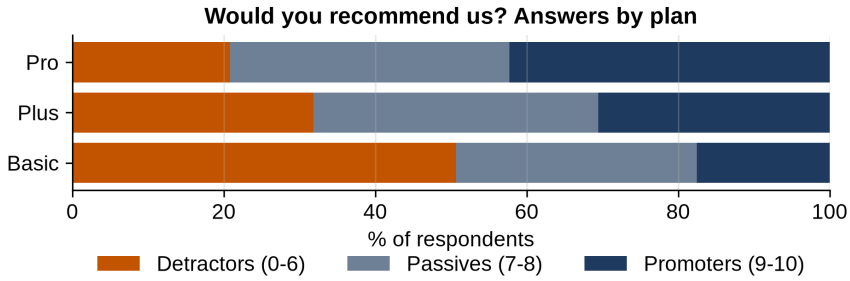
The Chart

```
import matplotlib.pyplot as plt
import pandas as pd

survey = pd.read_csv("../data/survey.csv")
survey["group"] = pd.cut(
    survey["recommend"],
    bins=[-1, 6, 8, 10],
    labels=["Detractors (0-6)", "Passives (7-8)", "Promoters (9-10)"],
)
shares = (
    pd.crosstab(survey["plan"], survey["group"], normalize="index").reindex(
        ["Basic", "Plus", "Pro"]
    )
    * 100
)
```

```
fig, ax = plt.subplots(figsize=(6.4, 2.6))
left = pd.Series(0.0, index=shares.index)
for group, color in zip(shares.columns, ["C1", "C2", "C0"]):
    ax.barh(
        shares.index, shares[group], left=left, color=color, label=group
    )
    left += shares[group]
ax.set_xlim(0, 100)
ax.set_xlabel("% of respondents")
ax.set_title("Would you recommend us? Answers by plan")
ax.legend(ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.3))
ax.grid(axis="y", visible=False)
fig.tight_layout()
fig.savefig("survey_chart.png")
print("Saved survey_chart.png")
```

```
Saved survey_chart.png
```



Stacked horizontal bars for the Basic, Plus and Pro plans, each split into detractors, passives and promoters: detractors shrink and promoters grow from Basic to Pro.

A **stacked bar** shows parts of a whole: each bar is 100% of a plan's respondents. The program stacks them by hand, keeping a running `left` position for where each segment starts. Detractors shrink and promoters grow from Basic to Pro, the whole story in one picture.

What to Tell the Product Team

- NPS is -12 overall, driven by the Basic plan (-33); Pro customers are clearly happy (+21).
- Slowness, of the app and of support, is the main complaint among unhappy customers.

- Suggested next step: investigate speed for Basic customers, then rerun the survey to track NPS by plan.

> **Common Mistake:** Reporting averages of survey scales without the distribution. An average satisfaction of 3.5 could mean everyone is lukewarm, or that half love you and half don't. Show the shares.

Your Turn

- Do satisfaction scores differ by age group within each plan?
- Is there a difference between responses from the first and second half of September?

Key Takeaways

- NPS turns a 0 to 10 answer into promoters, passives and detractors; report it by segment, not just overall.
- Theme counting with regular expressions is a fast first pass at comments; read samples before concluding.
- Stacked bars show the makeup of each group; always show distributions, not just averages.

CHAPTER 23 · PROJECT

Project: Website A/B Test

Chapter 17 showed that the new checkout page (B) converts better than the old one (A). Before the team switches everyone to B, a careful analyst asks three more questions:

1. Was the experiment run correctly?
2. Does B work for everyone, or only for some visitors?
3. How much money is the improvement worth, and how sure are we?

Was the Split Fair?

```
import numpy as np
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")

# 1. Sample ratio: did the split really give each variant about half?
counts = sessions["variant"].value_counts()
rng = np.random.default_rng(3)
fair = rng.binomial(len(sessions), 0.5, 10_000)
gap = abs(counts["A"] - len(sessions) / 2)
p = (abs(fair - len(sessions) / 2) >= gap).mean()
print(counts.to_string())
print(f"split p-value: {p:.2f}")
print()

# 2. Is the effect the same on desktop and mobile?
rates = sessions.pivot_table(
    index="device", columns="variant", values="converted", aggfunc="mean"
)
rates["lift"] = rates["B"] / rates["A"] - 1
print(rates.round(4))
```

```
variant
A    10024
B     9976
split p-value: 0.75

variant       A       B    lift
device
desktop  0.0463  0.0537  0.1608
mobile   0.0349  0.0427  0.2226
```

Visitors were meant to be split 50/50. A got 10,024 and B 9,976. Is a gap of 24 from a perfect 10,000 normal? The program simulates 10,000 fair coin-flip splits of 20,000 visitors and counts how often the gap is at least as big: 75% of the time. The split is fine. A very small number here, a **sample ratio mismatch**, would mean the assignment was broken, for example if a bug sent some visitors to the wrong page, and then none of the results could be trusted.

Does It Work for Everyone?

The pivot table splits conversion by device. B is better on both desktop (16% lift) and mobile (22% lift). Mobile visitors convert less overall, but the new page helps them at least as much, so there's no group being hurt by the change.

> **Warning:** If you check enough segments (device, country, browser, new or returning, time of day), one will look special by chance. Decide in advance which segments matter, and treat surprises in others as ideas for the next test, not conclusions.

How Much Is It Worth?

```
import numpy as np
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")
a = sessions.loc[sessions["variant"] == "A", "revenue"].to_numpy()
b = sessions.loc[sessions["variant"] == "B", "revenue"].to_numpy()

rng = np.random.default_rng(4)
diffs = [
    rng.choice(b, len(b)).mean() - rng.choice(a, len(a)).mean()
    for _ in range(5_000)
]
low, high = np.percentile(diffs, [2.5, 97.5])
```

```
print(f"revenue per session: A ${a.mean():.3f}, B ${b.mean():.3f}")
print(
    f"difference ${b.mean() - a.mean():.3f}, "
    f"95% interval ${low:.3f} to ${high:.3f}"
)
monthly_sessions = 600_000
print(
    f"at {monthly_sessions:,} sessions a month: "
    f"${low * monthly_sessions:,.0f} to ${high * monthly_sessions:,.0f}"
)
```

```
revenue per session: A $2.731, B $3.192
difference $0.461, 95% interval $0.008 to $0.903
at 600,000 sessions a month: $5,046 to $541,860
```

Revenue per session rose from $2.73 to $3.19, a difference of 46 cents. But the 95% bootstrap interval runs from less than a cent to 90 cents. At 600,000 sessions a month, that's somewhere between about $5,000 and $540,000 a month.

That huge range is the honest answer. Revenue varies a lot from order to order, so it's much noisier than the simple converted-or-not measure. The decision is still easy: the page is very likely better and not worse. But anyone who promises "$277,000 a month" from this test is promising more than the data can support.

The Chart

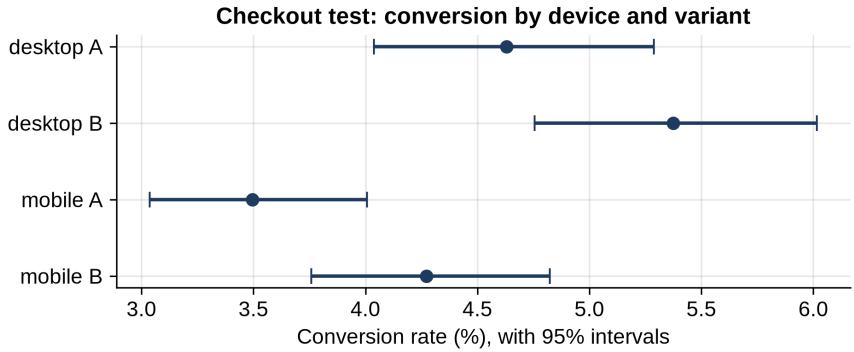
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")
rng = np.random.default_rng(5)

fig, ax = plt.subplots(figsize=(6.4, 2.8))
labels, rates, errors = [], [], []
for device in ["desktop", "mobile"]:
    for variant in ["A", "B"]:
        x = sessions.query("device == @device and variant == @variant")
        x = x["converted"].to_numpy()
        boot = [rng.choice(x, len(x)).mean() for _ in range(2_000)]
        low, high = np.percentile(boot, [2.5, 97.5])
        labels.append(f"{device} {variant}")
        rates.append(x.mean() * 100)
        errors.append([(x.mean() - low) * 100, (high - x.mean()) * 100])
ax.errorbar(rates, labels, xerr=np.array(errors).T, fmt="o", capsize=4)
ax.set_xlabel("Conversion rate (%), with 95% intervals")
ax.set_title("Checkout test: conversion by device and variant")
ax.invert_yaxis()
```

```
fig.tight_layout()
fig.savefig("ab_chart.png")
print("Saved ab_chart.png")
```

```
Saved ab_chart.png
```



Dots with 95% interval bars for conversion rate by device and variant: desktop B is highest, about 5.4%, mobile A lowest, about 3.5%; within each device, B's dot is to the right of A's.

Each dot is a conversion rate and each bar its 95% bootstrap interval. `query` with `@device` uses Python variables inside the query string. Within each device, B's dot sits to the right of A's. The intervals overlap, because each segment has only half the data, which is one more reason to decide on the overall comparison in advance and treat segments as supporting evidence.

What to Tell the Team

- The experiment ran correctly (the traffic split is consistent with 50/50).
- B raises conversion from 4.0% to 4.8% (95% interval for the difference: 0.2 to 1.3 points), on both desktop and mobile.
- Revenue per session is up about 46 cents, but this estimate is very uncertain: from almost nothing to about 90 cents.
- Recommendation: switch to B, and keep measuring revenue after launch.

Key Takeaways

- Check the experiment itself first: a sample ratio mismatch invalidates the results.
- Look at key segments to make sure no group is harmed, but decide which segments in advance.
- Revenue is noisy; give intervals, not single numbers, for its effect.
- Turn the analysis into a clear recommendation, with its uncertainty stated.

CHAPTER 24 · PROJECT

Project: Stock Price Study

`prices.csv` holds two years of daily closing prices for three fictional companies, Alder (ALDR), Brookline (BRKN) and Cedarlux (CDLX), and a market index. The question: how did each perform, how risky was each, and how much did each move with the market?

> **Note:** The companies and prices are invented, generated as random walks. Nothing in this chapter is investment advice; it's practice with the tools analysts use on any time series.

Returns, Volatility and Correlation

Analysts work with **returns**, the percentage change from one day to the next, rather than prices, because returns are comparable between a $30 stock and a $120 stock:

```
import numpy as np
import pandas as pd

prices = pd.read_csv(
    "../data/prices.csv", parse_dates=["date"], index_col="date"
)
returns = prices.pct_change().dropna()

summary = pd.DataFrame(
    {
        "total return": prices.iloc[-1] / prices.iloc[0] - 1,
        "annual volatility": returns.std() * np.sqrt(252),
        "best day": returns.max(),
        "worst day": returns.min(),
    }
```

```
)
print(summary.round(3))
print()
print(returns.corr().round(2))
```

```
       total return  annual volatility  best day  worst day
ALDR          0.595              0.196     0.042     -0.038
BRKN          0.866              0.320     0.078     -0.053
CDLX          0.307              0.134     0.029     -0.028
INDEX         0.527              0.140     0.028     -0.021

       ALDR  BRKN  CDLX  INDEX
ALDR   1.00  0.31  0.35   0.56
BRKN   0.31  1.00  0.32   0.52
CDLX   0.35  0.32  1.00   0.59
INDEX  0.56  0.52  0.59   1.00
```

- `pct_change()` turns prices into daily returns; `dropna()` drops the first day, which has no previous day.
- **Total return** compares the last price with the first: Brookline gained 87% over two years, Alder 60%, Cedarlux 31%, and the index 53%.
- **Volatility** is the standard deviation of daily returns, multiplied by the square root of 252 (the number of trading days in a year) to put it on a yearly scale. Brookline's 32% is more than twice Cedarlux's 13%: its higher return came with a much bumpier ride.
- The **correlation** table shows how the returns move together. Each company correlates more with the index (0.52 to 0.59) than with the others (0.31 to 0.35), because the index represents the shared market movement.

Drawdown and Beta

```
import numpy as np
import pandas as pd

prices = pd.read_csv(
    "../data/prices.csv", parse_dates=["date"], index_col="date"
)
returns = prices.pct_change().dropna()

peak = prices.cummax()
drawdown = prices / peak - 1
print("largest fall from a previous high:")
print(drawdown.min().round(3).to_string())
print("deepest point for BRKN:", drawdown["BRKN"].idxmin().date())
print()
```

```
for ticker in ["ALDR", "BRKN", "CDLX"]:
    slope, _ = np.polyfit(returns["INDEX"], returns[ticker], 1)
    print(f"{ticker} beta against the index: {slope:.2f}")
```

```
largest fall from a previous high:
ALDR    -0.255
BRKN    -0.223
CDLX    -0.169
INDEX   -0.153
deepest point for BRKN: 2025-04-03

ALDR beta against the index: 0.78
BRKN beta against the index: 1.19
CDLX beta against the index: 0.57
```

A **drawdown** is the fall from the highest price so far. `cummax()` gives the running peak, and dividing by it shows how far below its peak each day was. Alder's worst drawdown was 25.5%, despite its lower volatility than Brookline: averages don't show the worst stretch an investor sat through.

Beta measures how strongly a stock moves with the market: the slope of a straight line fitted through the stock's daily returns against the index's. `np.polyfit(x, y, 1)` fits that line. Brookline's beta of 1.19 means it moves about 1.2% for each 1% the index moves; Cedarlux, at 0.57, about half as much.

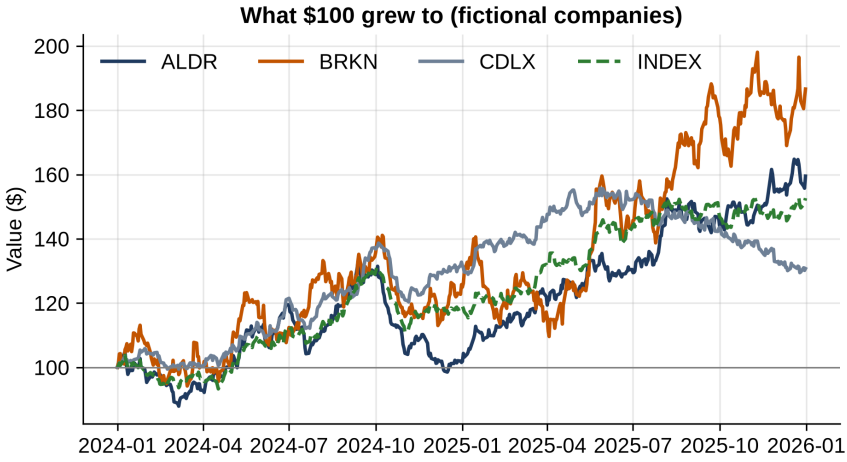
Growth of $100

```
import matplotlib.pyplot as plt
import pandas as pd

prices = pd.read_csv(
    "../data/prices.csv", parse_dates=["date"], index_col="date"
)
growth = 100 * prices / prices.iloc[0]  # $100 invested on day one

fig, ax = plt.subplots()
for column in growth.columns:
    style = "--" if column == "INDEX" else "-"
    ax.plot(growth.index, growth[column], style, label=column)
ax.axhline(100, color="gray", linewidth=0.8)
ax.set_title("What $100 grew to (fictional companies)")
ax.set_ylabel("Value ($)")
ax.legend(ncol=4)
fig.tight_layout()
fig.savefig("growth_chart.png")
print("Saved growth_chart.png")
```

```
Saved growth_chart.png
```



Lines showing what $100 invested at the start of 2024 grew to by the end of 2025: Brookline ends highest at about $187, after large swings; Alder near $160; the index near $153; Cedarlux near $131, with the smoothest path.

Dividing each column by its first value and multiplying by 100 puts all four on the same footing: the value of $100 invested on the first day. The index is dashed to set it apart. The chart shows both return and risk: Brookline ends highest, but with the deepest dips along the way.

Key Takeaways

- Analyze returns, not prices; `pct_change` converts one to the other.
- Annualized volatility, maximum drawdown and beta describe risk in different ways; report more than one.
- `cummax` finds running peaks; `np.polyfit` fits a straight line.
- "Growth of $100" charts compare series that start at different levels.

CHAPTER 25 · PROJECT

Project: Air Quality and Clinic Visits

Three fictional cities, Avonford, Brightwater and Cedarvale, record daily fine-particle pollution (PM2.5) from roadside sensors, and their clinics record daily visits for breathing problems. A city health officer asks: are clinic visits higher after polluted days, and how much higher?

PM2.5 means particles smaller than 2.5 micrometers, small enough to be breathed deep into the lungs. It's measured in micrograms per cubic meter of air.

Preparing the Data

```
import pandas as pd

air = pd.read_csv("../data/air_quality.csv", parse_dates=["date"])
visits = pd.read_csv("../data/clinic_visits.csv", parse_dates=["date"])

missing = air["pm25"].isna().groupby(air["city"]).sum()
print("missing PM2.5 readings:", missing.to_dict())

# Fill short sensor gaps (up to 3 days) from the neighboring days
air["pm25"] = air.groupby("city")["pm25"].transform(
    lambda s: s.interpolate(limit=3)
)
df = air.merge(visits, on=["date", "city"], validate="one_to_one")
print("rows:", len(df), "| still missing:", df["pm25"].isna().sum())

df["quarter"] = df["date"].dt.quarter
print(
    df.pivot_table(index="city", columns="quarter", values="pm25").round(1)
```

```
)
```

```
missing PM2.5 readings: {'Avonford': 9, 'Brightwater': 8, 'Cedarvale': 4}
rows: 1095 | still missing: 0
quarter         1     2     3     4
city
Avonford     25.2  14.9  12.3  22.4
Brightwater  41.8  24.4  19.7  39.8
Cedarvale    17.4   9.9   8.1  15.2
```

- The sensors failed on 21 days in total. `interpolate(limit=3)` fills each gap with values on a straight line between the readings on either side, but only for gaps of up to three days. Longer gaps stay missing, because inventing a week of pollution data would be guessing. `groupby("city")` with `transform` makes sure each city is filled only from its own readings.
- `merge(..., validate="one_to_one")` joins the two files on date and city, and checks that each pair appears once in each file. All 1,095 rows (365 days times 3 cities) match.
- The quarterly averages show strong seasons: pollution in the first and fourth quarters (winter) is roughly double the third quarter (summer), and Brightwater is the most polluted city all year.

Are Visits Higher After Polluted Days?

Chapter 16 found that visits in Brightwater follow pollution with a two-day delay. This program measures that delay's effect in all three cities:

```
import pandas as pd

air = pd.read_csv("../data/air_quality.csv", parse_dates=["date"])
visits = pd.read_csv("../data/clinic_visits.csv", parse_dates=["date"])
air["pm25"] = air.groupby("city")["pm25"].transform(
    lambda s: s.interpolate(limit=3)
)
df = air.merge(visits, on=["date", "city"]).sort_values(["city", "date"])
df["pm25_lag2"] = df.groupby("city")["pm25"].shift(2)

# WHO 2021 guideline for a 24-hour average: 15 micrograms per cubic meter
df["polluted"] = df["pm25_lag2"] > 15
rows = []
for city, g in df.dropna().groupby("city"):
    rows.append(
        {
            "city": city,
            "days >15": int(g["polluted"].sum()),
            "visits (clean)": round(
                g.loc[~g["polluted"], "visits"].mean(), 1
```

```
            ),
            "visits (polluted)": round(
                g.loc[g["polluted"], "visits"].mean(), 1
            ),
            "r (lag 2)": round(g["visits"].corr(g["pm25_lag2"]), 2),
        }
    )
print(pd.DataFrame(rows).set_index("city"))
```

```
             days >15  visits (clean)  visits (polluted)  r (lag 2)
city
Avonford          216            11.1               15.1       0.54
Brightwater       321            17.8               30.0       0.76
Cedarvale         106             7.5               10.7       0.48
```

The World Health Organization's 2021 guideline for fine particles is a 24-hour average of no more than 15 micrograms per cubic meter (and a yearly average of 5). The program shifts each city's pollution by two days with `groupby("city")["pm25"].shift(2)`, so each day is paired with the pollution two days earlier, and compares visits after days above and below the guideline:

- In **Brightwater**, visits after polluted days average 30.0, against 17.8 after clean days, and the correlation is strong (0.76).
- In **Avonford** and **Cedarvale**, the pattern is the same but smaller, with correlations around 0.5.
- Brightwater exceeded the guideline on 321 days of the year, Avonford on 216 and Cedarvale on 106.

Seeing It

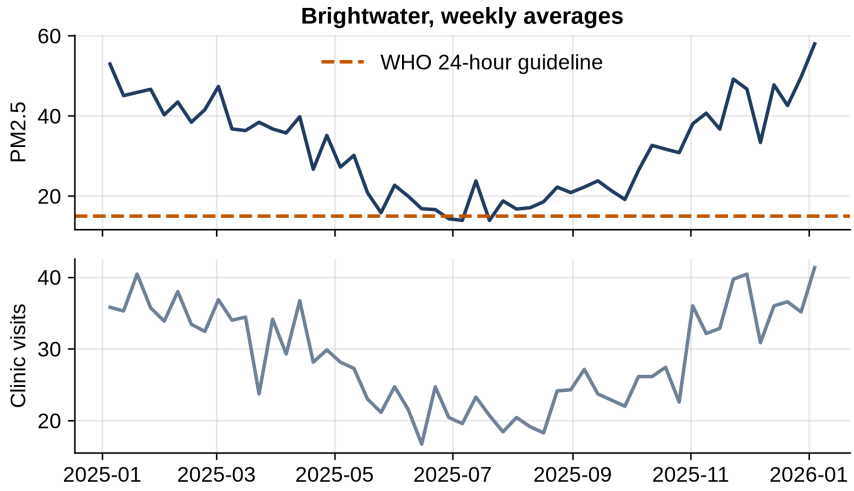
```
import matplotlib.pyplot as plt
import pandas as pd

air = pd.read_csv("../data/air_quality.csv", parse_dates=["date"])
visits = pd.read_csv("../data/clinic_visits.csv", parse_dates=["date"])
df = air.merge(visits, on=["date", "city"]).query("city == 'Brightwater'")
weekly = df.set_index("date")[["pm25", "visits"]].resample("W").mean()

fig, (top, bottom) = plt.subplots(2, 1, figsize=(6.4, 3.8), sharex=True)
top.plot(weekly.index, weekly["pm25"])
top.axhline(15, color="C1", linestyle="--", label="WHO 24-hour guideline")
top.set_ylabel("PM2.5")
top.set_title("Brightwater, weekly averages")
top.legend()
bottom.plot(weekly.index, weekly["visits"], color="C2")
bottom.set_ylabel("Clinic visits")
```

```
fig.tight_layout()
fig.savefig("health_chart.png")
print("Saved health_chart.png")
```

```
Saved health_chart.png
```



Two stacked line charts for Brightwater, weekly averages: PM2.5 on top, with the WHO guideline of 15 as a dashed line, and clinic visits below. Both are high in winter and low in summer.

Two charts stacked with a shared date axis (`sharex=True`) let the reader compare the two series' shapes directly, without the confusion of two different y-axes on one chart. Weekly averages smooth the daily noise.

Careful Conclusions

This is where honest analysis matters most. The data shows a strong **association**: more clinic visits two days after polluted days. It doesn't prove pollution *causes* the visits on its own. Winter brings both more pollution and more colds and flu; a careful study would compare days within the same season, or control for temperature and flu activity. (In this book's data, the link was built in on purpose, so you know it's real; with real data you wouldn't.)

A good summary for the health officer:

- Clinic visits are substantially higher two days after days above the WHO guideline, most clearly in Brightwater (30 against 18 visits a day).
- The pattern is consistent across all three cities, which makes chance unlikely.
- Seasonal factors such as flu may explain part of the link; a follow-up analysis should account for season and temperature.

Key Takeaways

- Fill only short gaps in time series, within each group, and say how you handled the rest.
- Validate joins with `validate` and check that the row count is what you expect.
- Use `groupby(...).shift()` to pair each day with an earlier day in the same city.
- Separate association from causation, and name the likely confounders, such as season.

CHAPTER 26 · PROJECT

Project: Household Budget

The final project is personal: a household's bank transactions for 2025, exported as a CSV, like most banks allow. The questions are the ones anyone asks about their own money: where does it go, how much are we saving, and which regular payments are we committed to?

Where the Money Goes

```
import pandas as pd

budget = pd.read_csv("../data/budget.csv", parse_dates=["date"])
rules = {
    "Income": r"SALARY",
    "Housing": r"RENT|POWER",
    "Groceries": r"FRESHMART",
    "Coffee": r"CAFE",
    "Transport": r"TRANSIT|RIDESHARE",
    "Shopping": r"ONLINE STORE",
    "Subscriptions": r"SUBSCRIPTION",
}
budget["category"] = "Other"
for name, pattern in rules.items():
    budget.loc[budget["description"].str.contains(pattern), "category"] = (
        name
    )

income = budget.loc[budget["amount"] > 0, "amount"].sum()
spending = -budget.loc[budget["amount"] < 0, "amount"].sum()
print(
    f"income ${income:,.2f}, spending ${spending:,.2f}, "
    f"saved {1 - spending / income:.1%}"
)
print()
monthly = (
    budget[budget["amount"] < 0]
    .pivot_table(
```

```
        index=budget["date"].dt.month,
        columns="category",
        values="amount",
        aggfunc="sum",
    )
    .mul(-1)
    .fillna(0)
)
print(
    monthly.mean()
    .sort_values(ascending=False)
    .round(2)
    .rename("average per month")
)
```

```
income $50,400.00, spending $26,552.35, saved 47.3%

category
Housing          1556.67
Groceries         414.82
Shopping          154.16
Coffee             44.51
Transport          29.55
Subscriptions      12.99
Name: average per month, dtype: float64
```

The program categorizes each transaction with patterns, as in Chapter 13, then:

- splits money in (positive amounts) from money out (negative), giving a savings rate of 47%: a fortunate household;
- pivots spending into months and categories, flips the sign (`mul(-1)`) so spending shows as positive numbers, and fills empty cells with 0;
- averages each category over the twelve months.

Housing (rent and utilities) dominates at about $1,557 a month, followed by groceries at $415. Coffee, at about $45 a month, is small, but that's still over $500 a year.

> **Tip:** Average *monthly* spending by category is more useful than yearly totals for a budget, because that's how bills arrive. Filling missing months with zero matters: a category with no spending in some months would otherwise have its average computed over fewer months, and look bigger than it is.

Finding Recurring Payments

```
import pandas as pd

budget = pd.read_csv("../data/budget.csv", parse_dates=["date"])
spend = budget[budget["amount"] < 0]

# A payment is recurring if the same description and amount appear in
# at least 10 different months
by_payment = spend.groupby(["description", "amount"])["date"].agg(
    months=lambda d: d.dt.month.nunique()
)
recurring = by_payment[by_payment["months"] >= 10].reset_index()
recurring["per year"] = -recurring["amount"] * recurring["months"]
print(
    recurring.sort_values("per year", ascending=False).to_string(
        index=False
    )
)
```

```
            description   amount  months  per year
     RENT MAPLE HOUSING -1450.00      12  17400.00
STREAMFLIX SUBSCRIPTION   -12.99      12    155.88
```

A payment counts as recurring if the same description and the same amount appear in at least 10 different months. Two qualify: rent ($17,400 a year) and a streaming subscription ($155.88 a year). The power bill is regular too, but its amount changes with the season, so the "same amount" rule misses it. Rules like this need tuning: you might allow a 30% variation in amount, or look for regular timing instead.

The Chart

```
import matplotlib.pyplot as plt
import pandas as pd

budget = pd.read_csv("../data/budget.csv", parse_dates=["date"])
budget = budget[budget["amount"] < 0]
rules = {
    "Housing": r"RENT|POWER",
    "Groceries": r"FRESHMART",
    "Coffee": r"CAFE",
    "Transport": r"TRANSIT|RIDESHARE",
}
budget["category"] = "Other"
for name, pattern in rules.items():
    budget.loc[budget["description"].str.contains(pattern), "category"] = (
        name
    )
monthly = (
```

```
    budget.pivot_table(
        index=budget["date"].dt.strftime("%m"),
        columns="category",
        values="amount",
        aggfunc="sum",
    )
    .mul(-1)
    .fillna(0)
)
monthly = monthly[["Housing", "Groceries", "Other", "Coffee", "Transport"]]

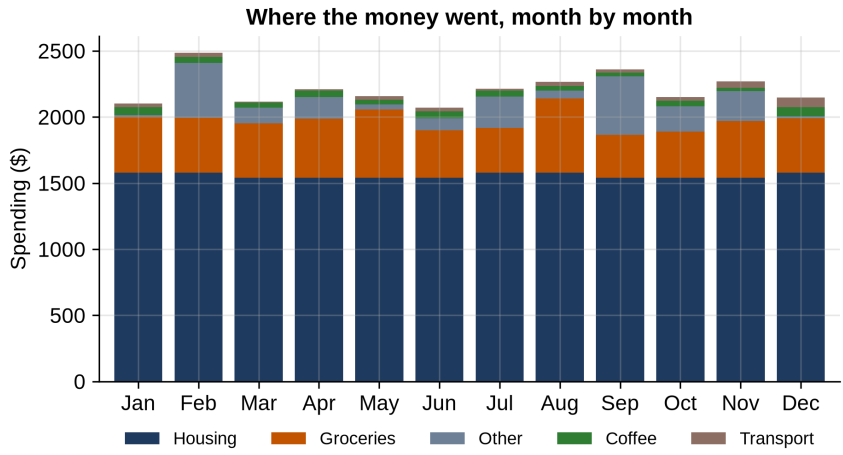
fig, ax = plt.subplots()
monthly.plot.bar(stacked=True, ax=ax, width=0.8)
ax.set_xticklabels(
    [
        "Jan",
        "Feb",
        "Mar",
        "Apr",
        "May",
        "Jun",
        "Jul",
        "Aug",
        "Sep",
        "Oct",
        "Nov",
        "Dec",
    ],
    rotation=0,
)
ax.set_xlabel("")
ax.set_ylabel("Spending ($)")
ax.set_title("Where the money went, month by month")
ax.legend(
    ncol=5, fontsize=9, loc="upper center", bbox_to_anchor=(0.5, -0.1)
)
fig.tight_layout()
fig.savefig("budget_chart.png")
print("Saved budget_chart.png")
```

```
Saved budget_chart.png
```

`DataFrame.plot.bar(stacked=True)` draws a stacked bar per month in one call, with one color per column. Housing is the steady base; the variable categories on top are where a budget can actually change.

Your Turn

- Export your own bank transactions (most banks offer CSV), adjust the patterns to your merchants, and run the same analysis. Keep the file on your own computer, and don't share it.



Stacked monthly bars of spending by category for 2025: housing is the large base of every bar; groceries, other spending, coffee and transport sit on top in smaller layers.

- Add a check that every transaction landed in a category other than "Other", and print the ones that didn't, so you can add patterns.

Key Takeaways

- Categorize transactions with patterns, and keep refining the rules as you meet new descriptions.
- Pivot into months by category, fill empty months with zero, then average.
- Simple rules find recurring payments, but test them against cases you know.
- Your own data is the best practice of all, and it's yours: keep it private.

PART V

Going Further

CHAPTER 27 · ADVANCED

Bigger Data and Faster Code

pandas works comfortably with tables up to a few million rows on a laptop. Beyond that, or when a program feels slow, a few habits go a long way: smaller column types, reading in chunks, whole-column operations, and readable step-by-step pipelines. This chapter covers them, and when to reach for other tools.

Smaller Types, Less Memory

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
before = orders.memory_usage(deep=True).sum()

smaller = orders.astype(
    {
        "region": "category",
        "channel": "category",
        "category": "category",
        "product": "category",
        "quantity": "int8",
        "discount": "float32",
    }
)
after = smaller.memory_usage(deep=True).sum()
print(
    f"before {before / 1e6:.2f} MB, after {after / 1e6:.2f} MB "
    f"({after / before:.0%} of the original)"
)
print(smaller.dtypes[["region", "quantity", "discount"]])
```

```
before 2.41 MB, after 1.08 MB (45% of the original)
region      category
quantity        int8
discount     float32
dtype: object
```

`memory_usage(deep=True)` measures the real memory a DataFrame uses, including its text. Changing types cuts this table to 45% of its size:

- `category` for text columns with few distinct values (four regions, three channels, fifteen products): each value is stored once, and each row keeps only a small code.
- `int8` for quantities that never exceed 127 (`int16` up to 32,767, `int32` up to about 2 billion).
- `float32` for decimals that don't need 15 significant digits.

> **Warning:** Small integer types overflow silently if a value is too large: an `int8` can't hold 300. Only shrink a column when you know its range, and check with `min()` and `max()` first.

Reading in Chunks

When a file is too big to load at once, read it in pieces and combine the partial results:

```
import pandas as pd

totals = {}
for chunk in pd.read_csv("../data/sessions.csv", chunksize=5_000):
    part = chunk.groupby("variant")["revenue"].sum()
    for variant, revenue in part.items():
        totals[variant] = totals.get(variant, 0) + revenue
    print(f"read {len(chunk):,} rows; running total A {totals['A']:,.2f}")

whole = pd.read_csv("../data/sessions.csv").groupby("variant")["revenue"]
print(
    "same as reading it all at once:",
    all(abs(totals[v] - s) < 0.01 for v, s in whole.sum().items()),
)
```

```
read 5,000 rows; running total A 6,800.99
read 5,000 rows; running total A 13,741.50
read 5,000 rows; running total A 20,230.35
read 5,000 rows; running total A 27,372.98
same as reading it all at once: True
```

`read_csv(..., chunksize=5_000)` returns the file 5,000 rows at a time. Each chunk is summarized and the partial totals are added up, so only one chunk is ever in memory. The final check confirms the result matches reading the whole file at once. This works for totals, counts, minimums and maximums; for medians and other statistics that need all the data together, you'll need a different tool.

Readable Pipelines

Method chaining writes an analysis as a sequence of steps, read top to bottom:

```
import pandas as pd


def add_revenue(df):
    return df.assign(
        revenue=df["quantity"] * df["unit_price"] * (1 - df["discount"])
    )


summary = (
    pd.read_csv("../data/orders.csv", parse_dates=["order_date"])
    .pipe(add_revenue)
    .query("channel != 'marketplace'")
    .groupby(["region", "channel"], as_index=False)["revenue"]
    .sum()
    .sort_values("revenue", ascending=False)
    .head(4)
    .round(0)
)
print(summary.to_string(index=False))
```

```
region channel  revenue
 North     web  55847.0
  East     web  52391.0
 South     web  43196.0
  West     web  37230.0
```

Each line is one step: read, add revenue, filter, group, sum, sort, take the top four, round. `pipe` passes the DataFrame to your own function, so custom steps fit into the chain. Wrapping the chain in parentheses lets it span several lines.

There are no intermediate variables to keep track of, and you can comment out a step to see what it changes.

Speed Habits

- **Whole-column operations** instead of loops and `apply(axis=1)` (Chapter 8). This is by far the biggest win.
- **Read only what you need:** `usecols`, `nrows` and `dtype` when loading.
- **Filter early:** drop rows and columns you don't need before merging and grouping.
- **Save cleaned data in a faster format:** Parquet files (`to_parquet`, which needs the `pyarrow` library) load much faster than CSV and keep column types.
- **Measure before optimizing:** time the slow part with `time.perf_counter()` and fix that, not what you guess is slow.

When pandas Isn't Enough

- **DuckDB** runs SQL directly on CSV and Parquet files, often far faster than pandas for large group-bys and joins, and returns pandas DataFrames.
- **Polars** is a newer DataFrame library, designed for speed and for data larger than memory.
- **A real database** (PostgreSQL, or a cloud data warehouse) when data is shared, updated continuously, or much bigger than one computer.

Everything you've learned carries over: they all filter, group, join and reshape tables.

Exercises

1. **Shrink the sessions.** Measure the memory used by `sessions.csv`, then convert `variant` and `device` to categories, `pages` to `int16`, `converted` to `int8` and `revenue` to `float32`. How much smaller is it?

Key Takeaways

- `category`, smaller integers and `float32` can cut memory by half or more; check value ranges first.
- `chunksize` processes files too big for memory, for totals and counts.
- Method chains with `pipe` make pipelines readable step by step.
- Vectorize, read only what you need, filter early, and measure before optimizing.

CHAPTER 28 · ADVANCED

Next Steps

You've worked through the whole craft: loading and cleaning data, transforming and combining it, summarizing and charting it, testing whether differences are real, writing SQL, testing your code, and reporting results honestly, in six complete projects. Here's how to keep going.

Practice on Real Data

The best next step is your own questions about real data. Good free sources:

- **Your country's statistics office and open-data portal:** population, prices, transport, health, weather.
- **Kaggle datasets:** thousands of datasets on every topic, with other people's analyses to learn from.
- **Your own life:** bank statements, fitness trackers, phone screen time, your music history.
- **Your workplace:** with permission, the reports everyone wishes someone would automate.

For each dataset, write down three questions *before* you look at it, then answer them using the six steps from Chapter 1. Publish your best analyses as short write-ups, with code, on a blog or GitHub: they make an excellent portfolio for job applications.

Skills to Learn Next

- **Interactive charts:** Plotly and Altair make zoomable, hoverable charts for the web.
- **Dashboards:** Streamlit turns a Python script into a simple web app with filters and charts.
- **Statistics and modeling:** the `statsmodels` library for regression and classic tests; `scikit-learn` for machine learning, prediction and clustering.
- **Big data tools:** DuckDB, Polars and cloud data warehouses (Chapter 27).
- **Data engineering:** scheduling pipelines, data quality checks and version control for data.

Habits That Make a Great Analyst

- Start with the question, and write it down.
- Look at the raw data before trusting any summary.
- Check row counts at every step.
- Compare like with like.
- Show the uncertainty.
- Make it repeatable, with code and tests.
- Say what you don't know.

Interviewing for Analyst Roles

Analyst interviews usually include SQL questions (Chapter 18), a pandas or spreadsheet exercise, a case question ("Sales dropped 10% last week. How would you investigate?") and a discussion of a past project. For the case questions, use the workflow: clarify the question, ask what data exists, check data quality first (a tracking bug is the most common cause of sudden "drops"), break the number into parts, compare with the same period last year, and only then look for causes.

Thank you for reading. Now go and answer some questions.

Key Takeaways

- Practice on real data with questions you care about, and publish your best work.
- Next skills: interactive charts, dashboards, statistics and machine learning, big data tools.
- The habits matter more than the tools: question first, check counts, compare like with like, show uncertainty, make it repeatable.

Appendix A: Exercise Solutions

Here are worked solutions to the exercises. Each one was run on Python 3.12, 3.13, 3.14 and 3.15, and the output shown is the output it produced. There's usually more than one good answer: if yours gives the same result and you can explain it, it's right. The same files are in the book's code repository as `solution_1.py`, `solution_2.py` and so on, in each chapter's folder.

Chapter 3: NumPy Essentials

Exercise 1.

```
import numpy as np

temps = np.array([[12, 15, 14, 18], [20, 22, 21, 19], [8, 9, 11, 10]])
print(temps.mean(axis=1))  # average per city (row)
print(temps.max(axis=0))  # warmest reading per day (column)
print((temps > 15).sum())  # readings above 15
```

```
[14.75 20.5   9.5 ]
[20 22 21 19]
5
```

Exercise 2.

```
import numpy as np

rng = np.random.default_rng(7)
flips = rng.integers(0, 2, size=(10_000, 10))  # 10,000 runs of 10 flips
heads = flips.sum(axis=1)
print("share of runs with 8 or more heads:", (heads >= 8).mean())
```

```
share of runs with 8 or more heads: 0.0584
```

The coin-flip simulation gives 0.0584, close to the exact probability of 56/1024, about 0.055. With more runs, it would get closer.

Chapter 4: Series and DataFrames

Exercise 1.

```
import pandas as pd

customers = pd.read_csv("../data/customers.csv")
print(customers.shape)
print(customers["segment"].value_counts())
print(customers["birth_year"].min(), customers["birth_year"].max())
```

```
(1200, 5)
segment
Consumer    1036
Business     164
Name: count, dtype: int64
1955 2005
```

Exercise 2.

```
import pandas as pd

products = pd.read_csv("../data/products.csv")
products["margin"] = 1 - products["unit_cost"] / products["list_price"]
print(
    products.sort_values("margin", ascending=False)[["product", "margin"]]
    .head(3)
    .round(3)
)
```

```
      product  margin
7  Floor Lamp   0.523
0  Chef Knife   0.523
9  Wall Clock   0.505
```

Chapter 5: Loading and Saving Data

Exercise 1.

```
import pandas as pd

survey = pd.read_csv(
    "../data/survey.csv",
    usecols=["plan", "satisfaction"],
    dtype={"plan": "category"},
)
print(survey.dtypes)
print(survey["plan"].cat.categories.tolist())
```

```
plan            category
satisfaction       int64
dtype: object
['Basic', 'Plus', 'Pro']
```

Exercise 2.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
for region, group in orders.groupby("region"):
    group.to_csv(f"orders_{region.lower()}.csv", index=False)
    print(f"orders_{region.lower()}.csv: {len(group)} rows")
```

```
orders_east.csv: 1560 rows
orders_north.csv: 1768 rows
orders_south.csv: 1300 rows
orders_west.csv: 1205 rows
```

Chapter 6: Selecting and Filtering

Exercise 1.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
lamps = orders[
    orders["product"].str.contains("Lamp") & (orders["discount"] == 0)
]
print(len(lamps))
print(lamps["product"].value_counts())
```

```
534
product
Desk Lamp     268
Floor Lamp    266
Name: count, dtype: int64
```

Exercise 2.

```
import pandas as pd

customers = pd.read_csv(
    "../data/customers.csv", parse_dates=["signup_date"]
)
recent = customers.query(
    "segment == 'Business' and signup_date >= '2025-01-01'"
)
print(len(recent))
print(
    recent.sort_values("signup_date").head(3)[
        ["customer_id", "signup_date"]
    ]
)
```

```
44
    customer_id signup_date
885       C0886  2025-01-01
659       C0660  2025-01-11
377       C0378  2025-01-14
```

Chapter 7: Cleaning Messy Data

Exercise 1.

```
import pandas as pd

messy = pd.Series([" 12 kg", "7kg", "n/a", "15 KG ", ""])
weights = pd.to_numeric(
    messy.str.lower().str.replace("kg", "").str.strip(), errors="coerce"
)
print(weights.tolist())
print("mean of known weights:", weights.mean())
```

```
[12.0, 7.0, nan, 15.0, nan]
mean of known weights: 11.333333333333334
```

`errors="coerce"` turns "n/a" and the empty string into missing values, which `mean()` skips.

Exercise 2.

```
import pandas as pd

raw = pd.read_csv("../data/orders_raw.csv")
dupes = raw[raw.duplicated(keep=False)]
print(dupes[["order_id", "product", "quantity"]].sort_values("order_id"))
```

```
   order_id        product  quantity
10   L00245  Throw Blanket       3.0
11   L00245  Throw Blanket       3.0
21   L00474     Floor Lamp       1.0
22   L00474     Floor Lamp       1.0
32   L00707    Duvet Cover       1.0
33   L00707    Duvet Cover       1.0
```

Chapter 8: Transforming Columns

Exercise 1.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
orders["price_band"] = pd.cut(
    orders["unit_price"],
    bins=[0, 25, 50, 200],
    labels=["budget", "mid", "premium"],
)
print(orders["price_band"].value_counts().sort_index())
```

```
price_band
budget     1876
mid        2379
premium    1578
Name: count, dtype: int64
```

Exercise 2.

```
import pandas as pd

survey = pd.read_csv("../data/survey.csv")
survey["mood"] = survey["satisfaction"].map(
    {1: "unhappy", 2: "unhappy", 3: "neutral", 4: "happy", 5: "happy"}
)
print(survey["mood"].value_counts())
```

```
mood
happy      423
neutral    270
unhappy    107
Name: count, dtype: int64
```

Chapter 9: Grouping and Aggregating

Exercise 1.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
best = orders.groupby(["category", "product"])["quantity"].sum()
print(best.groupby("category").idxmax().str[1])
```

```
category
Bedding     Throw Blanket
Decor       Picture Frame
Garden            Planter
Kitchen        Chef Knife
Lighting        Desk Lamp
Name: quantity, dtype: object
```

`idxmax()` on each category's group returns a (category, product) pair; `.str[1]` takes the product name from each pair.

Exercise 2.

```
import pandas as pd

survey = pd.read_csv("../data/survey.csv")
print(
    survey.pivot_table(
        index="age_group",
        columns="plan",
        values="satisfaction",
        aggfunc="mean",
    ).round(2)
)
```

```
plan       Basic  Plus   Pro
age_group
18-29       3.12  3.67  4.08
30-44       3.20  3.62  3.98
45-59       3.18  3.69  3.95
60+         3.38  3.92  4.07
```

Chapter 10: Combining Tables

Exercise 1.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
customers = pd.read_csv("../data/customers.csv")
check = customers.merge(
    orders[["customer_id"]].drop_duplicates(),
    on="customer_id",
    how="left",
    indicator=True,
)
never = check[check["_merge"] == "left_only"]
print("customers with no orders:", len(never))
print(never["customer_id"].head().tolist())
```

```
customers with no orders: 9
['C0005', 'C0064', 'C0191', 'C0201', 'C0211']
```

Nine customers never ordered.

Exercise 2.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
customers = pd.read_csv("../data/customers.csv")
df = orders.merge(
    customers[["customer_id", "birth_year"]], on="customer_id"
)
df["age"] = 2025 - df["birth_year"]
print(df.groupby("category")["age"].median())
```

```
category
Bedding     46.0
Decor       46.0
Garden      46.0
Kitchen     44.0
Lighting    45.0
Name: age, dtype: float64
```

Chapter 11: Reshaping Data

Exercise 1.

```
import pandas as pd

prices = pd.read_csv("../data/prices.csv")
long = prices.melt(id_vars="date", var_name="ticker", value_name="close")
print(long.shape)
print(long.groupby("ticker")["close"].max())
```

```
(2092, 3)
ticker
ALDR       83.54
BRKN       60.31
CDLX      183.30
INDEX    1507.67
Name: close, dtype: float64
```

Exercise 2.

```
import pandas as pd

orders = pd.read_csv("../data/orders.csv")
table = orders.pivot_table(
    index="region", columns="discount", values="order_id", aggfunc="count"
)
print(table)
```

```
discount   0.0  0.1  0.2
region
East      1071  331  158
North     1230  367  171
South      929  252  119
West       854  223  128
```

Chapter 12: Dates and Time Series

Exercise 1.

```
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv", parse_dates=["date"])
weekly = sessions.set_index("date")["converted"].resample("W").mean()
print(weekly.round(4))
```

```
date
2025-10-05    0.0495
2025-10-12    0.0384
2025-10-19    0.0429
2025-10-26    0.0477
2025-11-02    0.0401
Freq: W-SUN, Name: converted, dtype: float64
```

The last week is only partly in the data (sessions end on 28 October), so its rate is based on fewer sessions.

Exercise 2.

```
import pandas as pd

air = pd.read_csv("../data/air_quality.csv", parse_dates=["date"])
city = air[air["city"] == "Avonford"].set_index("date")["pm25"]
month = city.resample("MS").mean()
print("worst month:", month.idxmax().strftime("%B"), round(month.max(), 1))
print("best month:", month.idxmin().strftime("%B"), round(month.min(), 1))
```

```
worst month: January 27.7
best month: July 11.1
```

Chapter 13: Working With Text

Exercise 1.

```
import pandas as pd

budget = pd.read_csv("../data/budget.csv")
cafes = budget[budget["description"].str.contains("CAFE")]
print(cafes["description"].str.title().value_counts())
```

```
description
Cafe Luma           51
Bean & Leaf Cafe    36
Name: count, dtype: int64
```

Exercise 2.

```
import pandas as pd

codes = pd.Series(["INV-2025-0041", "inv-2025-0042", "PO-2024-0007"])
parts = codes.str.upper().str.extract(
    r"(?P<kind>\w+)-(?P<year>\d{4})-(?P<num>\d+)"
)
parts["num"] = parts["num"].astype(int)
```

```
print(parts)
```

```
  kind  year  num
0  INV  2025   41
1  INV  2025   42
2   PO  2024    7
```

Named groups `(?P<kind>...)` become the column names.

Chapter 14: Charts With matplotlib

```
import matplotlib.pyplot as plt
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")
pages = sessions["pages"].value_counts().sort_index()

fig, ax = plt.subplots()
ax.bar(pages.index, pages)
ax.set_xlabel("Pages viewed in the session")
ax.set_ylabel("Sessions")
ax.set_title("How many pages people view")
fig.savefig("pages.png")
print("Saved pages.png;", len(pages), "bars")
```

```
Saved pages.png; 14 bars
```

The chart is saved as `pages.png`; the printed line confirms how many bars it drew.

Chapter 16: Describing Data With Statistics

Exercise 1.

```
import pandas as pd

budget = pd.read_csv("../data/budget.csv")
coffee = -budget.loc[budget["description"].str.contains("CAFE"), "amount"]
print(coffee.describe().round(2))
```

```
count    87.00
mean      6.14
std       1.72
min       3.20
25%       4.72
50%       6.35
75%       7.70
max       8.96
```

```
Name: amount, dtype: float64
```

Exercise 2.

```
import pandas as pd

survey = pd.read_csv("../data/survey.csv")
print(survey[["satisfaction", "recommend"]].corr().round(2))
```

```
              satisfaction  recommend
satisfaction          1.00       0.82
recommend             0.82       1.00
```

Chapter 17: Comparing Groups and A/B Tests

```
import numpy as np
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")
mobile = sessions[sessions["device"] == "mobile"]
a = mobile.loc[mobile["variant"] == "A", "converted"].to_numpy()
b = mobile.loc[mobile["variant"] == "B", "converted"].to_numpy()
rng = np.random.default_rng(6)
diffs = [
    rng.choice(b, len(b)).mean() - rng.choice(a, len(a)).mean()
    for _ in range(5_000)
]
print(np.percentile(diffs, [2.5, 97.5]).round(4))
```

```
[0.0006 0.0149]
```

The interval excludes zero, so on mobile alone the data still supports B being better.

Chapter 18: SQL for Analysts

Exercise 1.

```
import sqlite3

import pandas as pd

conn = sqlite3.connect(":memory:")
pd.read_csv("../data/orders.csv").to_sql("orders", conn, index=False)
print(
    pd.read_sql(
        """
SELECT strftime('%m', order_date) AS month, COUNT(*) AS orders
```

```
FROM orders
GROUP BY month
ORDER BY orders DESC
LIMIT 3
""",
        conn,
    )
)
```

```
  month  orders
0    12     879
1    11     716
2    10     546
```

Exercise 2.

```
import sqlite3

import pandas as pd

conn = sqlite3.connect(":memory:")
pd.read_csv("../data/orders.csv").to_sql("orders", conn, index=False)
pd.read_csv("../data/customers.csv").to_sql("customers", conn, index=False)
print(
    pd.read_sql(
        """
SELECT c.segment, COUNT(DISTINCT o.customer_id) AS buyers,
       ROUND(AVG(o.quantity), 2) AS avg_qty
FROM orders AS o JOIN customers AS c USING (customer_id)
GROUP BY c.segment
""",
        conn,
    )
)
```

```
    segment  buyers  avg_qty
0  Business     163     2.44
1  Consumer    1028     1.48
```

Chapter 27: Bigger Data and Faster Code

```
import pandas as pd

sessions = pd.read_csv("../data/sessions.csv")
before = sessions.memory_usage(deep=True).sum()
small = sessions.astype(
    {
        "variant": "category",
        "device": "category",
        "pages": "int16",
        "converted": "int8",
        "revenue": "float32",
```

```
    }
)
after = small.memory_usage(deep=True).sum()
print(f"{before / 1e6:.2f} MB -> {after / 1e6:.2f} MB")
```

```
3.93 MB -> 1.52 MB
```

The sessions table shrinks to under 40% of its size.

Appendix B: pandas Cheat Sheet

The operations used most often in this book, with the chapter that explains each.

Loading and Saving (Chapter 5)

Task	Code
Read a CSV	`pd.read_csv("f.csv", parse_dates=["date"])`
Only some columns	`pd.read_csv("f.csv", usecols=["a", "b"])`
Read Excel, every sheet	`pd.read_excel("f.xlsx", sheet_name=None)`
Run SQL	`pd.read_sql("SELECT ...", conn)`
Save	`df.to_csv("out.csv", index=False)`, `df.to_excel(...)`, `df.to_sql(...)`

Looking (Chapter 4)

Task	Code
Size, first rows, types	`df.shape`, `df.head()`, `df.dtypes`
Summary of numbers	`df.describe()`
Count values	`df["col"].value_counts(normalize=True)`
Missing values	`df.isna().sum()`
Columns, memory	`df.info()`

Selecting (Chapter 6)

Task	Code
One column, several	`df["a"]`, `df[["a", "b"]]`
By label, by position	`df.loc["row", "col"]`, `df.iloc[0, 2]`
Filter	`df[(df["a"] > 5) & (df["b"] == "x")]`
In a list, in a range	`df["a"].isin([1, 2])`, `df["a"].between(1, 5)`
Query string	`df.query("a > 5 and b == 'x'")`
Sort, top rows	`df.sort_values("a", ascending=False)`, `df.nlargest(5, "a")`

Cleaning and Transforming (Chapters 7 and 8)

Task	Code
Remove duplicates	`df.drop_duplicates()`
Missing values	`df.dropna()`, `df["a"].fillna(value)`
Fix text	`df["a"].str.strip().str.title()`
Convert types	`df["a"].astype(float)`, `pd.to_numeric(s, errors="coerce")`
Parse dates	`pd.to_datetime(s, format="%Y-%m-%d")`
New column	`df["c"] = df["a"] * df["b"]`, `df.assign(c=...)`

Task	Code
Conditions	`np.select([cond1, cond2], [v1, v2], default=v3)`
Translate values	`df["a"].map({"x": "X"})`
Bands	`pd.cut(s, bins=[...], labels=[...])`, `pd.qcut(s, 4)`

Grouping, Combining, Reshaping (Chapters 9 to 11)

Task	Code
Group and sum	`df.groupby("a")["b"].sum()`
Several results	`df.groupby("a").agg(n=("b", "count"), total=("c", "sum"))`
Group value on each row	`df.groupby("a")["b"].transform("sum")`
Pivot table	`df.pivot_table(index="a", columns="b", values="c", aggfunc="sum")`
Count combinations	`pd.crosstab(df["a"], df["b"], normalize="index")`
Merge	`left.merge(right, on="key", how="left", validate="many_to_one")`
Stack tables	`pd.concat([df1, df2])`
Wide to long, long to wide	`df.melt(id_vars="a")`, `df.pivot(index=..., columns=..., values=...)`

Dates and Text (Chapters 12 and 13)

Task	Code
Date parts	`s.dt.year`, `s.dt.month`, `s.dt.day_name()`
Regroup by period	`df.set_index("date")["a"].resample("MS").sum()`
Rolling average	`s.rolling(7).mean()`
Change, earlier value	`s.pct_change()`, `s.shift(1)`

Task	Code
Contains, extract	`s.str.contains("x")`, `s.str.extract(r"#(\d+)")`

Charts (Chapter 14)

Task	Code
Start	`fig, ax = plt.subplots()`
Line, bars, histogram, scatter	`ax.plot(x, y)`, `ax.barh(x, y)`, `ax.hist(s, bins=20)`, `ax.scatter(x, y)`
Labels	`ax.set_title(...)`, `ax.set_xlabel(...)`, `ax.legend()`
Save	`fig.tight_layout()`, `fig.savefig("chart.png")`

Appendix C: Common Errors and Fixes

The errors analysts meet most often, what they mean, and how to fix them. The last line of a traceback names the error; start reading there.

Error message (last line)	Usual cause	Fix
`FileNotFoundError: ... orders.csv`	Running the program from a different folder	`cd` into the chapter folder, or check the path
`KeyError: 'Revenue'`	Column name misspelled or in different case	Print `df.columns` and copy the exact name
`ValueError: The truth value of a Series is ambiguous`	Using `and`/`or` between conditions	Use `&` and `|`, with parentheses around each condition
`TypeError: unsupported operand type(s) for *: 'str' and 'int'`	A numeric column was read as text	Clean it and convert with `astype` or `pd.to_numeric`
`ValueError: could not convert string to float: '$18.00'`	Symbols or spaces in numbers	Remove them with `str.replace`, then convert
`ValueError: Index contains duplicate entries, cannot reshape`	`pivot` with repeated index/column pairs	Use `pivot_table` with an `aggfunc`
`MergeError: Merge keys are not unique in right dataset`	`validate` found duplicated keys	Find the duplicates with `duplicated()`, and fix them

Error message (last line)	Usual cause	Fix
`ValueError: time data ... doesn't match format`	A date in an unexpected layout	Check the formats present; parse each explicitly
`SettingWithCopyWarning` (older pandas)	Changing a filtered piece of a DataFrame	Add `.copy()` after filtering, or use `df.loc[mask, "col"] = ...`
`ModuleNotFoundError: No module named 'pandas'`	Virtual environment not active	Activate it (Chapter 2), or install the package
`ImportError: Missing optional dependency 'openpyxl'`	Reading or writing Excel without openpyxl	`python -m pip install openpyxl`

Wrong Answers Without Errors

The worst problems give no error at all:

- **Row counts change unexpectedly** after a merge: duplicated keys (use `validate`).
- **Totals too high:** duplicated rows (`drop_duplicates`), or a many-to-many merge.
- **Categories split in two:** spaces or capitals (`str.strip().str.title()`).
- **Dates in the wrong month:** day and month swapped while parsing (parse explicitly).
- **Averages of averages:** weighting groups equally when they differ in size (Chapter 20).
- **Missing values silently skipped:** `mean()` ignores them; check `isna().sum()` first.

Appendix D: Glossary

A/B test. An experiment that randomly shows visitors one of two versions and compares the results (Chapters 17 and 23).

Aggregation. Combining many values into one, such as a sum, mean or count (Chapter 9).

Array. NumPy's table of numbers of one type, operated on all at once (Chapter 3).

Association. Two measures moving together; it doesn't by itself show that one causes the other (Chapter 25).

Bootstrap. Estimating uncertainty by resampling the data with replacement many times (Chapter 17).

Broadcasting. NumPy stretching a smaller array to match a larger one in arithmetic (Chapter 3).

Categorical type. A column type storing each distinct value once, with small codes per row (Chapters 5 and 27).

Confounder. A third factor that affects both things being compared, such as season (Chapter 25).

Correlation (r). A measure from -1 to +1 of how closely two measures move together in a straight line (Chapter 16).

CSV. Comma-separated values: a plain-text table format (Chapter 2).

DataFrame. pandas' table: columns of data sharing one index (Chapter 4).

Drawdown. The fall from the highest value so far (Chapter 24).

dtype. The data type of a column, such as `int64`, `float64`, `str` or `datetime64` (Chapter 4).

Index. The row labels of a Series or DataFrame (Chapter 4).

Interquartile range (IQR). The range of the middle half of the values, from the 25th to the 75th percentile (Chapter 16).

Join (merge). Combining two tables by matching key columns (Chapter 10).

Long and wide tables. One row per observation versus one column per period or category (Chapter 11).

Mask. An array or Series of True/False values used to select items (Chapters 3 and 6).

Median. The middle value when values are sorted (Chapter 16).

Net Promoter Score (NPS). Percentage of promoters (9 or 10) minus percentage of detractors (0 to 6) (Chapter 22).

Percentile. The value below which a given share of values falls; the 95th percentile has 95% below it (Chapter 3).

Permutation test. Estimating a p-value by shuffling group labels many times (Chapter 17).

Pivot table. A table that groups by one column along the rows and another along the columns (Chapter 9).

p-value. How often a difference at least as large would appear by chance alone if there were no real effect (Chapter 17).

Resampling (time series). Regrouping data into new time periods, such as days into months (Chapter 12).

Rolling average. The average of each value and a fixed number of values before it (Chapter 12).

Sample ratio mismatch. An experiment's groups differing in size more than chance allows, a sign of broken assignment (Chapter 23).

Seed. A starting number that makes "random" numbers repeatable (Chapter 2).

Series. pandas' labeled column of data (Chapter 4).

Simpson's paradox. A comparison that reverses when data is split into groups (Chapter 16).

Skew. Lopsidedness of a distribution; a long right tail means positive skew (Chapter 16).

SQL. The language of relational databases (Chapter 18).

Standard deviation. How far values typically are from their mean (Chapter 16).

Stopwords. Very common words such as "to" and "the", removed before counting words (Chapter 13).

Tidy data. One variable per column, one observation per row, one value per cell (Chapter 11).

Vectorized operation. An operation on a whole array or column at once, without a Python loop (Chapters 3 and 8).

Volatility. The standard deviation of returns, often annualized (Chapter 24).

Window function. A SQL calculation across related rows that keeps every row, such as `RANK() OVER (...)` (Chapter 18).

Appendix E: Sources for the Real-World Stories

The Real-World Story and Interesting Fact boxes use the following public sources, checked in October 2026.

Chapter	Story or fact	Source
1	Spreadsheet error in a 2010 economics paper	Herndon, Ash and Pollin, "Does High Public Debt Consistently Stifle Economic Growth?", PERI working paper, University of Massachusetts Amherst, 2013; Reinhart and Rogoff's response, 2013
3	NumPy's default random generator (PCG64)	NumPy documentation, "Random Generator"
4	Origins of pandas (2008, open-sourced 2009)	pandas project history; talks by Wes McKinney
5	15,841 COVID-19 cases left out of England's counts (2020)	Public Health England statement, 4 October 2020; reporting by The Guardian and The Register, 5 October 2020
5	Excel worksheet limits	Microsoft, "Excel specifications and limits"
7	Gene names converted to dates	Ziemann, Eren and El-Osta, "Gene name errors are widespread in the scientific literature", *Genome Biology*, 2016; Bruford et al., "Guidelines for human gene nomenclature", *Nature Genetics*, 2020

Chapter	Story or fact	Source
15	The Challenger O-ring chart	Report of the Presidential Commission on the Space Shuttle Challenger Accident, 1986; Edward Tufte, *Visual Explanations*, 1997
16	Berkeley graduate admissions, 1973	Bickel, Hammel and O'Connell, "Sex Bias in Graduate Admissions: Data from Berkeley", *Science*, 1975
16	Anscombe's quartet	F. J. Anscombe, "Graphs in Statistical Analysis", *The American Statistician*, 1973
17	The bootstrap	Bradley Efron, "Bootstrap Methods: Another Look at the Jackknife", *Annals of Statistics*, 1979
22	Net Promoter Score	Frederick Reichheld, "The One Number You Need to Grow", *Harvard Business Review*, December 2003; Keiningham et al., *Journal of Marketing*, 2007
25	PM2.5 guideline values	World Health Organization, *WHO Global Air Quality Guidelines*, 2021

Appendix F: The Data Generator

This program makes every dataset in the book. It is also in the book's code repository (github.com/aadhavanshines/data-analysis-with-python-hands-on) as `code/data/make_data.py`, together with the files it makes. To type it yourself, enter the parts below, in order, into one file called `make_data.py` in your `data` folder, and run it there, as Chapter 2 describes. With the library versions from Chapter 2, it produces exactly the files the book uses; the book's tests regenerate them on Python 3.12, 3.13, 3.14 and 3.15 and check that every file is identical.

Each function makes one dataset, with patterns built in on purpose, so your analyses have something real to find:

- `shop()`: products, customers and a year of orders for Larkspur Home, with holiday peaks, a spring garden season and larger orders when discounted; plus `orders_raw.csv`, 300 orders made messy on purpose. Four smaller functions do the work.
- `survey()`: satisfaction that rises from the Basic to the Pro plan, and comments that match the ratings.
- `sessions()`: a checkout experiment in which version B converts a little better.
- `prices()`: random-walk prices for three companies that move partly with a market index.
- `health()`: seasonal pollution, sensor outages, and clinic visits that follow pollution two days later.

- `budget()`: a household's income, rent, bills and everyday spending.

> **Tip:** Each part starts with a line at the left margin (`def`, `import` or `if`). Every line of a function is indented at least four spaces, including the lines that continue on the next page. If Python reports an `IndentationError`, compare the line it names with the line above it in the book.

Setup

The fixed seed, 2026, is what makes every run identical.

```
"""Make every dataset used in this book. All data is fictional.

The random generator has a fixed seed, so every run, on every computer,
makes exactly the same files.
"""

import numpy as np
import pandas as pd

rng = np.random.default_rng(2026)
DAYS_2025 = pd.date_range("2025-01-01", "2025-12-31", freq="D")
```

The Shop

```
def shop_products():
    """The shop's 15 products: a catalog of names and list prices,
    and a unit cost for each."""
    catalog = {
        "Kitchen": [
            ("Chef Knife", 45.0),
            ("Cast Iron Pan", 60.0),
            ("Mug Set", 24.0),
        ],
        "Bedding": [
            ("Duvet Cover", 70.0),
            ("Pillow Pair", 35.0),
            ("Throw Blanket", 50.0),
        ],
        "Lighting": [
            ("Desk Lamp", 40.0),
            ("Floor Lamp", 120.0),
            ("String Lights", 18.0),
        ],
        "Decor": [
            ("Wall Clock", 32.0),
            ("Vase", 28.0),
            ("Picture Frame", 15.0),
        ],
```

```
        "Garden": [
            ("Planter", 22.0),
            ("Hose Reel", 55.0),
            ("Seed Kit", 12.0),
        ],
    }
    products = pd.DataFrame(
        [
            (name, cat, price, round(price * rng.uniform(0.45, 0.6), 2))
            for cat, items in catalog.items()
            for name, price in items
        ],
        columns=["product", "category", "list_price", "unit_cost"],
    )
    return catalog, products
```

```
def shop_customers():
    """1,200 customers, each with a signup date, region and segment."""
    regions = ["North", "South", "East", "West"]
    customers = pd.DataFrame(
        {
            "customer_id": [f"C{i:04d}" for i in range(1, 1201)],
            "signup_date": pd.to_datetime("2023-01-01")
            + pd.to_timedelta(rng.integers(0, 1000, 1200), unit="D"),
            "region": rng.choice(regions, 1200, p=[0.3, 0.25, 0.25, 0.2]),
            "segment": rng.choice(
                ["Consumer", "Business"], 1200, p=[0.85, 0.15]
            ),
            "birth_year": rng.integers(1955, 2006, 1200),
        }
    )
    return customers
```

```
def shop_orders(catalog, customers):
    """A year of orders, with holiday peaks, a spring garden season
    and larger orders when discounted."""
    # Busier in November and December; garden goods sell in spring.
    month_weight = np.array(
        [0.8, 0.75, 0.9, 1.0, 1.05, 1.0, 0.95, 0.95, 1.0, 1.1, 1.6, 1.9]
    )
    rows = []
    for day in DAYS_2025:
        n = rng.poisson(
            14
            * month_weight[day.month - 1]
            * (1.15 if day.dayofweek >= 5 else 1.0)
        )
        for _ in range(n):
            cust = customers.iloc[rng.integers(0, 1200)]
            garden = 2.5 if day.month in (4, 5, 6) else 0.6
            cat_p = np.array([1.0, 1.0, 0.9, 0.8, garden])
            cat = rng.choice(list(catalog), p=cat_p / cat_p.sum())
            name, price = catalog[cat][rng.integers(0, 3)]
            discount = rng.choice([0.0, 0.1, 0.2], p=[0.7, 0.2, 0.1])
            qty = 1 + rng.poisson(
                0.4
                + discount * 2
```

```
                + (1.0 if cust.segment == "Business" else 0)
            )
            rows.append(
                (
                    day,
                    cust.customer_id,
                    cust.region,
                    rng.choice(
                        ["web", "app", "marketplace"], p=[0.5, 0.3, 0.2]
                    ),
                    cat,
                    name,
                    int(qty),
                    price,
                    discount,
                )
            )
    orders = pd.DataFrame(
        rows,
        columns=[
            "order_date",
            "customer_id",
            "region",
            "channel",
            "category",
            "product",
            "quantity",
            "unit_price",
            "discount",
        ],
    )
    orders.insert(
        0, "order_id", [f"L{i:05d}" for i in range(1, len(orders) + 1)]
    )
    return orders
```

```
def messy_copy(orders):
    """300 of the orders, exported badly on purpose (Chapter 7)."""
    raw = orders.sample(300, random_state=7).sort_values("order_id").copy()
    raw["region"] = [
        r.lower() if i % 4 == 0 else r + " " if i % 7 == 0 else r
        for i, r in enumerate(raw["region"])
    ]
    raw["unit_price"] = [f"${p:.2f}" for p in raw["unit_price"]]
    raw["order_date"] = [
        d.strftime("%d/%m/%Y" if i % 5 == 0 else "%Y-%m-%d")
        for i, d in enumerate(raw["order_date"])
    ]
    raw = raw.astype({"quantity": "object"})
    raw.loc[raw.index[::23], "quantity"] = None
    raw.loc[raw.index[50], "quantity"] = 400
    raw = pd.concat([raw, raw.iloc[[10, 20, 30]]]).sort_values("order_id")
    return raw
```

```
def shop():
    """Larkspur Home, an online homeware shop: products, customers,
    orders, and a deliberately messy export of some orders."""
    catalog, products = shop_products()
    customers = shop_customers()
    orders = shop_orders(catalog, customers)
    return products, customers, orders, messy_copy(orders)
```

The Survey and the Experiment

```
def survey():
    """A customer survey: ratings, a 0-10 'would you recommend' score
    and a free-text comment."""
    n = 800
    plan = rng.choice(["Basic", "Plus", "Pro"], n, p=[0.5, 0.3, 0.2])
    base = np.select([plan == "Basic", plan == "Plus"], [3.2, 3.7], 4.0)
    sat = np.clip(np.round(base + rng.normal(0, 0.9, n)), 1, 5).astype(int)
    recommend = np.clip(
        np.round(sat * 2 + rng.normal(0, 1.3, n)), 0, 10
    ).astype(int)
    good = ["easy to use", "great support", "fast delivery", "good value"]
    bad = ["too slow", "price is high", "support was slow", "confusing app"]
    comments = [
        rng.choice(good)
        if s >= 4
        else rng.choice(bad)
        if s <= 2
        else rng.choice(good + bad + [""])
        for s in sat
    ]
    return pd.DataFrame(
        {
            "respondent": range(1, n + 1),
            "date": rng.choice(
                pd.date_range("2025-09-01", "2025-09-30"), n
            ),
            "age_group": rng.choice(["18-29", "30-44", "45-59", "60+"], n),
            "plan": plan,
            "satisfaction": sat,
            "recommend": recommend,
            "comment": comments,
        }
    ).sort_values(["date", "respondent"])
```

```
def sessions():
    """An A/B test of a new checkout page: B converts a little better."""
    n = 20_000
    variant = rng.choice(["A", "B"], n)
    device = rng.choice(["desktop", "mobile"], n, p=[0.45, 0.55])
    rate = np.where(variant == "B", 0.046, 0.040) * np.where(
        device == "mobile", 0.85, 1.2
    )
    converted = rng.random(n) < rate
    revenue = np.where(converted, np.round(rng.gamma(3, 22, n), 2), 0.0)
```

```
    return pd.DataFrame(
        {
            "session": range(1, n + 1),
            "date": rng.choice(
                pd.date_range("2025-10-01", "2025-10-28"), n
            ),
            "variant": variant,
            "device": device,
            "pages": 1 + rng.poisson(np.where(converted, 6, 3)),
            "converted": converted.astype(int),
            "revenue": revenue,
        }
    )
```

Prices, Health and the Budget

```
def prices():
    """Daily closing prices for three fictional companies and an index."""
    days = pd.bdate_range("2024-01-01", "2025-12-31")
    market = rng.normal(0.0003, 0.009, len(days))
    out = {"date": days}
    for ticker, beta, vol, start in [
        ("ALDR", 0.8, 0.010, 52.0),
        ("BRKN", 1.4, 0.018, 31.0),
        ("CDLX", 0.5, 0.007, 120.0),
    ]:
        daily = beta * market + rng.normal(0.0002, vol, len(days))
        out[ticker] = np.round(start * np.exp(np.cumsum(daily)), 2)
    out["INDEX"] = np.round(1000 * np.exp(np.cumsum(market)), 2)
    return pd.DataFrame(out)
```

```
def health():
    """Daily fine-particle pollution (PM2.5) and clinic visits for
    breathing problems in three cities; visits follow pollution with a
    delay of about two days."""
    air, visits = [], []
    for city, level, size in [
        ("Avonford", 18, 1.0),
        ("Brightwater", 30, 1.6),
        ("Cedarvale", 12, 0.7),
    ]:
        doy = DAYS_2025.dayofyear.to_numpy()
        season = 1 + 0.45 * np.cos((doy - 15) / 365 * 2 * np.pi)
        pm = np.clip(
            level * season * rng.lognormal(0, 0.3, len(DAYS_2025)), 2, None
        )
        pm[rng.random(len(pm)) < 0.02] = np.nan  # sensor outages
        lagged = pd.Series(pm).ffill().shift(2).bfill().to_numpy()
        weekday = np.where(DAYS_2025.dayofweek == 6, 0.6, 1.0)
        mean = size * (8 + 0.35 * lagged) * weekday
        air += zip(DAYS_2025, [city] * len(pm), np.round(pm, 1))
        visits += zip(DAYS_2025, [city] * len(pm), rng.poisson(mean))
    return (
        pd.DataFrame(air, columns=["date", "city", "pm25"]),
        pd.DataFrame(visits, columns=["date", "city", "visits"]),
```

```
    )
```

```
def budget():
    """One household's bank transactions for 2025."""
    rows = []
    for day in DAYS_2025:
        if day.day == 1:
            rows += [
                (day, "SALARY BRIGHTCO LTD", 4200.00),
                (day, "RENT MAPLE HOUSING", -1450.00),
            ]
        if day.day == 15:
            rows.append(
                (
                    day,
                    "CITY POWER & WATER",
                    -round(90 + 40 * (day.month in (1, 2, 12, 7, 8)), 2),
                )
            )
        if day.dayofweek == 5:
            rows.append(
                (
                    day,
                    f"FRESHMART #{rng.integers(10, 40)}",
                    -round(rng.uniform(60, 140), 2),
                )
            )
        if rng.random() < 0.25:
            rows.append(
                (
                    day,
                    rng.choice(["CAFE LUMA", "BEAN & LEAF CAFE"]),
                    -round(rng.uniform(3, 9), 2),
                )
            )
        if rng.random() < 0.08:
            rows.append(
                (
                    day,
                    rng.choice(["METRO TRANSIT", "RIDESHARE TRIP"]),
                    -round(rng.uniform(2.5, 25), 2),
                )
            )
        if rng.random() < 0.05:
            rows.append(
                (day, "ONLINE STORE ORDER", -round(rng.uniform(15, 180), 2))
            )
        if day.day == 20:
            rows.append((day, "STREAMFLIX SUBSCRIPTION", -12.99))
    return pd.DataFrame(rows, columns=["date", "description", "amount"])
```

Writing the Files

```
if __name__ == "__main__":
    products, customers, orders, raw = shop()
    products.to_csv("products.csv", index=False)
    customers.to_csv("customers.csv", index=False)
    orders.to_csv("orders.csv", index=False)
    raw.to_csv("orders_raw.csv", index=False)
    survey().to_csv("survey.csv", index=False)
    sessions().to_csv("sessions.csv", index=False)
    prices().to_csv("prices.csv", index=False)
    air, visits = health()
    air.to_csv("air_quality.csv", index=False)
    visits.to_csv("clinic_visits.csv", index=False)
    budget().to_csv("budget.csv", index=False)
    for name, table in [
        ("orders", orders),
        ("customers", customers),
        ("products", products),
        ("orders_raw", raw),
    ]:
        print(f"{name + '.csv':<18} {len(table):>6,} rows")
    print("survey.csv, sessions.csv, prices.csv, air_quality.csv,")
    print("clinic_visits.csv and budget.csv written too")
```

About the Author

Aadhavan Muthurengan is an automation and artificial intelligence professional with more than 15 years of experience in the automotive industry.

Throughout a career focused on product development and validation, Aadhavan has designed and built a wide range of in-house tools that automate complex engineering workflows, speed up testing, and help teams deliver reliable, high-quality results. Much of that work starts with data: test logs, measurements and reports that have to be cleaned, combined and turned into clear answers. This book teaches the same habits that work demands: check the data, test the code, and say what you don't know.

Thank you for reading *Data Analysis with Python, Hands-On*. May your data be clean and your questions sharp.

More Books by the Author

If you enjoyed this book, you may like these, also by Aadhavan Muthurengan. All are available on Amazon in paperback and Kindle editions.



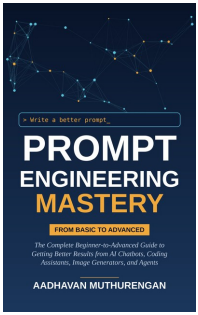
Python Programming Hands-On: Learn Python from Basic to Advanced by Writing Real Programs, with Exercises, Solutions and Every Example Tested. The natural companion to this book. Python from your first line of code to tested, packaged programs, a database app, a web API and automation scripts, with every example run before publication.



Vibe Coding for Beginners: Build Real Apps with Claude Code, from Your First Prompt to Tested, Deployed Software. Build real web and mobile apps by working with an AI coding assistant, from the first prompt to tested, deployed software.



Building AI Agents with Claude: A Beginner's Guide to Agents That Use Tools, Remember, and Get Real Work Done, from Your First Agent to Production. Build AI agents that use tools, remember context and complete real tasks, from a first agent to production.



Prompt Engineering Mastery: The Complete Beginner-to-Advanced Guide to Getting Better Results from AI Chatbots, Coding Assistants, Image Generators, and Agents. Practical techniques for getting reliable, high-quality results from AI tools, from everyday chat to coding assistants and agents.

Python Data Structures and Algorithms: A Comprehensive Guide. A guide to the data structures and algorithms behind efficient Python programs.